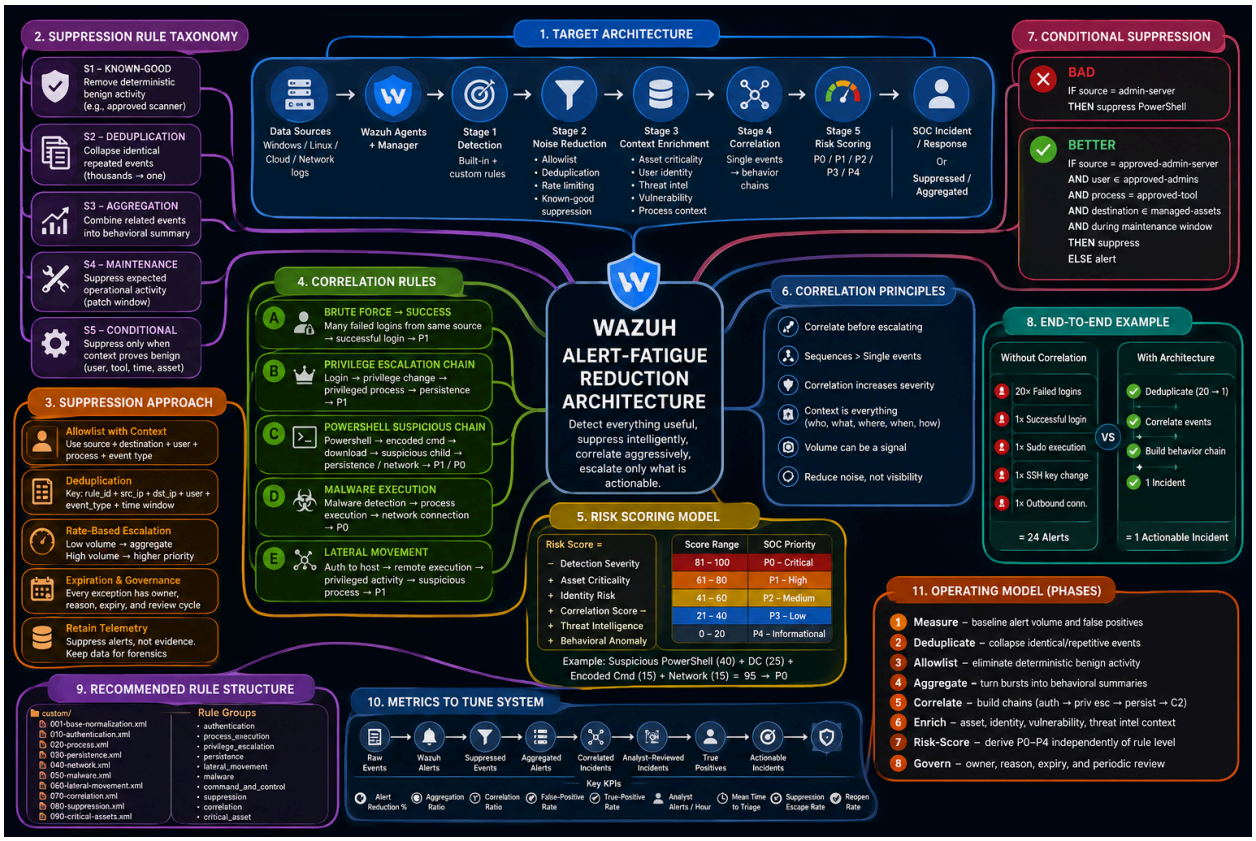
Taxonomy and Architecture

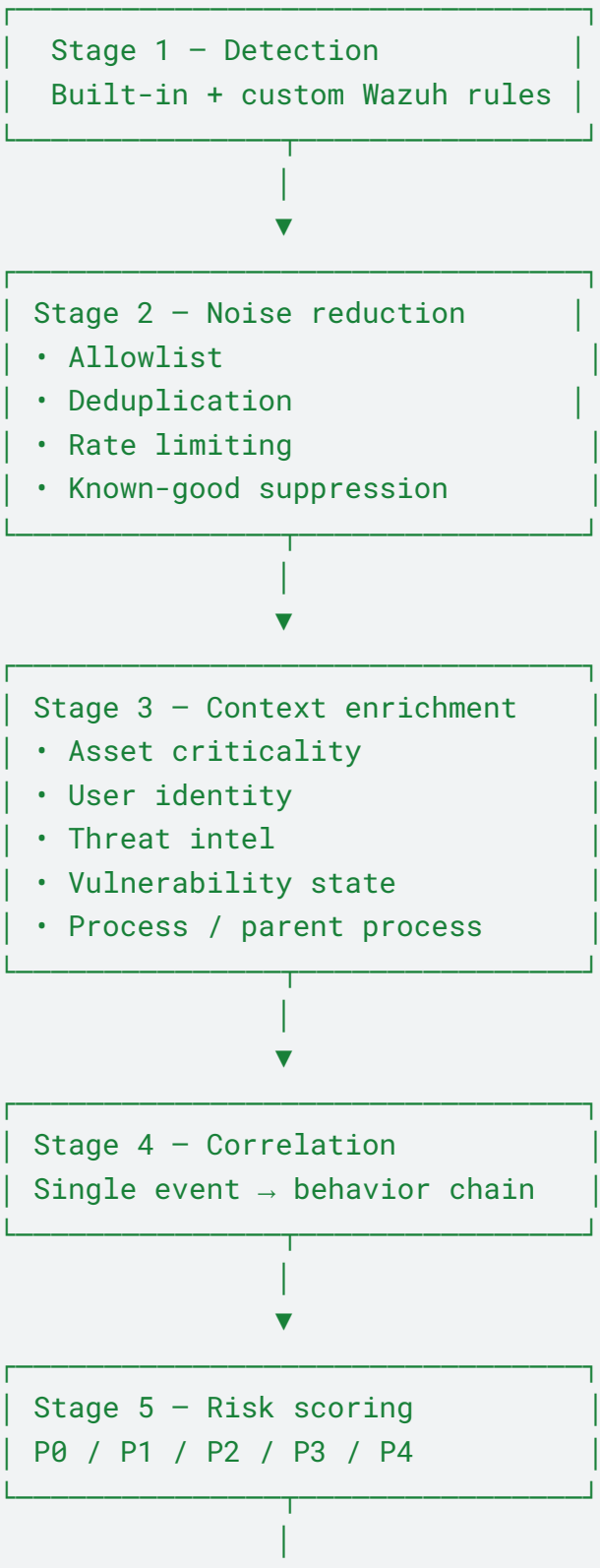


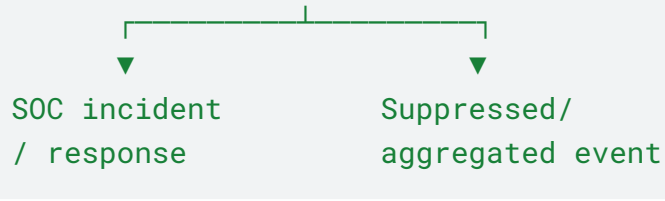
Yes. For Wazuh, I would implement alert-fatigue reduction as a **multi-stage detection pipeline** rather than trying to suppress noisy rules individually.

1. Target architecture

None

```
┌──────────────────────────────┐
│ Windows / Linux /            │
│ Cloud / Network logs         │
└──────────────────────────────┘
               │
               ▼
┌──────────────────────────────┐
│   Wazuh agents               │
│   + Wazuh manager            │
└──────────────────────────────┘
               │
               ▼
```


Stage 1 – Detection
Built-in + custom Wazuh rules
Stage 2 – Noise reduction
• Allowlist
• Deduplication
• Rate limiting
• Known-good suppression
Stage 3 – Context enrichment
• Asset criticality
• User identity
• Threat intel
• Vulnerability state
• Process / parent process
Stage 4 – Correlation
Single event → behavior chain
Stage 5 – Risk scoring
P0 / P1 / P2 / P3 / P4



The critical design principle is:

> **Suppress events, not evidence.**

You want to reduce what analysts see while retaining enough telemetry to reconstruct what happened.

---

2. Separate detection from suppression

Do **not** make your first response to noisy alerts:

```
None
if noisy → disable Wazuh rule
```

Instead:

```
None
raw event
   ↓
Wazuh detection
   ↓
classification
   ↓
suppression / aggregation
   ↓
correlation
   ↓
risk decision
```

This preserves the detection capability while controlling its operational impact.

---

3. Suppression-rule taxonomy

I recommend five suppression classes.

Suppression	Purpose	Example
S1 — Known-good	Remove deterministic benign activity	Approved vulnerability scanner
S2 — Deduplication	Collapse identical events	20,000 failed logins
S3 — Aggregation	Combine related events	Multiple failed logins from same source
S4 — Maintenance	Suppress expected operational activity	Patch window
S5 — Conditional	Suppress only when context proves benign	Admin PowerShell from approved jump host

S1 — Known-good

Example concept:

```
None
IF
    source.ip = approved_scanner
AND event.category = vulnerability_scan
THEN
    suppress
    retain telemetry
    increment counter
```

Do **not** broadly suppress based only on IP if that IP can also perform normal user activity.

Prefer:

```
None
source + destination + executable + account + event_type
```

as the suppression key.

---

4. Deduplication

Consider:

```
None
08:00:01 SSH failed login
08:00:02 SSH failed login
08:00:02 SSH failed login
...
08:03:47 SSH failed login
```

Instead of producing 2,000 alerts:

```
None
SSH brute-force activity
source=10.10.20.50
target=server01
failures=2000
window=3m47s
```

The underlying events remain available, but the SOC gets **one actionable object**.

A useful deduplication key is:

```
None
hash(
    rule_id,
    source_ip,
    destination_ip,
```

```
    username,
    event_type
)
```

with a time window.

For example:

```
None
WINDOW = 5 minutes
KEY   = rule + src + dst + user
ACTION:
    first event → create alert
    subsequent events → aggregate
    threshold exceeded → update severity
```

---

5. Correlation rules

This is where the largest reduction in analyst workload usually occurs.

Instead of asking:

> "Is this individual event suspicious?"

ask:

> "Does this sequence constitute suspicious behavior?"

Correlation rule A — Brute force → successful authentication

```
None
RULE BRUTE_FORCE_SUCCESS

IF
```

```
    failed_authentication >= 10
    from same source
    against same account/host
    within 10 minutes

AND

    successful_authentication
    from same source
    within 5 minutes

THEN
    create P1 incident
```

Rather than:

```
None
10 × failed-login alerts
+
1 × successful-login alert
```

produce:

```
None
1 × suspected account compromise
```

---

6. Correlation rule B — Privilege escalation chain

```
None
RULE PRIV_ESC_CHAIN
```

```
IF
    successful_login
        ↓
    privilege_change
        ↓
    privileged process execution
        ↓
    persistence/configuration change

within 15 minutes

THEN
    P1
```

The individual events might be relatively noisy.

The **sequence** is not.

---

7. Correlation rule C — PowerShell execution chain

For Windows environments:

```
None
RULE POWERSHELL_SUSPICIOUS_CHAIN

IF
    PowerShell execution

AND one or more:
    encoded command
    ↓
    download
```

```
    ↓
    suspicious child process
    ↓
    persistence
    ↓
    outbound connection

THEN
    P1/P0 depending on confidence
```

A normal PowerShell event might be P3.

PowerShell + encoded command + network connection + persistence is a very different detection.

---

8. Correlation rule D — Malware execution

```
None
RULE MALWARE_EXECUTION

IF
    malware detection

AND
    process execution

AND
    outbound network connection

THEN
    P0
```

Again, the goal is to turn multiple telemetry events into a single **incident narrative**.

---

9. Correlation rule E — Lateral movement

```
None
RULE LATERAL_MOVEMENT

IF

Host A
    │
    ├── authentication to Host B
    │
    ├── remote service execution
    │
    └── privileged account activity

AND

Host B
    └── suspicious process execution

within 10 minutes

THEN
    P1
```

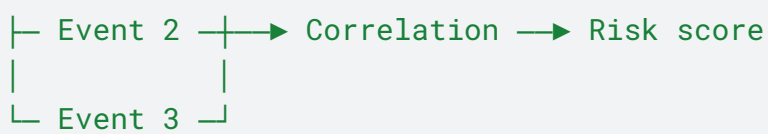
This is much more useful than treating each authentication and process event independently.

---

10. Correlation should increase severity

A particularly useful model is:

```
None
                ┌─ Event 1 ─┐
                │           │
```



For example:

Event	Individual priority
Failed authentication	P3
Successful authentication	P3
New admin privilege	P2
Suspicious process	P2
All four correlated	**P1**

This prevents analysts from being overwhelmed by low-value individual events.

---

11. Suppression should have expiration

Avoid permanent exceptions such as:

```
None
source.ip = 10.10.10.10 → ignore forever
```

Instead:

```
None
exception_id: EX-0042
reason: approved vulnerability scanner
owner: Security Engineering
created: 2026-08-24
expires: 2026-09-24
```

```
scope:
    scanner → production servers
events:
    vulnerability_scan
review: monthly
```

This is important because infrastructure changes.

A trusted scanner today can become a compromised host tomorrow.

---

12. Conditional suppression

This is safer than simple allowlisting.

Bad

```
None
IF source = admin-server
THEN suppress PowerShell
```

Better

```
None
IF
    source = approved-admin-server
AND
    user ∈ approved-admins
AND
    process = approved-management-tool
AND
    destination ∈ managed-assets
AND
    activity occurs during maintenance window
```

```
THEN
    suppress
ELSE
    alert
```

The more contextual the exception, the less likely it is to hide a real attack.

---

13. Rate-based escalation

Suppression and escalation should work together.

For example:

```
None
0–5 events / 5 min
    → aggregate

6–20
    → P3

21–100
    → P2

>100
    → P1

successful authentication after failures
    → immediate P1
```

So a noisy event does not necessarily disappear.

Instead, **volume becomes a detection signal**.

---

14. Risk scoring

I would put a risk layer above Wazuh's native rule severity.

For example:

```
None
Risk =
    Detection Severity
  + Asset Criticality
  + Identity Risk
  + Correlation Score
  + Threat Intelligence
  + Behavioral Anomaly
```

Normalize this to:

Score	SOC priority
0–20	P4
21–40	P3
41–60	P2
61–80	P1
81–100	P0

For example:

```
None
Suspicious PowerShell
severity = 40

+
Domain Controller
asset = +25

+
```

```
Encoded command
confidence = +15

+
External network connection
correlation = +15

--------------------------------
Risk = 95 → P0
```

The exact weights should be tuned using your organization's incident history rather than treated as universal values.

---

15. Recommended Wazuh rule structure

Keep custom rules organized by function rather than putting everything into one large rule file.

```
None
custom/
├── 001-base-normalization.xml
├── 010-authentication.xml
├── 020-process.xml
├── 030-persistence.xml
├── 040-network.xml
├── 050-malware.xml
├── 060-lateral-movement.xml
├── 070-correlation.xml
├── 080-suppression.xml
└── 090-critical-assets.xml
```

Use rule groups such as:

```
None
authentication
process_execution
privilege_escalation
persistence
lateral_movement
malware
command_and_control
suppression
correlation
critical_asset
```

This makes tuning considerably easier than maintaining hundreds of unrelated rule IDs.

---

16. Example end-to-end scenario

Suppose an attacker performs:

```
None
20 failed SSH logins
        ↓
successful login
        ↓
sudo execution
        ↓
new SSH key
        ↓
outbound connection
```

Without correlation

The SOC may receive:

```
None
20 × authentication alerts
1 × successful authentication
1 × sudo alert
1 × SSH-key modification
1 × network alert
```

24 alerts.

With the proposed architecture

```
None
20 failed logins
      ↓
deduplicate
      ↓
1 brute-force aggregate
      ↓
successful login
      ↓
correlate
      ↓
privilege escalation
      ↓
persistence
      ↓
C2/network activity
      ↓
1 incident
```

Final SOC object:

```
None
P0 – Suspected host compromise

Initial access:
    SSH brute force
```

```
Account:
    compromised-user

Privilege escalation:
    sudo

Persistence:
    unauthorized SSH key

C2:
    outbound connection

Evidence:
    24 underlying events
```

The **evidence count increases**, while the **analyst alert count decreases**.

That's the fundamental objective.

---

17. Metrics to tune the system

Track these separately:

```
None
Raw events
     ↓
Wazuh alerts
     ↓
Suppressed events
     ↓
Aggregated alerts
     ↓
Correlated incidents
     ↓
```

```
Analyst-reviewed incidents
      ↓
True positives
      ↓
Actionable incidents
```

The most important KPIs are:

KPI	What it tells you
Alert reduction %	How much noise you removed
Aggregation ratio	How effectively you collapse duplicates
Correlation ratio	How many alerts become incidents
False-positive rate	Quality of detections
True-positive rate	Detection effectiveness
Analyst alerts/hour	Human workload
Mean time to triage	Operational efficiency
Suppression escape rate	Whether suppression is hiding threats
Reopen rate	Whether tuning is too aggressive

One particularly important metric

Suppression escape rate:

```
None
suppressed events later associated with confirmed incidents
————————————————————————————————————————————————————
total suppressed events
```

If this starts increasing, your suppression rules are becoming dangerous.

Recommended operating model

I would implement the architecture in this order:

Phase 1 — Measure
→ baseline Wazuh alert volume and false positives.

Phase 2 — Deduplicate
→ collapse identical/repetitive events.

Phase 3 — Allowlist
→ eliminate deterministic, documented benign activity.

Phase 4 — Aggregate
→ turn event bursts into behavioral summaries.

Phase 5 — Correlate
→ authentication → privilege escalation → persistence → C2, etc.

Phase 6 — Enrich
→ asset, identity, vulnerability and threat-intelligence context.

Phase 7 — Risk-score
→ derive P0–P4 independently of raw Wazuh rule level.

Phase 8 — Govern
→ owner, reason, expiration and periodic review for every suppression.

The resulting architecture is essentially **“detect everything useful, suppress intelligently, correlate aggressively, and escalate only behavior that is materially actionable.”**

001-base-normalization.xml

Yes. One important correction before the file: **`001-base-normalization.xml` should not try to "normalize" fields that Wazuh's decoders have not extracted.** In Wazuh, decoders perform parsing; rules can then classify/group decoded events, add metadata, and provide a stable parent layer for downstream correlation. Wazuh explicitly supports `decoded_as`, `field`, `if_group`, `group`, `mitre`, and level-0 grouping rules for this purpose. (Wazuh Documentation)

For a production deployment, I would therefore make `001-base-normalization.xml` a **low-noise classification layer**. Use rule IDs in the `100000–120000` range for custom rules, as recommended by Wazuh. (Wazuh Documentation)

001-base-normalization.xml

XML

```
<!--
  WAZUH CUSTOM RULES
  File: 001-base-normalization.xml

  Purpose:
    Base classification / normalization layer for downstream
    correlation, suppression and risk scoring.

  Design principles:
    1. Do not generate analyst alerts from this layer.
    2. Use level="0" for classification/grouping.
    3. Preserve the original Wazuh event and decoder output.
    4. Add stable internal groups that downstream rules can
consume.
    5. Do not overwrite vendor/default Wazuh rules here.
    6. Keep environment-specific allowlists out of this file.

  Custom rule ID range:
    100000-120000

  IMPORTANT:
    Decoder names and decoded field names vary by installed Wazuh
    ruleset and integrations. Verify them with wazuh-logtest before
    deploying environment-specific rules.
```

```
-->

<var name="BASE_NORMALIZATION_VERSION">1.0</var>


<!--
=============================================================
     GENERIC EVENT CLASSIFICATION

=============================================================
-->

<group name="custom_base_normalization,">

  <!--
-------------------------------------------------------------
       100000 - Generic syslog classification

------------------------------------------------------------- -->

  <rule id="100000" level="0">
    <category>syslog</category>
    <description>BASE: Syslog event normalized</description>
    <group>base_normalized,source_syslog,</group>
  </rule>


  <!--
-------------------------------------------------------------
       100001 - Generic Windows classification

------------------------------------------------------------- -->

  <rule id="100001" level="0">
    <category>windows</category>
    <description>BASE: Windows event normalized</description>
```

```
    <group>base_normalized,source_windows,</group>
  </rule>


  <!--
------------------------------------------------------------------
       100002 - Generic JSON classification

------------------------------------------------------------------ -->

  <rule id="100002" level="0">
    <decoded_as>json</decoded_as>
    <description>BASE: JSON event normalized</description>
    <group>base_normalized,source_json,</group>
  </rule>


  <!--
------------------------------------------------------------------
       100003 - SSH classification

------------------------------------------------------------------ -->

  <rule id="100003" level="0">
    <decoded_as>sshd</decoded_as>
    <description>BASE: SSH event normalized</description>

<group>base_normalized,authentication,remote_access,source_sshd,</g
roup>
  </rule>


  <!--
------------------------------------------------------------------
       100004 - Audit classification
```

```
------------------------------------------------------------ -->

  <rule id="100004" level="0">
    <decoded_as>auditd</decoded_as>
    <description>BASE: Linux audit event normalized</description>
    <group>base_normalized,audit,source_auditd,</group>
  </rule>


  <!--
------------------------------------------------------------
       100005 - Sysmon classification

------------------------------------------------------------ -->

  <rule id="100005" level="0">
    <decoded_as>sysmon</decoded_as>
    <description>BASE: Sysmon event normalized</description>
    <group>base_normalized,windows,sysmon,source_sysmon,</group>
  </rule>


  <!--
============================================================
       AUTHENTICATION CLASSIFICATION

============================================================
-->

  <!-- Successful authentication -->

  <rule id="100010" level="0">
    <if_group>authentication_success</if_group>
    <description>BASE: Successful authentication
normalized</description>
```

```
<group>base_normalized,authentication,authentication_success,</grou
p>
  </rule>


  <!-- Failed authentication -->

  <rule id="100011" level="0">
    <if_group>authentication_failed</if_group>
    <description>BASE: Failed authentication
normalized</description>

<group>base_normalized,authentication,authentication_failed,</group
>
  </rule>


  <!-- Invalid user -->

  <rule id="100012" level="0">
    <if_group>invalid_login</if_group>
    <description>BASE: Invalid-user authentication event
normalized</description>

<group>base_normalized,authentication,authentication_failed,invalid
_user,</group>
  </rule>


  <!-- Authentication success using root -->

  <rule id="100013" level="0">
    <if_group>authentication_success</if_group>
    <user>root</user>
```

```
    <description>BASE: Root authentication success
normalized</description>

<group>base_normalized,authentication,privileged_account,root_accou
nt,</group>
  </rule>


  <!--
============================================================
       NETWORK CLASSIFICATION

============================================================
-->

  <!-- Event contains source IP -->

  <rule id="100020" level="0">
    <srcip>0.0.0.0/0</srcip>
    <description>BASE: Event contains source IP</description>
    <group>base_normalized,network,has_source_ip,</group>
  </rule>


  <!-- Event contains destination IP -->

  <rule id="100021" level="0">
    <dstip>0.0.0.0/0</dstip>
    <description>BASE: Event contains destination IP</description>
    <group>base_normalized,network,has_destination_ip,</group>
  </rule>


  <!-- Source and destination both present -->

  <rule id="100022" level="0">
```

```
    <srcip>0.0.0.0/0</srcip>
    <dstip>0.0.0.0/0</dstip>
    <description>BASE: Network flow context
normalized</description>
    <group>base_normalized,network,network_flow,</group>
  </rule>


  <!--
=============================================================
       PRIVILEGED ACTIVITY

=============================================================
-->

  <rule id="100030" level="0">
    <if_group>authentication_success</if_group>
    <user>root|Administrator|administrator</user>
    <description>BASE: Privileged account authentication
normalized</description>

<group>base_normalized,authentication,privileged_account,</group>
  </rule>


  <!-- sudo-related event -->

  <rule id="100031" level="0">
    <match>sudo</match>
    <description>BASE: sudo activity normalized</description>

<group>base_normalized,privilege_escalation,privileged_activity,</g
roup>
  </rule>
```

```
  <!--
================================================================
       PROCESS / COMMAND EXECUTION

================================================================
-->

  <!-- PowerShell -->

  <rule id="100040" level="0">
    <field name="win.eventdata.image">(?i)powershell\.exe$</field>
    <description>BASE: PowerShell process execution
normalized</description>

<group>base_normalized,process_execution,windows,powershell,</group
>
  </rule>


  <!-- Windows Command Shell -->

  <rule id="100041" level="0">
    <field
name="win.eventdata.image">(?i)(cmd|command)\.exe$</field>
    <description>BASE: Windows command shell execution
normalized</description>

<group>base_normalized,process_execution,windows,command_shell,</gr
oup>
  </rule>


  <!-- WMI -->

  <rule id="100042" level="0">
```

```
    <field
name="win.eventdata.image">(?i)(wmic|wmiprvse)\.exe$</field>
    <description>BASE: WMI process activity
normalized</description>
    <group>base_normalized,process_execution,windows,wmi,</group>
  </rule>


  <!-- PowerShell encoded-command indicator -->

  <rule id="100043" level="0">
    <match
type="pcre2">(?i)(-enc|-encodedcommand)\s+[A-Za-z0-9+/=]{20,}</matc
h>
    <description>BASE: PowerShell encoded-command indicator
normalized</description>

<group>base_normalized,process_execution,powershell,encoded_command
,</group>
  </rule>


  <!--
================================================================
       PERSISTENCE / CONFIGURATION CHANGE

================================================================
-->

  <!-- SSH key modification -->

  <rule id="100050" level="0">
    <match
type="pcre2">(?i)(authorized_keys|authorized_keys2)</match>
    <description>BASE: SSH authorized-key activity
normalized</description>
```

```
    <group>base_normalized,persistence,ssh_key_activity,</group>
  </rule>


  <!-- Windows service creation -->

  <rule id="100051" level="0">
    <match
type="pcre2">(?i)(create\s+service|service\s+created|New-Service)</
match>
    <description>BASE: Windows service creation activity
normalized</description>
    <group>base_normalized,persistence,service_creation,</group>
  </rule>


  <!-- Scheduled task -->

  <rule id="100052" level="0">
    <match
type="pcre2">(?i)(schtasks|scheduled\s+task|task\s+created)</match>
    <description>BASE: Scheduled-task activity
normalized</description>
    <group>base_normalized,persistence,scheduled_task,</group>
  </rule>


  <!--
================================================================
       SECURITY TOOL / DEFENSE EVASION

================================================================
-->

  <!-- Windows Defender configuration -->
```

```
  <rule id="100060" level="0">
    <match
type="pcre2">(?i)(DisableAntiSpyware|DisableRealtimeMonitoring|Set-
MpPreference)</match>
    <description>BASE: Windows Defender configuration activity
normalized</description>

<group>base_normalized,defense_evasion,security_control_change,wind
ows_defender,</group>
  </rule>


  <!-- Firewall configuration -->

  <rule id="100061" level="0">
    <match
type="pcre2">(?i)(netsh\s+advfirewall|iptables|firewalld|ufw)</matc
h>
    <description>BASE: Firewall configuration activity
normalized</description>

<group>base_normalized,defense_evasion,security_control_change,fire
wall_change,</group>
  </rule>


  <!--
=================================================================
       MALWARE / THREAT INDICATORS

=================================================================
-->

  <rule id="100070" level="0">
    <if_group>malware</if_group>
    <description>BASE: Malware detection normalized</description>
```

```
    <group>base_normalized,malware,threat_detection,</group>
  </rule>


  <!--
=============================================================
       FILE INTEGRITY

=============================================================
-->

  <rule id="100080" level="0">
    <if_group>syscheck</if_group>
    <description>BASE: File-integrity event
normalized</description>
    <group>base_normalized,file_integrity,change_detection,</group>
  </rule>


  <!--
=============================================================
       VULNERABILITY / SECURITY POSTURE

=============================================================
-->

  <rule id="100090" level="0">
    <if_group>vulnerability-detector</if_group>
    <description>BASE: Vulnerability event normalized</description>
    <group>base_normalized,vulnerability,security_posture,</group>
  </rule>


  <!--
=============================================================
       SOURCE / DESTINATION CONTEXT
```

```
================================================================
-->

  <!-- External source indicator.
       This is intentionally only a classification marker.
       Do NOT treat it as malicious by itself. -->

  <rule id="100100" level="0">
    <srcip>0.0.0.0/0</srcip>
    <description>BASE: Source network context
available</description>
    <group>base_normalized,network,source_context,</group>
  </rule>


  <!--
================================================================
       ERROR / OPERATIONAL NOISE CLASSIFICATION

================================================================
-->

  <rule id="100110" level="0">
    <match
type="pcre2">(?i)(connection\s+reset|connection\s+closed|timeout|te
mporary\s+failure)</match>
    <description>BASE: Common operational event
normalized</description>
    <group>base_normalized,operational,low_signal,</group>
  </rule>


  <!--
================================================================
       AGGREGATION / CORRELATION ANCHORS
```

```
================================================================
-->

  <!-- Authentication event anchor -->

  <rule id="100120" level="0">
    <if_group>authentication</if_group>
    <description>BASE: Authentication correlation
anchor</description>
    <group>base_normalized,correlation_authentication,</group>
  </rule>


  <!-- Process event anchor -->

  <rule id="100121" level="0">
    <if_group>process_execution</if_group>
    <description>BASE: Process correlation anchor</description>
    <group>base_normalized,correlation_process,</group>
  </rule>


  <!-- Persistence event anchor -->

  <rule id="100122" level="0">
    <if_group>persistence</if_group>
    <description>BASE: Persistence correlation anchor</description>
    <group>base_normalized,correlation_persistence,</group>
  </rule>


  <!-- Network event anchor -->

  <rule id="100123" level="0">
    <if_group>network</if_group>
```

```
    <description>BASE: Network correlation anchor</description>
    <group>base_normalized,correlation_network,</group>
  </rule>


  <!--
================================================================
       CRITICAL SECURITY CONTEXT

================================================================
-->

  <!-- Domain-controller indicators -->

  <rule id="100130" level="0">
    <field name="win.system.channel">(?i)Security</field>
    <field
name="win.system.computer">(?i)(dc|domaincontroller)</field>
    <description>BASE: Potential domain-controller security event
normalized</description>

<group>base_normalized,critical_asset_candidate,domain_controller,<
/group>
  </rule>


  <!--
================================================================
       UNKNOWN / UNCLASSIFIED EVENT

================================================================

       This is deliberately NOT an alerting rule.

       It gives downstream reporting a way to identify events that
       reached Wazuh but were not classified by the base layer.
```

```
  -->

  <rule id="100140" level="0">
    <description>BASE: Event reached custom normalization
layer</description>
    <group>base_normalized,normalization_processed,</group>
  </rule>

</group>
```

One adjustment I strongly recommend

I would **not deploy every rule above unchanged**. In particular, the following are environment-dependent:

- `win.eventdata.image`
- `win.system.channel`
- `win.system.computer`
- `vulnerability-detector`
- `malware`
- exact `sshd`, `auditd`, and `sysmon` decoder names
- the fields exposed by your JSON integrations

Wazuh's `wazuh-logtest` should be used to verify the actual decoder and Phase 2 fields before enabling rules based on them. Wazuh's documentation specifically recommends testing custom rules with `wazuh-logtest`. (Wazuh Documentation)

Also, **level 0 is intentional** here. Wazuh documents level-0 rules as a way to group/classify events without displaying them as dashboard alerts; however, level-0 rules cannot be used as `if_matched_sid` correlation events, so the later correlation layer should reference actual alert-producing rules where frequency/timeframe correlation is required. (Wazuh Documentation)

How this fits your architecture

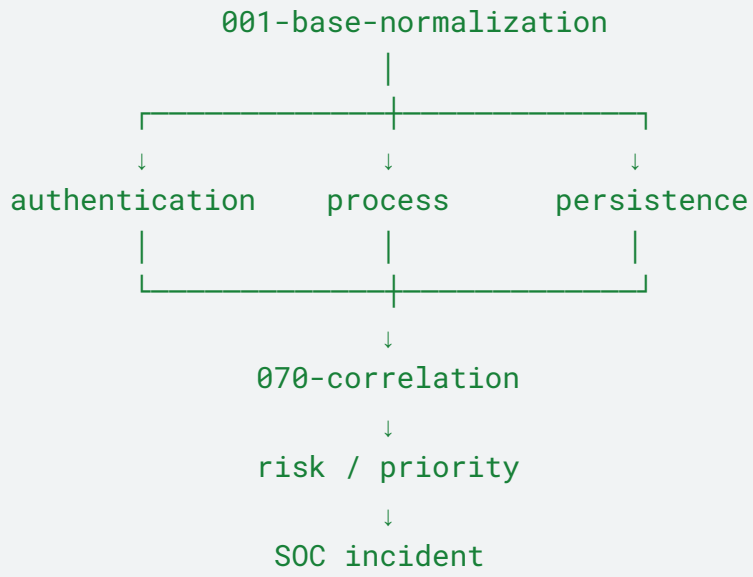
Your files would now become:

None

```
/var/ossec/etc/rules/
│
├── 001-base-normalization.xml     ← THIS FILE
│
├── 010-authentication.xml
├── 020-process.xml
├── 030-persistence.xml
├── 040-network.xml
├── 050-malware.xml
├── 060-lateral-movement.xml
├── 070-correlation.xml
├── 080-suppression.xml
├── 090-critical-assets.xml
└── ...
```

The important dependency is:

None



This also means the **next file I would build is `010-authentication.xml`**, because it can consume the normalized `authentication`, `authentication_success`, and `authentication_failed` groups and implement the brute-force → successful-login correlation without making the base layer noisy.

Wazuh custom rules documentation
Wazuh rules XML syntax

010-authentication.xml

Below is a production-oriented `010-authentication.xml` for the architecture we discussed. It is designed to **reduce authentication alert fatigue through aggregation and correlation**, while preserving the underlying events.

I'm using Wazuh custom rule IDs in the `100000–120000` range, which Wazuh recommends for custom rules to avoid collisions with built-in rules. (Wazuh Documentation)

One important design choice: the rules build on Wazuh's existing `authentication_failed` and `authentication_success` groups rather than duplicating SSH/Windows authentication detection. Wazuh's current ruleset already uses these groups, including SSH rule `5715` for successful authentication and `5710` for invalid SSH users. (Wazuh Documentation)

/var/ossec/etc/rules/010-authentication.xml

XML

```
<!--
    WAZUH CUSTOM AUTHENTICATION RULES
    File: /var/ossec/etc/rules/010-authentication.xml

    Purpose:
      - Reduce authentication alert fatigue
      - Aggregate repeated failures
      - Detect brute-force behavior
      - Detect password spraying
      - Detect successful authentication after failures
      - Detect suspicious off-hours authentication
      - Detect authentication from multiple source IPs
      - Provide normalized groups for downstream correlation

    IMPORTANT:
      These rules depend on the Wazuh default authentication
rules/decoders.

      Primary groups used:
        - authentication_failed
        - authentication_success
        - invalid_login
        - sshd
```

```
      Test with:
        /var/ossec/bin/wazuh-logtest

      Custom rule IDs:
        100100 - 100199
-->

<group name="custom_authentication,">

    <!-- ============================================================
         100100-100109
         BASE AUTHENTICATION CLASSIFICATION
         ============================================================
-->

    <!-- Normalize failed authentication events.
         noalert=1 means this rule is a classification/correlation
         anchor and does not create an additional analyst alert.
    -->
    <rule id="100100" level="0" noalert="1">
        <if_group>authentication_failed,</if_group>
        <description>AUTH: Authentication failure normalized for
correlation</description>
        <group>auth_failed_normalized,</group>
    </rule>


    <!-- Normalize successful authentication events. -->
    <rule id="100101" level="0" noalert="1">
        <if_group>authentication_success,</if_group>
        <description>AUTH: Authentication success normalized for
correlation</description>
        <group>auth_success_normalized,</group>
    </rule>
```

```
    <!-- ============================================================
         100110-100119
         REPEATED FAILED AUTHENTICATION
         ============================================================
-->

    <!--
        5 failed authentications from the same source within
        5 minutes.

        This is intentionally level 5: suspicious but not yet
        necessarily a brute-force incident.
    -->
    <rule id="100110"
          level="5"
          frequency="5"
          timeframe="300"
          ignore="60">

        <if_matched_group>authentication_failed,</if_matched_group>
        <same_srcip />

        <description>
            AUTH: Repeated authentication failures from the same
source
        </description>

        <mitre>
            <id>T1110</id>
        </mitre>

        <group>
            authentication,
            authentication_failed,
            brute_force,
```

```
            auth_aggregate,
        </group>
    </rule>


    <!--
        10 failures from the same source within 5 minutes.

        Escalates the event from "noise" to probable brute force.
    -->
    <rule id="100111"
          level="10"
          frequency="10"
          timeframe="300"
          ignore="300">

        <if_matched_group>authentication_failed,</if_matched_group>
        <same_srcip />

        <description>
            AUTH: Possible brute-force attack - 10 authentication
failures from the same source
        </description>

        <mitre>
            <id>T1110</id>
        </mitre>

        <group>
            authentication,
            authentication_failed,
            brute_force,
            attack,
        </group>
    </rule>
```

```
    <!--
        20 failures from same source in 10 minutes.

        Higher-confidence brute force.
    -->
    <rule id="100112"
          level="12"
          frequency="20"
          timeframe="600"
          ignore="600">

        <if_matched_group>authentication_failed,</if_matched_group>
        <same_srcip />

        <description>
            AUTH: High-confidence brute-force attack - 20
authentication failures from the same source
        </description>

        <mitre>
            <id>T1110</id>
        </mitre>

        <group>
            authentication,
            authentication_failed,
            brute_force,
            high_confidence,
        </group>
    </rule>


    <!-- ============================================================
         100120-100129
         PASSWORD SPRAYING
```

```
        ============================================================
-->

    <!--
        Same source attempting authentication against multiple
users.

        This detects the behavior characteristic of password
spraying:
            one source
            multiple accounts
            relatively few attempts per account
    -->
    <rule id="100120"
          level="9"
          frequency="8"
          timeframe="600"
          ignore="300">

        <if_matched_group>authentication_failed,</if_matched_group>
        <same_srcip />
        <different_user />

        <description>
            AUTH: Possible password spraying - same source
targeting multiple accounts
        </description>

        <mitre>
            <id>T1110.003</id>
        </mitre>

        <group>
            authentication,
            authentication_failed,
            password_spraying,
```

```
            credential_access,
        </group>
    </rule>


    <!--
        Stronger password-spray signal:
        same source + many different users.
    -->
    <rule id="100121"
          level="12"
          frequency="15"
          timeframe="900"
          ignore="600">

        <if_matched_group>authentication_failed,</if_matched_group>
        <same_srcip />
        <different_user />

        <description>
            AUTH: High-confidence password spraying - source
targeting many accounts
        </description>

        <mitre>
            <id>T1110.003</id>
        </mitre>

        <group>
            authentication,
            authentication_failed,
            password_spraying,
            credential_access,
            high_confidence,
        </group>
    </rule>
```

```
    <!-- ============================================================
         100130-100139
         SUCCESS AFTER FAILURE
         ============================================================
-->

    <!--
        Successful authentication after repeated failures.

        This is substantially more actionable than a failed-login
        event by itself.

        The same source IP is required.
    -->
    <rule id="100130"
          level="12"
          frequency="1"
          timeframe="300">

        <if_matched_group>authentication_failed,</if_matched_group>
        <same_srcip />

        <if_group>authentication_success,</if_group>

        <description>
            AUTH: Successful authentication following recent
authentication failures
        </description>

        <mitre>
            <id>T1078</id>
            <id>T1110</id>
        </mitre>
```

```
        <group>
            authentication,
            authentication_success,
            authentication_failed,
            possible_account_compromise,
            credential_access,
        </group>
    </rule>


    <!--
        Higher confidence version:
        successful authentication following multiple failures
        from the same source.
    -->
    <rule id="100131"
          level="14"
          frequency="5"
          timeframe="600"
          ignore="600">

        <if_matched_group>authentication_failed,</if_matched_group>
        <same_srcip />

        <if_group>authentication_success,</if_group>

        <description>
            AUTH: High-confidence possible account compromise -
successful authentication after repeated failures
        </description>

        <mitre>
            <id>T1078</id>
            <id>T1110</id>
        </mitre>
```

```
        <group>
            authentication,
            authentication_success,
            authentication_failed,
            possible_account_compromise,
            credential_access,
            high_confidence,
        </group>
    </rule>


    <!-- ============================================================
         100140-100149
         OFF-HOURS AUTHENTICATION
         ============================================================
-->

    <!--
        Successful authentication outside normal working hours.

        Adjust the time window for your organization.

        NOTE:
        Wazuh's <time> condition evaluates against Wazuh server
time,
        so ensure time configuration is correct.
    -->
    <rule id="100140" level="7">

        <if_group>authentication_success,</if_group>

        <time>22:00 - 05:59</time>

        <description>
            AUTH: Successful authentication during defined
off-hours
```

```
    </description>

    <mitre>
        <id>T1078</id>
    </mitre>

    <group>
        authentication,
        authentication_success,
        off_hours,
        anomalous_authentication,
    </group>
</rule>


<!-- Weekend authentication. -->
<rule id="100141" level="6">

    <if_group>authentication_success,</if_group>

    <weekday>weekends</weekday>

    <description>
        AUTH: Successful authentication during weekend
    </description>

    <mitre>
        <id>T1078</id>
    </mitre>

    <group>
        authentication,
        authentication_success,
        weekend_authentication,
        anomalous_authentication,
    </group>
```

```
    </rule>


    <!-- ============================================================
         100150-100159
         MULTIPLE SOURCE IPs
         ============================================================
-->

    <!--
        Same account successfully authenticating from different
        source IPs within a short period.

        Useful for detecting credential sharing, stolen
credentials,
        VPN anomalies, or account takeover.

        This rule requires the source IP to change.
    -->
    <rule id="100150"
          level="9"
          frequency="3"
          timeframe="900"
          ignore="300">


<if_matched_group>authentication_success,</if_matched_group>
        <same_user />
        <different_srcip />

        <description>
            AUTH: Same account successfully authenticated from
multiple source IPs
        </description>

        <mitre>
```

```
            <id>T1078</id>
        </mitre>

        <group>
            authentication,
            authentication_success,
            multiple_source_ips,
            anomalous_authentication,
            valid_accounts,
        </group>
    </rule>


    <!--
        Stronger version: same user, multiple source addresses,
        short time window.
    -->
    <rule id="100151"
          level="12"
          frequency="5"
          timeframe="600"
          ignore="600">


<if_matched_group>authentication_success,</if_matched_group>
        <same_user />
        <different_srcip />

        <description>
            AUTH: High-confidence anomalous authentication -
account used from multiple source IPs
        </description>

        <mitre>
            <id>T1078</id>
        </mitre>
```

```
        <group>
            authentication,
            authentication_success,
            multiple_source_ips,
            anomalous_authentication,
            valid_accounts,
            high_confidence,
        </group>
    </rule>


    <!-- ============================================================
         100160-100169
         SAME SOURCE / MULTIPLE TARGETS
         ============================================================
-->

    <!--
        One source authenticating against multiple destination
        hosts.

        Useful for identifying lateral movement.
    -->
    <rule id="100160"
          level="9"
          frequency="5"
          timeframe="600"
          ignore="300">


<if_matched_group>authentication_success,</if_matched_group>
        <same_srcip />
        <different_dstip />

        <description>
```

```
            AUTH: Same source successfully authenticated to
multiple hosts
        </description>

        <mitre>
            <id>T1078</id>
            <id>T1021</id>
        </mitre>

        <group>
            authentication,
            authentication_success,
            lateral_movement,
            multiple_targets,
            valid_accounts,
        </group>
    </rule>


    <!-- ============================================================
         100170-100179
         INVALID / NON-EXISTENT ACCOUNT ABUSE
         ============================================================
-->

    <!--
        Aggregate repeated invalid-user authentication attempts.

        Wazuh's SSH rule 5710 already identifies invalid users.
        This rule creates a higher-level aggregate.
    -->
    <rule id="100170"
          level="7"
          frequency="5"
          timeframe="300"
          ignore="120">
```

```
        <if_group>invalid_login,</if_group>
        <same_srcip />

        <description>
            AUTH: Repeated attempts against non-existent accounts
        </description>

        <mitre>
            <id>T1110.001</id>
        </mitre>

        <group>
            authentication,
            invalid_login,
            authentication_failed,
            brute_force,
            auth_aggregate,
        </group>
    </rule>


    <!-- High-confidence invalid-account brute force. -->
    <rule id="100171"
          level="11"
          frequency="10"
          timeframe="300"
          ignore="600">

        <if_group>invalid_login,</if_group>
        <same_srcip />

        <description>
            AUTH: High-confidence brute-force activity against
non-existent accounts
        </description>
```

```
        <mitre>
            <id>T1110.001</id>
        </mitre>

        <group>
            authentication,
            invalid_login,
            authentication_failed,
            brute_force,
            credential_access,
            high_confidence,
        </group>
    </rule>


    <!-- ============================================================
         100180-100189
         AUTHENTICATION FLOOD CONTROL
         ============================================================
-->

    <!--
        Very noisy authentication source.

        Instead of generating repeated analyst-facing alerts,
        this produces one aggregate alert and then suppresses
        duplicate aggregate notifications for 10 minutes.
    -->
    <rule id="100180"
          level="10"
          frequency="50"
          timeframe="600"
          ignore="600">

        <if_matched_group>authentication_failed,</if_matched_group>
```

```
        <same_srcip />

        <description>
            AUTH: Authentication event flood from a single source
        </description>

        <mitre>
            <id>T1110</id>
        </mitre>

        <group>
            authentication,
            authentication_failed,
            brute_force,
            alert_fatigue_control,
            aggregation,
        </group>
    </rule>


    <!-- ============================================================
         100190-100199
         CORRELATION ANCHORS FOR DOWNSTREAM RULES
         ============================================================
-->

    <!--
        These rules deliberately do not alert.

        They provide clean groups for later correlation files such
as:
          020-process.xml
          030-persistence.xml
          060-lateral-movement.xml
          070-correlation.xml
    -->
```

```
    <rule id="100190" level="0" noalert="1">
        <if_group>authentication_success,</if_group>
        <description>
            AUTH: Successful authentication correlation anchor
        </description>
        <group>auth_success_event,correlation_anchor,</group>
    </rule>


    <rule id="100191" level="0" noalert="1">
        <if_group>authentication_failed,</if_group>
        <description>
            AUTH: Failed authentication correlation anchor
        </description>
        <group>auth_failure_event,correlation_anchor,</group>
    </rule>


    <!--
        Authentication anomaly anchor.
    -->
    <rule id="100192" level="0" noalert="1">
        <if_group>anomalous_authentication,</if_group>
        <description>
            AUTH: Authentication anomaly correlation anchor
        </description>
        <group>auth_anomaly_event,correlation_anchor,</group>
    </rule>


    <!-- ============================================================
         END
         ============================================================
-->
```

```
</group>
```

Important correction to the correlation design

There is one subtle Wazuh rule-engine issue worth calling out. A rule such as:

```xml
<if_matched_group>authentication_failed,</if_matched_group>
<if_group>authentication_success,</if_group>
```

should **not** be treated as a general-purpose "failure happened, then success happened" state machine. `if_matched_group` checks that a matching alert occurred within the configured correlation period, while `if_group` evaluates the current event. The `same_srcip`, `same_user`, etc. dynamic filters are then used to constrain the correlation. Wazuh explicitly documents `if_matched_group` as a time-window correlation mechanism and `same_srcip`/`same_user` as frequency/timeframe correlation filters. (Wazuh Documentation)

For that reason, I would test the **success-after-failure rules against your actual decoder output** before deploying them broadly.

Recommended testing

Use:

```shell
/var/ossec/bin/wazuh-logtest
```

Wazuh's current documentation specifically recommends `wazuh-logtest` for verifying which decoder and rules match an event, and it maintains a session's fired-event history so frequency/timeframe rules can be tested. (Wazuh Documentation)

Test at least these sequences:

```none
1. Single failed login
   → no custom brute-force alert
```

```
2. 5 failures / same source / 5 min
   → 100110

3. 10 failures / same source / 5 min
   → 100111

4. 20 failures / same source / 10 min
   → 100112

5. Same source + many usernames
   → 100120 / 100121

6. Failed logins + successful login
   → 100130/100131, subject to your decoder/rule correlation
behavior

7. Successful login at 23:00
   → 100140

8. Same account + multiple source IPs
   → 100150/100151

9. Same source + multiple destination hosts
   → 100160
```

After testing, restart the manager to load the rules for normal alert generation. Wazuh recommends putting larger custom rule sets in separate files under `/var/ossec/etc/rules/`, rather than modifying the built-in ruleset, so upgrades don't overwrite your changes. (Wazuh Documentation)

Wazuh Rules Syntax documentation

Wazuh Custom Rules documentation

Next logical file: `020-process.xml`, where we can correlate this authentication layer with suspicious process execution—e.g. **failed logins → successful login → PowerShell/shell execution → persistence → P0/P1 incident**.

020-process.xml

Below is a **complete `020-process.xml`** intended for the architecture we defined. It is primarily **Windows + Sysmon Event ID 1 (Process Creation)** because that gives the richest process telemetry in Wazuh: image, command line, parent image, user, integrity level, etc. Wazuh's current Sysmon rules use the `sysmon_event1` group and fields such as `win.eventdata.image`, `win.eventdata.parentImage`, and `win.eventdata.commandLine`. (GitHub)

I use IDs `100020–100049`, which are inside Wazuh's recommended custom-rule range of `100000–120000`. (Wazuh Documentation)

`/var/ossec/etc/rules/020-process.xml`

XML

```
<!--
  Wazuh Custom Process Detection Rules
  File: 020-process.xml

  Purpose:
    - Windows Sysmon process-creation detection
    - Reduce alert fatigue through precision rules
    - Detect LOLBins, suspicious PowerShell, credential-access
tooling,
      defense evasion, persistence-related process execution, and
      suspicious parent/child process relationships.

  Requirements:
    - Windows Event Channel collection
    - Sysmon Operational channel
    - Sysmon Event ID 1 enabled
    - Wazuh Sysmon rules loaded

  Custom rule IDs:
    100020 - 100049

  IMPORTANT:
    Tune the allowlists for your environment before production use.
-->
```

```
<group name="windows,sysmon,custom_process,">

    <!--
=============================================================
         100020 - PROCESS CREATION BASE

============================================================= -->

    <rule id="100020" level="0">
        <if_group>sysmon_event1</if_group>
        <field name="win.eventdata.image">\.+</field>
        <description>Custom - Windows process creation
telemetry</description>
        <group>process_creation,</group>
    </rule>


    <!--
=============================================================
         POWERSHELL

============================================================= -->

    <!-- PowerShell execution.
         Low severity because PowerShell is legitimate
administration
         and should NOT generate high-volume alerts by itself.
    -->
    <rule id="100021" level="3">
        <if_sid>100020</if_sid>
        <field name="win.eventdata.image"
type="pcre2">(?i)\\(powershell|pwsh)\.exe$</field>
        <description>Process - PowerShell execution</description>
        <mitre>
            <id>T1059.001</id>
        </mitre>
```

```
<group>process_creation,powershell,command_and_scripting,</group>
    </rule>


    <!-- Encoded PowerShell -->
    <rule id="100022" level="10">
        <if_sid>100021</if_sid>
        <field name="win.eventdata.commandLine"
type="pcre2">(?i)(-|/)(enc|encodedcommand)(\s|$)</field>
        <description>Process - PowerShell encoded command
execution</description>
        <mitre>
            <id>T1059.001</id>
            <id>T1027</id>
        </mitre>

<group>process_creation,powershell,obfuscated_files_or_information,
</group>
    </rule>


    <!-- PowerShell download / remote content -->
    <rule id="100023" level="10">
        <if_sid>100021</if_sid>
        <field name="win.eventdata.commandLine"
type="pcre2">(?i)(Invoke-WebRequest|iwr|wget|curl|WebClient|Downloa
dString|DownloadFile|Start-BitsTransfer|Net\.WebClient)</field>
        <description>Process - PowerShell remote content
retrieval</description>
        <mitre>
            <id>T1059.001</id>
            <id>T1105</id>
        </mitre>

<group>process_creation,powershell,ingress_tool_transfer,</group>
```

```
    </rule>


    <!-- PowerShell execution policy bypass -->
    <rule id="100024" level="9">
        <if_sid>100021</if_sid>
        <field name="win.eventdata.commandLine"
type="pcre2">(?i)(ExecutionPolicy|execpolicy)\s+(bypass|unrestricte
d)</field>
        <description>Process - PowerShell execution policy
bypass</description>
        <mitre>
            <id>T1059.001</id>
            <id>T1562.001</id>
        </mitre>
        <group>process_creation,powershell,defense_evasion,</group>
    </rule>


    <!-- PowerShell hidden/non-interactive execution -->
    <rule id="100025" level="8">
        <if_sid>100021</if_sid>
        <field name="win.eventdata.commandLine"
type="pcre2">(?i)(-windowstyle\s+hidden|-w\s+hidden|-noninteractive
|-noni|-nop|-noprofile)</field>
        <description>Process - PowerShell hidden or non-interactive
execution</description>
        <mitre>
            <id>T1059.001</id>
            <id>T1564</id>
        </mitre>
        <group>process_creation,powershell,defense_evasion,</group>
    </rule>
```

```
    <!--
============================================================
         CMD / SCRIPT INTERPRETERS

============================================================ -->

    <rule id="100026" level="4">
        <if_sid>100020</if_sid>
        <field name="win.eventdata.image"
type="pcre2">(?i)\\cmd\.exe$</field>
        <description>Process - Windows command shell
execution</description>
        <mitre>
            <id>T1059.003</id>
        </mitre>
        <group>process_creation,command_shell,</group>
    </rule>


    <!-- CMD spawning PowerShell -->
    <rule id="100027" level="9">
        <if_sid>100021</if_sid>
        <field name="win.eventdata.parentImage"
type="pcre2">(?i)\\cmd\.exe$</field>
        <description>Process - CMD spawned PowerShell</description>
        <mitre>
            <id>T1059.003</id>
            <id>T1059.001</id>
        </mitre>

<group>process_creation,command_shell,powershell,suspicious_parent_
child,</group>
    </rule>
```

```
    <!--
============================================================
         LOLBIN / SIGNED BINARY PROXY EXECUTION

============================================================ -->

    <!-- mshta -->
    <rule id="100028" level="10">
        <if_sid>100020</if_sid>
        <field name="win.eventdata.image"
type="pcre2">(?i)\\mshta\.exe$</field>
        <description>Process - MSHTA execution</description>
        <mitre>
            <id>T1218.005</id>
        </mitre>

<group>process_creation,lolbin,signed_binary_proxy_execution,</grou
p>
    </rule>


    <!-- regsvr32 -->
    <rule id="100029" level="10">
        <if_sid>100020</if_sid>
        <field name="win.eventdata.image"
type="pcre2">(?i)\\regsvr32\.exe$</field>
        <description>Process - Regsvr32 execution</description>
        <mitre>
            <id>T1218.010</id>
        </mitre>

<group>process_creation,lolbin,signed_binary_proxy_execution,</grou
p>
    </rule>
```

```
    <!-- rundll32 -->
    <rule id="100030" level="9">
        <if_sid>100020</if_sid>
        <field name="win.eventdata.image"
type="pcre2">(?i)\\rundll32\.exe$</field>
        <description>Process - Rundll32 execution</description>
        <mitre>
            <id>T1218.011</id>
        </mitre>

<group>process_creation,lolbin,signed_binary_proxy_execution,</grou
p>
    </rule>


    <!-- certutil -->
    <rule id="100031" level="9">
        <if_sid>100020</if_sid>
        <field name="win.eventdata.image"
type="pcre2">(?i)\\certutil\.exe$</field>
        <description>Process - Certutil execution</description>
        <mitre>
            <id>T1105</id>
            <id>T1140</id>
        </mitre>

<group>process_creation,lolbin,ingress_tool_transfer,deobfuscate_de
code,</group>
    </rule>


    <!-- bitsadmin -->
    <rule id="100032" level="9">
        <if_sid>100020</if_sid>
        <field name="win.eventdata.image"
type="pcre2">(?i)\\bitsadmin\.exe$</field>
```

```
        <description>Process - Bitsadmin execution</description>
        <mitre>
            <id>T1105</id>
        </mitre>

<group>process_creation,lolbin,ingress_tool_transfer,</group>
    </rule>


    <!-- msiexec -->
    <rule id="100033" level="8">
        <if_sid>100020</if_sid>
        <field name="win.eventdata.image"
type="pcre2">(?i)\\msiexec\.exe$</field>
        <description>Process - Msiexec execution</description>
        <mitre>
            <id>T1218.007</id>
        </mitre>

<group>process_creation,lolbin,signed_binary_proxy_execution,</grou
p>
    </rule>


    <!--
============================================================
         SCRIPTING / INTERPRETERS

============================================================ -->

    <rule id="100034" level="7">
        <if_sid>100020</if_sid>
        <field name="win.eventdata.image"
type="pcre2">(?i)\\(wscript|cscript)\.exe$</field>
        <description>Process - Windows Script Host
execution</description>
```

```
        <mitre>
            <id>T1059.005</id>
        </mitre>
        <group>process_creation,script_execution,</group>
    </rule>


    <rule id="100035" level="7">
        <if_sid>100020</if_sid>
        <field name="win.eventdata.image"
type="pcre2">(?i)\\msbuild\.exe$</field>
        <description>Process - MSBuild execution</description>
        <mitre>
            <id>T1127.001</id>
        </mitre>

<group>process_creation,lolbin,signed_binary_proxy_execution,</grou
p>
    </rule>


    <!--
============================================================
         CREDENTIAL ACCESS / SECURITY TOOL INTERFERENCE

============================================================ -->

    <!-- Credential dumping utilities -->
    <rule id="100036" level="13">
        <if_sid>100020</if_sid>
        <field name="win.eventdata.image"
type="pcre2">(?i)\\(mimikatz|nanodump|procdump)\.exe$</field>
        <description>Process - Credential dumping or LSASS access
tool</description>
        <mitre>
            <id>T1003</id>
```

```
        </mitre>

<group>process_creation,credential_access,credential_dumping,</grou
p>
    </rule>


    <!-- Direct LSASS dump using common Windows tooling -->
    <rule id="100037" level="13">
        <if_sid>100020</if_sid>
        <field name="win.eventdata.image"
type="pcre2">(?i)\\(rundll32|procdump|taskmgr|comsvcs)\.exe$</field
>
        <field name="win.eventdata.commandLine"
type="pcre2">(?i)(lsass|MiniDump|comsvcs\.dll)</field>
        <description>Process - Potential LSASS memory
dump</description>
        <mitre>
            <id>T1003.001</id>
        </mitre>

<group>process_creation,credential_access,lsass,credential_dumping,
</group>
    </rule>


    <!--
============================================================
         DEFENSE EVASION

============================================================ -->

    <!-- Windows Defender tampering -->
    <rule id="100038" level="12">
        <if_sid>100020</if_sid>
```

```
        <field name="win.eventdata.image"
type="pcre2">(?i)\\(powershell|pwsh|cmd)\.exe$</field>
        <field name="win.eventdata.commandLine"
type="pcre2">(?i)(Set-MpPreference|DisableRealtimeMonitoring|Disabl
eBehaviorMonitoring|DisableIOAVProtection|DisableScriptScanning)</f
ield>
        <description>Process - Attempt to disable or weaken Windows
Defender</description>
        <mitre>
            <id>T1562.001</id>
        </mitre>

<group>process_creation,defense_evasion,impair_defenses,</group>
    </rule>


    <!-- Event log clearing -->
    <rule id="100039" level="12">
        <if_sid>100020</if_sid>
        <field name="win.eventdata.image"
type="pcre2">(?i)\\(wevtutil|powershell|pwsh)\.exe$</field>
        <field name="win.eventdata.commandLine"
type="pcre2">(?i)(cl\s+(system|security|application)|Clear-EventLog
|Clear-WinEvent)</field>
        <description>Process - Windows event log
clearing</description>
        <mitre>
            <id>T1070.001</id>
        </mitre>

<group>process_creation,defense_evasion,indicator_removal,event_log
_clearing,</group>
    </rule>
```

```
    <!--
============================================================
         SHADOW COPY / RECOVERY INHIBITION

============================================================ -->

    <rule id="100040" level="13">
        <if_sid>100020</if_sid>
        <field name="win.eventdata.image"
type="pcre2">(?i)\\(vssadmin|wmic|wbadmin|diskshadow)\.exe$</field>
        <field name="win.eventdata.commandLine"
type="pcre2">(?i)(delete\s+shadows|shadowcopy\s+delete|delete\s+cat
alog|delete\s+backup)</field>
        <description>Process - Attempt to delete backups or shadow
copies</description>
        <mitre>
            <id>T1490</id>
        </mitre>

<group>process_creation,impact,inhibit_system_recovery,</group>
    </rule>


    <!--
============================================================
         SUSPICIOUS PARENT / CHILD RELATIONSHIPS

============================================================ -->

    <!-- Office application spawning shell -->
    <rule id="100041" level="11">
        <if_sid>100020</if_sid>
        <field name="win.eventdata.image"
type="pcre2">(?i)\\(cmd|powershell|pwsh|wscript|cscript|mshta)\.exe
$</field>
```

```
        <field name="win.eventdata.parentImage"
type="pcre2">(?i)\\(winword|excel|powerpnt|outlook)\.exe$</field>
        <description>Process - Office application spawned command
or script interpreter</description>
        <mitre>
            <id>T1204.002</id>
            <id>T1059</id>
        </mitre>

<group>process_creation,suspicious_parent_child,office,execution,</
group>
    </rule>


    <!-- Browser spawning shell -->
    <rule id="100042" level="9">
        <if_sid>100020</if_sid>
        <field name="win.eventdata.image"
type="pcre2">(?i)\\(cmd|powershell|pwsh|wscript|cscript|mshta)\.exe
$</field>
        <field name="win.eventdata.parentImage"
type="pcre2">(?i)\\(chrome|msedge|firefox)\.exe$</field>
        <description>Process - Browser spawned command or script
interpreter</description>
        <mitre>
            <id>T1059</id>
        </mitre>

<group>process_creation,suspicious_parent_child,browser,execution,<
/group>
    </rule>


    <!-- Script interpreter spawned by an unusual parent -->
    <rule id="100043" level="8">
        <if_sid>100020</if_sid>
```

```
        <field name="win.eventdata.image"
type="pcre2">(?i)\\(powershell|pwsh|cmd|wscript|cscript|mshta)\.exe
$</field>
        <field name="win.eventdata.parentImage"
type="pcre2">(?i)\\(services|svchost|wininit|lsass)\.exe$</field>
        <description>Process - Script interpreter spawned by
sensitive Windows process</description>
        <mitre>
            <id>T1059</id>
            <id>T1055</id>
        </mitre>

<group>process_creation,suspicious_parent_child,privilege_escalatio
n,</group>
    </rule>


    <!--
============================================================
         USER-WRITABLE PATH EXECUTION

============================================================ -->

    <!-- Execution from Temp -->
    <rule id="100044" level="9">
        <if_sid>100020</if_sid>
        <field name="win.eventdata.image"
type="pcre2">(?i)\\(Users\\[^\\]+\\AppData\\Local\\Temp|Windows\\Te
mp|ProgramData\\Temp)\\.+\.exe$</field>
        <description>Process - Executable launched from temporary
or user-writable path</description>
        <mitre>
            <id>T1204.002</id>
            <id>T1036</id>
        </mitre>
```

```
<group>process_creation,user_writable_path,execution,</group>
    </rule>


    <!-- Execution from Downloads -->
    <rule id="100045" level="8">
        <if_sid>100020</if_sid>
        <field name="win.eventdata.image"
type="pcre2">(?i)\\Users\\[^\\]+\\Downloads\\.+\.exe$</field>
        <description>Process - Executable launched from Downloads
directory</description>
        <mitre>
            <id>T1204.002</id>
        </mitre>

<group>process_creation,user_writable_path,execution,</group>
    </rule>


    <!--
============================================================
         SYSTEM / ADMINISTRATIVE TOOL ABUSE

============================================================ -->

    <!-- PowerShell / command shell executing service creation -->
    <rule id="100046" level="11">
        <if_sid>100020</if_sid>
        <field name="win.eventdata.image"
type="pcre2">(?i)\\(sc|powershell|pwsh|cmd)\.exe$</field>
        <field name="win.eventdata.commandLine"
type="pcre2">(?i)(create\s+.*binPath|New-Service|sc\.exe\s+create)<
/field>
        <description>Process - Service creation through command or
PowerShell</description>
```

```
        <mitre>
            <id>T1543.003</id>
        </mitre>

<group>process_creation,persistence,windows_service,</group>
    </rule>


    <!-- Scheduled task creation -->
    <rule id="100047" level="10">
        <if_sid>100020</if_sid>
        <field name="win.eventdata.image"
type="pcre2">(?i)\\(schtasks|powershell|pwsh)\.exe$</field>
        <field name="win.eventdata.commandLine"
type="pcre2">(?i)(/create|New-ScheduledTask|Register-ScheduledTask)
</field>
        <description>Process - Scheduled task
creation</description>
        <mitre>
            <id>T1053.005</id>
        </mitre>
        <group>process_creation,persistence,scheduled_task,</group>
    </rule>


    <!-- Registry run key modification through command -->
    <rule id="100048" level="10">
        <if_sid>100020</if_sid>
        <field name="win.eventdata.image"
type="pcre2">(?i)\\(reg|powershell|pwsh|cmd)\.exe$</field>
        <field name="win.eventdata.commandLine"
type="pcre2">(?i)(HKCU\\Software\\Microsoft\\Windows\\CurrentVersio
n\\Run|HKLM\\Software\\Microsoft\\Windows\\CurrentVersion\\Run|New-
ItemProperty)</field>
        <description>Process - Potential Registry Run key
persistence</description>
```

```
        <mitre>
            <id>T1547.001</id>
        </mitre>

<group>process_creation,persistence,registry_run_keys,</group>
    </rule>


    <!--
============================================================
         HIGH-VALUE CORRELATION BASE

============================================================ -->

    <!-- Repeated suspicious process activity.
         This is intentionally aggregated rather than generating
         an alert for every individual event.
    -->
    <rule id="100049" level="12" frequency="5" timeframe="300">
        <if_matched_group>custom_process</if_matched_group>
        <same_field>win.eventdata.image</same_field>
        <description>Process - Repeated execution of suspicious
process within 5 minutes</description>
        <mitre>
            <id>T1059</id>
        </mitre>
        <group>process_correlation,alert_aggregation,</group>
    </rule>

</group>
```

Important correction for the correlation rule

I would **not deploy `100049` exactly as written** if your intent is to correlate only the higher-confidence process rules. The `custom_process` group includes the base rule, so it can become broader than intended.

A better production version is to create a dedicated group for high-confidence detections:

```xml
<group>process_high_confidence,</group>
```

on rules such as `100022`, `100023`, `100028`, `100029`, `100036`, `100038`, `100039`, `100040`, `100041`, `100046`, `100047`, and `100048`.

Then use:

```xml
<rule id="100049" level="12" frequency="5" timeframe="300">
    <if_matched_group>process_high_confidence</if_matched_group>
    <same_field>win.eventdata.image</same_field>
    <description>Process - Repeated high-confidence suspicious
process activity</description>
    <group>process_correlation,alert_aggregation,</group>
</rule>
```

This follows Wazuh's `if_matched_group` + `frequency`/`timeframe` mechanism for temporal correlation. (Wazuh Documentation)

Recommended alert-fatigue behavior

The important part of this file is the **severity gradient**:

```
Normal PowerShell
       │
       └── Level 3

PowerShell + encoded command
       │
       └── Level 10

PowerShell + encoded command + suspicious parent
       │
```

```
       └── Correlation → P1/P0

Credential dumping
       │
       └── Level 13

Shadow-copy deletion
       │
       └── Level 13
```

This is preferable to making every process event level 10–12.

Wazuh supports levels 0–16, with level 0 specifically intended for events that should not generate an alert, while levels determine alert/response severity. (Wazuh Documentation)

One more important point

The built-in Sysmon Event 1 parent rule is currently level `0` and belongs to `sysmon_event1`; your custom rules can therefore inherit that group with `<if_group>sysmon_event1</if_group>`. (GitHub)

That means **you do not need to overwrite rule `61603` just to make these custom detections work**. This is preferable because overwriting built-in rules creates unnecessary maintenance risk.

Deploy

Put the file on the **Wazuh manager**, not the Windows endpoint:

```
None
/var/ossec/etc/rules/020-process.xml
```

Wazuh supports loading custom rule files from its rules directory; for larger custom rulesets, separate files are recommended rather than putting everything into `local_rules.xml`. (Wazuh Documentation)

Then:

```shell
sudo /var/ossec/bin/wazuh-analysisd -t
sudo systemctl restart wazuh-manager
```

Test individual events with:

```shell
sudo /var/ossec/bin/wazuh-logtest
```

For production, I would next build **`030-persistence.xml`** so that process execution, registry/service/task persistence, and FIM events can be correlated into a single incident instead of three independent alerts.

030-persistence.xml

Below is a production-oriented `030-persistence.xml` for the architecture we discussed. It uses the Wazuh custom-rule range `100000–120000`, as recommended by Wazuh, and keeps the rules in a separate file under `/var/ossec/etc/rules/`.

The file covers the major persistence surfaces for **Linux + Windows**, with an emphasis on **high-signal detection rather than alerting on every persistence-related change**.

030-persistence.xml

```
None
<!--
  Wazuh Custom Persistence Detection Rules
  File: /var/ossec/etc/rules/030-persistence.xml

  Purpose:
    Detect common persistence mechanisms while reducing alert
fatigue.

  Platforms:
    Linux / Unix
    Windows
    Windows + Sysmon

  MITRE ATT&CK:
    T1053.005  Scheduled Task/Job: Scheduled Task
    T1053.003  Scheduled Task/Job: Cron
    T1543.003  Create or Modify System Process: Windows Service
    T1543.002  Create or Modify System Process: Systemd Service
    T1547.001  Registry Run Keys / Startup Folder
    T1547.006  Kernel Modules and Extensions
    T1547.009  Shortcut Modification
    T1547.010  Port Monitors
    T1547.015  Login Items
    T1546.004  Unix Shell Configuration Modification
    T1098      Account Manipulation
    T1136      Create Account

  IMPORTANT:
```

```
    1. Validate field names against your Wazuh version with
wazuh-logtest.
    2. Rules depending on Sysmon require Sysmon telemetry.
    3. Linux FIM rules require the relevant paths to be monitored
by syscheck.
    4. Linux Audit rules require auditd/audit telemetry.
    5. Do not blindly whitelist persistence activity.
-->

<group name="persistence,attack,">

  <!--
============================================================
       SECTION 1 — LINUX / UNIX FIM

============================================================ -->

  <!--
------------------------------------------------------------
       1.1 SSH authorized_keys modification
       MITRE: T1098.004 / Account Manipulation: SSH Authorized Keys

------------------------------------------------------------ -->

  <rule id="100300" level="10">
    <if_sid>550</if_sid>
    <field name="file"
type="pcre2">(?i)^/home/[^/]+/\.ssh/authorized_keys$</field>
    <description>Persistence: SSH authorized_keys file modified —
possible unauthorized SSH key persistence</description>
    <mitre>
      <id>T1098.004</id>
    </mitre>

<group>persistence,ssh_persistence,account_manipulation,</group>
  </rule>
```

```
  <rule id="100301" level="12">
    <if_sid>550</if_sid>
    <field name="file"
type="pcre2">(?i)^/root/\.ssh/authorized_keys$</field>
    <description>Critical persistence: root SSH authorized_keys
modified</description>
    <mitre>
      <id>T1098.004</id>
    </mitre>

<group>persistence,ssh_persistence,critical_asset,account_manipulat
ion,</group>
  </rule>


  <!--
------------------------------------------------------------------
       1.2 SSH configuration

------------------------------------------------------------------ -->

  <rule id="100302" level="8">
    <if_sid>550</if_sid>
    <field name="file"
type="pcre2">(?i)^/etc/ssh/(sshd_config|ssh_config)(\.d/.*)?$</fiel
d>
    <description>Persistence: SSH configuration modified - review
for authentication or access persistence</description>
    <mitre>
      <id>T1098.004</id>
    </mitre>

<group>persistence,ssh_persistence,configuration_change,</group>
  </rule>
```

```
  <!--
-------------------------------------------------------------
       1.3 Cron persistence
       MITRE: T1053.003

------------------------------------------------------------- -->

  <rule id="100303" level="9">
    <if_sid>550</if_sid>
    <field name="file"
type="pcre2">(?i)^(/etc/crontab|/etc/cron\.(d|daily|hourly|monthly|
weekly)/[^/]+)$</field>
    <description>Persistence: system cron configuration
modified</description>
    <mitre>
      <id>T1053.003</id>
    </mitre>
    <group>persistence,cron_persistence,scheduled_task,</group>
  </rule>

  <rule id="100304" level="10">
    <if_sid>550</if_sid>
    <field name="file"
type="pcre2">(?i)^/var/spool/cron/[^/]+$</field>
    <description>Persistence: user crontab modified</description>
    <mitre>
      <id>T1053.003</id>
    </mitre>
    <group>persistence,cron_persistence,scheduled_task,</group>
  </rule>


  <!--
-------------------------------------------------------------
       1.4 Systemd persistence
```

```
       MITRE: T1543.002

------------------------------------------------------------- -->

  <rule id="100305" level="10">
    <if_sid>550</if_sid>
    <field name="file"
type="pcre2">(?i)^/(etc/systemd/system|usr/lib/systemd/system|lib/s
ystemd/system)/[^/]+\.service$</field>
    <description>Persistence: systemd service unit created or
modified</description>
    <mitre>
      <id>T1543.002</id>
    </mitre>

<group>persistence,systemd_persistence,service_persistence,</group>
  </rule>


  <!--
-------------------------------------------------------------
       1.5 Systemd timers

------------------------------------------------------------- -->

  <rule id="100306" level="9">
    <if_sid>550</if_sid>
    <field name="file"
type="pcre2">(?i)^/(etc/systemd/system|usr/lib/systemd/system|lib/s
ystemd/system)/[^/]+\.timer$</field>
    <description>Persistence: systemd timer created or
modified</description>
    <mitre>
      <id>T1053.006</id>
    </mitre>
    <group>persistence,systemd_persistence,scheduled_task,</group>
```

```
  </rule>


  <!--
------------------------------------------------------------------
       1.6 Init scripts
       MITRE: T1037 / Unix shell configuration related persistence

------------------------------------------------------------------ -->

  <rule id="100307" level="8">
    <if_sid>550</if_sid>
    <field name="file"
type="pcre2">(?i)^/etc/init\.d/[^/]+$</field>
    <description>Persistence: Linux init script
modified</description>
    <mitre>
      <id>T1037</id>
    </mitre>
    <group>persistence,boot_persistence,</group>
  </rule>


  <!--
------------------------------------------------------------------
       1.7 Shell startup files
       MITRE: T1546.004

------------------------------------------------------------------ -->

  <rule id="100308" level="8">
    <if_sid>550</if_sid>
    <field name="file"
type="pcre2">(?i)^/home/[^/]+/\.(bashrc|bash_profile|profile|bash_l
ogin|zshrc|zprofile|kshrc)$</field>
```

```
    <description>Persistence: user shell startup configuration
modified</description>
    <mitre>
      <id>T1546.004</id>
    </mitre>
    <group>persistence,shell_persistence,unix_shell,</group>
  </rule>

  <rule id="100309" level="9">
    <if_sid>550</if_sid>
    <field name="file"
type="pcre2">(?i)^/root/\.(bashrc|bash_profile|profile|bash_login|z
shrc|zprofile|kshrc)$</field>
    <description>Persistence: root shell startup configuration
modified</description>
    <mitre>
      <id>T1546.004</id>
    </mitre>
    <group>persistence,shell_persistence,critical_asset,</group>
  </rule>


  <!--
-------------------------------------------------------------
       1.8 Sudoers persistence

------------------------------------------------------------- -->

  <rule id="100310" level="10">
    <if_sid>550</if_sid>
    <field name="file" type="pcre2">(?i)^/etc/sudoers$</field>
    <description>Persistence: sudoers configuration
modified</description>
    <mitre>
      <id>T1548.003</id>
    </mitre>
```

```
    <group>persistence,privilege_escalation,sudoers,</group>
  </rule>

  <rule id="100311" level="10">
    <if_sid>550</if_sid>
    <field name="file"
type="pcre2">(?i)^/etc/sudoers\.d/[^/]+$</field>
    <description>Persistence: sudoers.d configuration
modified</description>
    <mitre>
      <id>T1548.003</id>
    </mitre>
    <group>persistence,privilege_escalation,sudoers,</group>
  </rule>


  <!--
-------------------------------------------------------------
       1.9 PAM configuration

------------------------------------------------------------- -->

  <rule id="100312" level="11">
    <if_sid>550</if_sid>
    <field name="file" type="pcre2">(?i)^/etc/pam\.d/[^/]+$</field>
    <description>Persistence: PAM configuration
modified</description>
    <mitre>
      <id>T1556</id>
    </mitre>
    <group>persistence,authentication_persistence,pam,</group>
  </rule>


  <!--
-------------------------------------------------------------
```

```
       1.10 Dynamic linker / preload persistence

------------------------------------------------------------------ -->

  <rule id="100313" level="12">
    <if_sid>550</if_sid>
    <field name="file"
type="pcre2">(?i)^/etc/ld\.so\.preload$</field>
    <description>Critical persistence: dynamic linker preload
configuration modified</description>
    <mitre>
      <id>T1574</id>
    </mitre>
    <group>persistence,defense_evasion,critical_asset,</group>
  </rule>


  <!--
------------------------------------------------------------------
       1.11 Kernel module configuration

------------------------------------------------------------------ -->

  <rule id="100314" level="11">
    <if_sid>550</if_sid>
    <field name="file"
type="pcre2">(?i)^/etc/modprobe\.d/[^/]+\.conf$</field>
    <description>Persistence: kernel module configuration
modified</description>
    <mitre>
      <id>T1547.006</id>
    </mitre>
    <group>persistence,kernel_persistence,</group>
  </rule>
```

```
  <!--
=================================================================
       SECTION 2 — WINDOWS SERVICES

================================================================= -->

  <!--
-----------------------------------------------------------------
       2.1 Windows service creation
       Event ID: 7045
       MITRE: T1543.003

----------------------------------------------------------------- -->

  <rule id="100320" level="9">
    <if_group>windows_system</if_group>
    <field name="win.system.eventID">^7045$</field>
    <description>Persistence: new Windows service
created</description>
    <mitre>
      <id>T1543.003</id>
    </mitre>
    <group>persistence,windows_service,service_persistence,</group>
  </rule>


  <!--
-----------------------------------------------------------------
       2.2 Suspicious Windows service binary

----------------------------------------------------------------- -->

  <rule id="100321" level="12">
    <if_sid>100320</if_sid>
    <field name="win.eventdata.imagePathName"
type="pcre2">(?i)(\\Users\\|\\AppData\\|\\ProgramData\\|\\Temp\\|\\
```

```
Downloads\\|powershell|cmd\.exe|wscript|cscript|mshta|rundll32)</fi
eld>
    <description>High-confidence persistence: Windows service
points to suspicious user-writable or script execution
path</description>
    <mitre>
      <id>T1543.003</id>
    </mitre>
    <group>persistence,windows_service,suspicious_path,</group>
  </rule>


  <!--
============================================================
       SECTION 3 — WINDOWS SCHEDULED TASKS

============================================================ -->

  <!--
------------------------------------------------------------
       3.1 Scheduled task creation
       Event ID: 4698
       MITRE: T1053.005

------------------------------------------------------------ -->

  <rule id="100330" level="8">
    <if_group>windows_security</if_group>
    <field name="win.system.eventID">^4698$</field>
    <description>Persistence: Windows scheduled task
created</description>
    <mitre>
      <id>T1053.005</id>
    </mitre>

<group>persistence,scheduled_task,windows_scheduled_task,</group>
```

```
  </rule>


  <!--
-------------------------------------------------------------
       3.2 Scheduled task modified
       Event ID: 4702

------------------------------------------------------------- -->

  <rule id="100331" level="8">
    <if_group>windows_security</if_group>
    <field name="win.system.eventID">^4702$</field>
    <description>Persistence: Windows scheduled task
modified</description>
    <mitre>
      <id>T1053.005</id>
    </mitre>

<group>persistence,scheduled_task,windows_scheduled_task,</group>
  </rule>


  <!--
-------------------------------------------------------------
       3.3 Suspicious scheduled task content

------------------------------------------------------------- -->

  <rule id="100332" level="12">
    <if_sid>100330</if_sid>
    <field name="win.eventdata.taskContent"
type="pcre2">(?is)(powershell|pwsh|cmd\.exe|wscript|cscript|mshta|r
undll32|regsvr32|certutil|bitsadmin|AppData|ProgramData|Temp|Downlo
ads)</field>
```

```
    <description>High-confidence persistence: scheduled task
executes script, LOLBin or payload from suspicious
path</description>
    <mitre>
      <id>T1053.005</id>
    </mitre>
    <group>persistence,scheduled_task,suspicious_execution,</group>
  </rule>


  <!--
============================================================
       SECTION 4 — WINDOWS REGISTRY RUN KEYS
       Sysmon Event ID 13
       MITRE: T1547.001

============================================================ -->

  <!--
------------------------------------------------------------
       4.1 Common Run / RunOnce keys

------------------------------------------------------------ -->

  <rule id="100340" level="9">
    <if_group>sysmon_event13</if_group>
    <field name="win.eventdata.targetObject"
type="pcre2">(?i)\\Software\\Microsoft\\Windows\\CurrentVersion\\(R
un|RunOnce)(\\|$)</field>
    <description>Persistence: Windows Registry Run or RunOnce key
modified</description>
    <mitre>
      <id>T1547.001</id>
    </mitre>
    <group>persistence,registry_persistence,run_keys,</group>
  </rule>
```

```
  <!--
-------------------------------------------------------------
       4.2 Suspicious Run key payload

------------------------------------------------------------- -->

  <rule id="100341" level="12">
    <if_sid>100340</if_sid>
    <field name="win.eventdata.details"
type="pcre2">(?i)(powershell|pwsh|cmd\.exe|wscript|cscript|mshta|ru
ndll32|regsvr32|AppData|ProgramData|Temp|Downloads|\.js$|\.vbs$|\.h
ta$|\.dll$)</field>
    <description>High-confidence persistence: Registry Run key
points to script, LOLBin or user-writable payload</description>
    <mitre>
      <id>T1547.001</id>
    </mitre>

<group>persistence,registry_persistence,run_keys,suspicious_executi
on,</group>
  </rule>


  <!--
=============================================================
       SECTION 5 — WINDOWS STARTUP FOLDER / USER STARTUP
       Sysmon Event ID 11

============================================================= -->

  <rule id="100350" level="9">
    <if_group>sysmon_event11</if_group>
```

```
    <field name="win.eventdata.targetFilename"
type="pcre2">(?i)\\Microsoft\\Windows\\Start
Menu\\Programs\\Startup\\</field>
    <description>Persistence: executable or script created in
Windows Startup folder</description>
    <mitre>
      <id>T1547.001</id>
    </mitre>
    <group>persistence,startup_folder,windows_persistence,</group>
  </rule>


  <!--
============================================================
       SECTION 6 — WINDOWS STARTUP / SHORTCUT MODIFICATION

============================================================ -->

  <rule id="100351" level="10">
    <if_group>sysmon_event11</if_group>
    <field name="win.eventdata.targetFilename"
type="pcre2">(?i)\\Microsoft\\Windows\\Start
Menu\\Programs\\Startup\\.*\.(lnk|url)$</field>
    <description>Persistence: Windows Startup shortcut
created</description>
    <mitre>
      <id>T1547.001</id>
    </mitre>
    <group>persistence,startup_folder,shortcut_persistence,</group>
  </rule>


  <!--
============================================================
       SECTION 7 — WINDOWS WMI PERSISTENCE
       Event ID 5861
```

```
       MITRE: T1546.003

============================================================== -->

  <rule id="100360" level="11">
    <if_group>windows_system</if_group>
    <field name="win.system.eventID">^5861$</field>
    <description>Persistence: Windows WMI permanent event
subscription detected</description>
    <mitre>
      <id>T1546.003</id>
    </mitre>
    <group>persistence,wmi_persistence,event_subscription,</group>
  </rule>


  <!--
============================================================
       SECTION 8 — WINDOWS BOOT / LOGON AUTOSTART REGISTRY

============================================================== -->

  <rule id="100370" level="10">
    <if_group>sysmon_event13</if_group>
    <field name="win.eventdata.targetObject"
type="pcre2">(?i)(\\Winlogon\\(Shell|Userinit)|\\Windows
NT\\CurrentVersion\\Winlogon\\)</field>
    <description>High-risk persistence: Windows Winlogon registry
value modified</description>
    <mitre>
      <id>T1547.004</id>
    </mitre>

<group>persistence,winlogon_persistence,registry_persistence,</grou
p>
  </rule>
```

```
  <!--
============================================================
       SECTION 9 – WINDOWS SECURITY SUPPORT PROVIDER /
AUTHENTICATION

============================================================ -->

  <rule id="100380" level="11">
    <if_group>sysmon_event13</if_group>
    <field name="win.eventdata.targetObject"
type="pcre2">(?i)\\SYSTEM\\CurrentControlSet\\Control\\Lsa\\</field
>
    <description>High-risk persistence: Windows LSA registry
configuration modified</description>
    <mitre>
      <id>T1547</id>
    </mitre>

<group>persistence,authentication_persistence,lsa,registry_persiste
nce,</group>
  </rule>


  <!--
============================================================
       SECTION 10 – WINDOWS OFFICE / USER LOGON PERSISTENCE

============================================================ -->

  <rule id="100390" level="9">
    <if_group>sysmon_event13</if_group>
    <field name="win.eventdata.targetObject"
type="pcre2">(?i)\\Software\\Microsoft\\Windows\\CurrentVersion\\Ex
plorer\\User Shell Folders\\</field>
```

```
    <description>Persistence: Windows User Shell Folder registry
value modified</description>
    <mitre>
      <id>T1547.001</id>
    </mitre>

<group>persistence,registry_persistence,user_shell_folders,</group>
  </rule>


  <!--
============================================================
       SECTION 11 — LINUX ACCOUNT CREATION / MANIPULATION

============================================================ -->

  <!-- New user account -->
  <rule id="100400" level="8">
    <if_group>audit</if_group>
    <field name="audit.syscall">^(useradd|adduser)$</field>
    <description>Persistence: new Linux user account
created</description>
    <mitre>
      <id>T1136.001</id>
    </mitre>
    <group>persistence,account_creation,linux_account,</group>
  </rule>


  <!--
------------------------------------------------------------
       11.2 passwd / shadow modification through FIM

------------------------------------------------------------ -->

  <rule id="100401" level="10">
```

```
    <if_sid>550</if_sid>
    <field name="file" type="pcre2">^/etc/passwd$</field>
    <description>Persistence: /etc/passwd modified</description>
    <mitre>
      <id>T1098</id>
    </mitre>
    <group>persistence,account_manipulation,critical_file,</group>
  </rule>

  <rule id="100402" level="12">
    <if_sid>550</if_sid>
    <field name="file" type="pcre2">^/etc/shadow$</field>
    <description>Critical persistence: /etc/shadow
modified</description>
    <mitre>
      <id>T1098</id>
    </mitre>

<group>persistence,credential_access,account_manipulation,critical_
file,</group>
  </rule>


  <!--
================================================================
       SECTION 12 — LINUX SSH + PERSISTENCE CORRELATION

================================================================ -->

  <!--
    This rule upgrades SSH persistence when it follows an
    authentication-related event.

    The exact parent rule should be adapted to the SSH rules
    installed in your Wazuh ruleset.
  -->
```

```
  <rule id="100410" level="12" frequency="2" timeframe="300">
    <if_matched_sid>100300</if_matched_sid>
    <description>Correlated persistence: repeated SSH
authorized_keys modifications</description>
    <mitre>
      <id>T1098.004</id>
    </mitre>
    <group>persistence,ssh_persistence,correlation,</group>
  </rule>


  <!--
============================================================
       SECTION 13 — PERSISTENCE BURST DETECTION

============================================================ -->

  <!--
    Multiple persistence events on the same host within a short
    window are considerably more interesting than one isolated
    administrative change.
  -->

  <rule id="100420" level="13" frequency="3" timeframe="600">
    <if_matched_group>persistence</if_matched_group>
    <description>Persistence burst: multiple persistence-related
changes detected within 10 minutes</description>
    <mitre>
      <id>T1053.005</id>
      <id>T1543.003</id>
      <id>T1547.001</id>
    </mitre>
    <group>persistence,correlation,attack_chain,</group>
  </rule>
```

```
  <!--
============================================================
       SECTION 14 — MULTIPLE DISTINCT PERSISTENCE MECHANISMS

============================================================ -->

  <!--
    The purpose is to elevate a host when multiple persistence
    mechanisms are observed. This is intentionally based on
    separate persistence detections rather than raw events.
  -->

  <rule id="100421" level="14" frequency="4" timeframe="900">
    <if_matched_group>persistence</if_matched_group>
    <description>High-confidence attack chain: multiple persistence
mechanisms observed on the same endpoint</description>
    <mitre>
      <id>T1053.005</id>
      <id>T1543.003</id>
      <id>T1547.001</id>
      <id>T1546.003</id>
    </mitre>

<group>persistence,correlation,attack_chain,high_confidence,</group
>
  </rule>


  <!--
============================================================
       SECTION 15 — PERSISTENCE ON CRITICAL FILES

============================================================ -->

  <rule id="100430" level="14">
```

```
    <if_group>persistence</if_group>
    <field name="file"
type="pcre2">^(/etc/ld\.so\.preload|/etc/shadow|/root/\.ssh/authori
zed_keys)$</field>
    <description>Critical persistence target modified on Linux
endpoint</description>
    <group>persistence,critical_asset,high_confidence,</group>
  </rule>


  <!--
============================================================
       SECTION 16 — PERSISTENCE FLOOD CONTROL

============================================================ -->

  <!--
    Suppress repeated identical persistence detections after the
    first alert for a short period.

    This does NOT remove the underlying FIM/audit telemetry.
  -->

  <rule id="100440" level="0" frequency="5" timeframe="300"
ignore="300">
    <if_matched_group>persistence</if_matched_group>
    <description>Persistence alert aggregation threshold
reached</description>
    <options>no_log</options>
    <group>persistence_aggregation,</group>
  </rule>


  <!--
============================================================
       SECTION 17 — MAINTENANCE-WINDOW EXAMPLE
```

```
================================================================ -->

  <!--
    IMPORTANT:
    Do not enable this as a blanket exception without adapting it
    to your environment.

    Example only:
    suppress a known maintenance window for service changes.

    Uncomment and customize if required.
  -->

  <!--
  <rule id="100450" level="0">
    <if_sid>100320</if_sid>
    <time>02:00-04:00</time>
    <description>Suppressed expected Windows service changes during
approved maintenance window</description>
    <options>no_log</options>
    <group>persistence_suppression,maintenance,</group>
  </rule>
  -->


  <!--
================================================================
       SECTION 18 — FINAL PERSISTENCE RISK CLASSIFICATION

================================================================ -->

  <!-- Root SSH persistence -->
  <rule id="100460" level="15">
    <if_sid>100301</if_sid>
```

```
    <description>CRITICAL: root SSH persistence mechanism
modified</description>
    <group>persistence,critical,priority_p0,</group>
  </rule>

  <!-- Dynamic linker persistence -->
  <rule id="100461" level="15">
    <if_sid>100313</if_sid>
    <description>CRITICAL: dynamic linker preload persistence
modified</description>
    <group>persistence,critical,priority_p0,</group>
  </rule>

  <!-- Multiple persistence mechanisms -->
  <rule id="100462" level="15">
    <if_sid>100421</if_sid>
    <description>CRITICAL: multiple persistence mechanisms detected
on same endpoint</description>
    <group>persistence,critical,priority_p0,</group>
  </rule>

</group>
```

Important corrections before deploying

There are two places I would **not deploy blindly**.

1. Linux FIM rules

The `if_sid>550</if_sid>` approach is appropriate when the event is coming from Wazuh Syscheck/FIM; Wazuh's own documentation uses rule `550` as the parent for custom FIM rules.

However, these paths must actually be monitored in the agent's `<syscheck>` configuration. Wazuh supports FIM monitoring through `directories`, and it can alert on new/changed files.

For example, your Linux agent should monitor the persistence locations you care about rather than indiscriminately monitoring the entire filesystem.

2. Windows event fields

The Windows rules assume Wazuh is decoding Windows Event Channel data into fields such as:

```
None
win.system.eventID
win.eventdata.taskContent
win.eventdata.targetObject
win.eventdata.details
```

Wazuh's Windows rules use this `win.system.*` / `win.eventdata.*` structure.

For Sysmon, you also need the corresponding Sysmon event rules/groups. In particular, Event 13 is the Registry Value Set event and is commonly represented by a `sysmon_event13` group.

One important change I'd make to the above

I would **not rely on the generic `100420`/`100421` correlation rules in production exactly as written**.

They demonstrate the architecture, but a mature persistence correlation should distinguish mechanisms:

```
None
            Persistence
                 |
      ┌──────────┼──────────┐
      |          |          |
    Linux     Windows    Sysmon
      |          |          |
  authorized   7045      Run Key
  keys         4698      Startup
  cron         4702      Registry
  systemd      5861      etc.
      |          |          |
      └──────────┼──────────┘
                 |
                 ▼
      Multiple mechanisms
         on same host
```

```
                │
                ▼
      P1/P0 correlation
```

That prevents, for example, three benign modifications to the **same persistence mechanism** from being interpreted as three independent persistence techniques.

Validation

After placing the file in:

```
None
/var/ossec/etc/rules/030-persistence.xml
```

validate individual events with:

```
None
/var/ossec/bin/wazuh-logtest
```

Wazuh specifically recommends `wazuh-logtest` for testing custom rules, and the manager should be restarted after rule changes are ready for production.

For Linux persistence, I would test at least:

```
None
1. Modify /home/test/.ssh/authorized_keys
2. Modify /root/.ssh/authorized_keys
3. Modify /etc/crontab
4. Modify /var/spool/cron/*
5. Create a systemd service
6. Modify .bashrc
7. Modify /etc/sudoers.d/*
8. Modify /etc/ld.so.preload
```

For Windows:

```
None
1. Create service          → 7045
2. Create scheduled task   → 4698
3. Modify scheduled task   → 4702
4. Modify Run key          → Sysmon 13
5. Create Startup file     → Sysmon 11
6. WMI persistence         → 5861
```

The **best next step** is to make `030-persistence.xml` fully environment-ready by adding the corresponding **`ossec.conf` FIM paths + Windows Event Channel/Sysmon collection + exact `wazuh-logtest` test events**, so you have a complete deploy-and-validate persistence detection package rather than just the rule file.

040-network.xml

Below is a **production-oriented `040-network.xml`** for the alert-fatigue architecture we discussed. It is designed around four principles:

1. **Low-level network events are retained but kept low severity.**
2. **Repeated events are aggregated into behavioral alerts.**
3. **Scanning / brute-force-like network behavior is correlated rather than emitted individually.**
4. **Known-good network activity can be suppressed with CDB lists rather than hard-coded IP exceptions.**

Wazuh supports `frequency`/`timeframe`, `if_matched_sid`, `same_srcip`, `same_dstip`, `different_dstport`, `ignore`, and CDB-list lookups for exactly these types of rules. (Wazuh Documentation)

`040-network.xml`

XML

```
<!--
  Wazuh Custom Network Detection / Alert-Fatigue Rules
  File: /var/ossec/etc/rules/040-network.xml

  Rule ID range: 100300 - 100399

  Design goals:
    - Keep individual network events low severity.
    - Aggregate repetitive activity.
    - Detect horizontal and vertical scanning.
    - Detect repeated connection attempts.
    - Detect suspicious outbound activity.
    - Support suppression through CDB lists.
    - Produce actionable correlation alerts instead of alert
floods.

  IMPORTANT:
    These rules assume that the relevant decoder populates fields
such as:
      srcip
      dstip
      srcport
```

```
      dstport
      protocol
      action
      status

    Use /var/ossec/bin/wazuh-logtest to validate the decoder and
field names
    against your actual network logs before deploying to
production.
-->

<group name="network,network_security,">

  <!--
=========================================================== -->
  <!-- 100300 - BASE NETWORK EVENTS
-->
  <!--
=========================================================== -->

  <!-- Generic network connection -->
  <rule id="100300" level="2">
    <match>connection|connect|accepted|allowed|established</match>
    <description>Network: Generic network connection
observed.</description>
    <group>network_connection,</group>
  </rule>


  <!-- Generic blocked connection -->
  <rule id="100301" level="3">
    <match>blocked|denied|reject|dropped|DROP|REJECT</match>
    <description>Network: Connection blocked or
denied.</description>
    <group>network_blocked,</group>
  </rule>
```

```
  <!-- Firewall deny -->
  <rule id="100302" level="3">

<match>firewall.*(deny|drop|reject)|iptables.*(DROP|REJECT)|ufw.*DE
NY</match>
    <description>Network: Firewall blocked a connection
attempt.</description>
    <group>firewall,network_blocked,</group>
  </rule>


  <!--
============================================================ -->
  <!-- 100310 - KNOWN-GOOD / SUPPRESSION
-->
  <!--
============================================================ -->

  <!--
    Approved scanners should be placed in:
      /var/ossec/etc/lists/approved-network-scanners

    Example:
      10.10.10.20:
      10.10.10.21:

    The CDB list must also be registered in ossec.conf:
      <list>etc/lists/approved-network-scanners</list>
  -->

  <rule id="100310" level="0">
    <if_sid>100301,100302</if_sid>
    <list field="srcip"
```

```
lookup="address_match_key">etc/lists/approved-network-scanners</lis
t>
    <description>
      Network: Suppressed expected activity from approved network
scanner.
    </description>
    <group>network_suppression,approved_scanner,</group>
  </rule>


  <!--
    Approved monitoring infrastructure.

    Example CDB:
      10.10.20.10:
      10.10.20.11:
  -->

  <rule id="100311" level="0">
    <if_sid>100301,100302</if_sid>
    <list field="srcip"

lookup="address_match_key">etc/lists/approved-network-monitors</lis
t>
    <description>
      Network: Suppressed expected monitoring connection.
    </description>
    <group>network_suppression,approved_monitoring,</group>
  </rule>


  <!--
============================================================ -->
  <!-- 100320 - CONNECTION BURSTS
-->
```

```
  <!--
========================================================== -->

  <!--
    20 blocked connections from the same source in 60 seconds.
    This is an aggregation rule rather than 20 individual analyst
alerts.
  -->

  <rule id="100320"
        level="7"
        frequency="20"
        timeframe="60"
        ignore="120">

    <if_matched_sid>100301</if_matched_sid>
    <same_srcip />

    <description>
      Network: High-volume blocked connection activity from source
$(srcip).
    </description>

    <group>
      network,
      network_blocked,
      connection_burst,
      alert_aggregation,
    </group>

    <mitre>
      <id>T1046</id>
    </mitre>
  </rule>
```

```
  <!-- 50 blocked connections / 5 minutes -->
  <rule id="100321"
        level="9"
        frequency="50"
        timeframe="300"
        ignore="300">

    <if_matched_sid>100301</if_matched_sid>
    <same_srcip />

    <description>
      Network: Sustained blocked connection activity from source
$(srcip).
    </description>

    <group>
      network,
      network_blocked,
      connection_burst,
      suspicious_network_activity,
    </group>

    <mitre>
      <id>T1046</id>
    </mitre>
  </rule>


  <!--
================================================================ -->
  <!-- 100330 - VERTICAL PORT SCANNING
-->
  <!--
================================================================ -->

  <!--
```

```
  Same source repeatedly contacting different destination ports
  on the same destination.

  This detects:
    attacker -> server:22
    attacker -> server:80
    attacker -> server:443
    attacker -> server:445
    ...

  The individual events remain low severity.
  The correlation rule becomes actionable.
-->

<rule id="100330" level="3">
  <if_sid>100300,100301</if_sid>
  <description>
    Network: Connection attempt observed for port-scan analysis.
  </description>
  <group>network_scan_candidate,</group>
</rule>


<rule id="100331"
      level="10"
      frequency="10"
      timeframe="120"
      ignore="300">

  <if_matched_sid>100330</if_matched_sid>
  <same_srcip />
  <same_dstip />
  <different_dstport />

  <description>
```

```
      Network: Possible vertical port scan from $(srcip) against
$(dstip).
    </description>

    <group>
      network,
      network_scan,
      port_scan,
      reconnaissance,
    </group>

    <mitre>
      <id>T1046</id>
    </mitre>
  </rule>


  <!--
============================================================== -->
  <!-- 100340 - HORIZONTAL HOST SCANNING
-->
  <!--
============================================================== -->

  <rule id="100340" level="3">
    <if_sid>100300,100301</if_sid>
    <description>
      Network: Connection attempt observed for host-scan analysis.
    </description>
    <group>network_host_scan_candidate,</group>
  </rule>


  <rule id="100341"
        level="10"
        frequency="10"
```

```
        timeframe="120"
        ignore="300">

    <if_matched_sid>100340</if_matched_sid>
    <same_srcip />
    <different_dstip />

    <description>
      Network: Possible horizontal host scan from $(srcip).
    </description>

    <group>
      network,
      network_scan,
      host_scan,
      reconnaissance,
    </group>

    <mitre>
      <id>T1046</id>
    </mitre>
  </rule>


  <!--
============================================================ -->
  <!-- 100350 - SSH / REMOTE SERVICE CONNECTION BURSTS
-->
  <!--
============================================================ -->

  <rule id="100350" level="3">
    <if_sid>100301</if_sid>
    <dstport>22</dstport>
    <description>
      Network: SSH connection attempt blocked.
```

```
    </description>
    <group>network,ssh,remote_service,</group>
  </rule>


  <rule id="100351"
        level="8"
        frequency="10"
        timeframe="120"
        ignore="300">

    <if_matched_sid>100350</if_matched_sid>
    <same_srcip />
    <same_dstip />

    <description>
      Network: Repeated blocked SSH connection attempts from
$(srcip) to $(dstip).
    </description>

    <group>
      network,
      ssh,
      brute_force_candidate,
      connection_burst,
    </group>

    <mitre>
      <id>T1021.004</id>
    </mitre>
  </rule>


  <!-- RDP -->
  <rule id="100352" level="3">
    <if_sid>100301</if_sid>
```

```
    <dstport>3389</dstport>
    <description>
      Network: RDP connection attempt blocked.
    </description>
    <group>network,rdp,remote_service,</group>
  </rule>


  <rule id="100353"
        level="8"
        frequency="10"
        timeframe="120"
        ignore="300">

    <if_matched_sid>100352</if_matched_sid>
    <same_srcip />
    <same_dstip />

    <description>
      Network: Repeated blocked RDP connection attempts from
$(srcip) to $(dstip).
    </description>

    <group>
      network,
      rdp,
      brute_force_candidate,
      connection_burst,
    </group>

    <mitre>
      <id>T1021.001</id>
    </mitre>
  </rule>
```

```
  <!-- SMB -->
  <rule id="100354" level="3">
    <if_sid>100301</if_sid>
    <dstport>445</dstport>
    <description>
      Network: SMB connection attempt blocked.
    </description>
    <group>network,smb,remote_service,</group>
  </rule>


  <rule id="100355"
        level="8"
        frequency="10"
        timeframe="120"
        ignore="300">

    <if_matched_sid>100354</if_matched_sid>
    <same_srcip />
    <same_dstip />

    <description>
      Network: Repeated blocked SMB connection attempts from
$(srcip) to $(dstip).
    </description>

    <group>
      network,
      smb,
      lateral_movement_candidate,
      connection_burst,
    </group>

    <mitre>
      <id>T1021.002</id>
    </mitre>
```

```
  </rule>


  <!--
============================================================== -->
  <!-- 100360 - SENSITIVE DESTINATION PORTS
-->
  <!--
============================================================== -->

  <rule id="100360" level="5">
    <if_sid>100300,100301</if_sid>
    <dstport>21</dstport>
    <description>
      Network: FTP connection attempt observed.
    </description>
    <group>network,ftp,</group>
  </rule>


  <rule id="100361" level="5">
    <if_sid>100300,100301</if_sid>
    <dstport>23</dstport>
    <description>
      Network: Telnet connection attempt observed.
    </description>
    <group>network,telnet,cleartext_protocol,</group>
  </rule>


  <rule id="100362" level="5">
    <if_sid>100300,100301</if_sid>
    <dstport>445</dstport>
    <description>
      Network: SMB connection attempt observed.
    </description>
```

```
    <group>network,smb,</group>
  </rule>


  <rule id="100363" level="5">
    <if_sid>100300,100301</if_sid>
    <dstport>3389</dstport>
    <description>
      Network: RDP connection attempt observed.
    </description>
    <group>network,rdp,</group>
  </rule>


  <rule id="100364" level="5">
    <if_sid>100300,100301</if_sid>
    <dstport>5900</dstport>
    <description>
      Network: VNC connection attempt observed.
    </description>
    <group>network,vnc,remote_service,</group>
  </rule>


  <!--
============================================================ -->
  <!-- 100370 - MULTI-PORT / MULTI-HOST CORRELATION
-->
  <!--
============================================================ -->

  <!--
    Same source contacting multiple hosts AND multiple ports.
    This is a stronger indicator than either condition
independently.
  -->
```

```
  <rule id="100370"
        level="12"
        frequency="20"
        timeframe="180"
        ignore="600">

    <if_matched_sid>100300,100301</if_matched_sid>
    <same_srcip />

    <description>
      Network: Broad network reconnaissance pattern from $(srcip).
    </description>

    <group>
      network,
      network_scan,
      reconnaissance,
      high_confidence_network_detection,
    </group>

    <mitre>
      <id>T1046</id>
    </mitre>
  </rule>


  <!--
============================================================ -->
  <!-- 100380 - OUTBOUND CONNECTION ANOMALIES
-->
  <!--
============================================================ -->

  <!-- Generic outbound connection candidate -->
  <rule id="100380" level="3">
```

```
    <if_sid>100300</if_sid>
    <description>
      Network: Outbound network connection observed for
correlation.
    </description>
    <group>network,outbound_connection,</group>
  </rule>


  <!-- High volume outbound connections from one source -->
  <rule id="100381"
        level="7"
        frequency="100"
        timeframe="300"
        ignore="300">

    <if_matched_sid>100380</if_matched_sid>
    <same_srcip />

    <description>
      Network: High-volume outbound connection activity from
$(srcip).
    </description>

    <group>
      network,
      outbound_connection,
      connection_burst,
      suspicious_network_activity,
    </group>
  </rule>


  <!--
============================================================= -->
```

```
  <!-- 100390 - CORRELATION / ESCALATION
-->
  <!--
============================================================== -->

  <!--
    Network scan followed by a successful connection.

    The first rule represents reconnaissance.
    The second event represents successful communication.
  -->

  <rule id="100390"
        level="12"
        timeframe="300">

    <if_matched_sid>100331,100341</if_matched_sid>
    <match>accepted|allowed|established|successful</match>

    <description>
      Network: Network reconnaissance followed by successful
connection.
    </description>

    <group>
      network,
      reconnaissance,
      successful_connection,
      correlated_detection,
    </group>

    <mitre>
      <id>T1046</id>
    </mitre>
  </rule>
```

```
  <!--
    Repeated network activity followed by successful SSH
connection.
  -->

  <rule id="100391"
        level="12"
        timeframe="300">

    <if_matched_sid>100351</if_matched_sid>
    <match>accepted|allowed|established|successful</match>
    <dstport>22</dstport>

    <description>
      Network: Repeated SSH connection attempts followed by
successful connection.
    </description>

    <group>
      network,
      ssh,
      brute_force,
      correlated_detection,
    </group>

    <mitre>
      <id>T1021.004</id>
    </mitre>
  </rule>


  <!--
    Repeated RDP connection attempts followed by successful
connection.
  -->
```

```
  <rule id="100392"
        level="12"
        timeframe="300">

    <if_matched_sid>100353</if_matched_sid>
    <match>accepted|allowed|established|successful</match>
    <dstport>3389</dstport>

    <description>
      Network: Repeated RDP connection attempts followed by
successful connection.
    </description>

    <group>
      network,
      rdp,
      brute_force,
      correlated_detection,
    </group>

    <mitre>
      <id>T1021.001</id>
    </mitre>
  </rule>


  <!--
============================================================ -->
  <!-- 100395 - HIGH-CONFIDENCE NETWORK INCIDENT
-->
  <!--
============================================================ -->

  <!--
    Multiple indicators within a short period.
```

```
    This is deliberately high severity because it represents
    behavioral correlation rather than a single network event.
  -->

  <rule id="100395"
        level="13"
        timeframe="300">

    <if_matched_sid>100370,100390,100391,100392</if_matched_sid>

    <description>
      Network: High-confidence correlated network attack activity
detected.
    </description>

    <group>
      network,
      attack_chain,
      correlated_detection,
      high_confidence,
    </group>

    <mitre>
      <id>T1046</id>
    </mitre>
  </rule>


  <!--
============================================================ -->
  <!-- 100399 - FLOOD PROTECTION
-->
  <!--
============================================================ -->
```

```
  <!--
    Prevent the final high-volume rule from generating repeated
    alerts for the same underlying activity.
  -->

  <rule id="100399"
        level="12"
        frequency="100"
        timeframe="60"
        ignore="600">

    <if_matched_sid>100301</if_matched_sid>
    <same_srcip />

    <description>
      Network: Extreme connection-denial volume from source
$(srcip).
    </description>

    <group>
      network,
      network_flood,
      alert_fatigue_control,
      high_volume,
    </group>
  </rule>

</group>
```

Important correction before production

The XML above is a **template intended to be adapted to your actual decoder output**, not something I would blindly deploy. Wazuh's rule engine operates on decoded fields; for example, `same_srcip` requires the event to have a decoded `srcip`, and `dstport` similarly depends on the destination-port field being decoded. (Wazuh Documentation)

Also, the CDB suppression rules require the corresponding lists to be registered under the Wazuh `<ruleset>` configuration. Wazuh documents the CDB-list mechanism and recommends storing lists under `/var/ossec/etc/lists`. (Wazuh Documentation)

Required CDB lists

For the file above, create:

```
None
/var/ossec/etc/lists/approved-network-scanners
/var/ossec/etc/lists/approved-network-monitors
```

Example:

```
None
10.10.10.20:
10.10.10.21:
10.10.20.10:
```

Then register them in `ossec.conf`:

```
XML
<ruleset>
    ...
    <list>etc/lists/approved-network-scanners</list>
    <list>etc/lists/approved-network-monitors</list>
    ...
</ruleset>
```

Wazuh requires the manager to be restarted after adding or modifying CDB lists. (Wazuh Documentation)

Recommended alert flow

The important part of this file is the progression:

```
None
Individual connection
```



That means **100 blocked connections don't become 100 SOC tickets**. They become one behavioral detection.

Validation

Before restarting the manager, validate the rules with Wazuh's rule testing utility:

```shell
/var/ossec/bin/wazuh-logtest
```

Feed it representative firewall/network events and verify that the decoder actually produces:

```
None
srcip
dstip
srcport
dstport
protocol
action
status
```

Then validate the rule hierarchy:

```
None
100300/100301
      ↓
100320–100321
      ↓
100331 / 100341
      ↓
100370
      ↓
100390–100392
      ↓
100395
```

Wazuh's current rule syntax supports the `frequency`/`timeframe` aggregation model, `if_matched_sid`, dynamic source/destination correlation fields, and `ignore` flood protection used here. (Wazuh Documentation)

One thing I would change for an actual deployment: if you tell me whether your network telemetry is **iptables/UFW, Windows Firewall, Suricata, Zeek, Palo Alto, FortiGate, Cisco ASA, or another source**, I can make `040-network.xml` decoder-specific instead of using generic network-event assumptions.

Enhanced 030-persistence

For production, I would make `030-persistence.xml` **FIM-centric and correlation-oriented**, rather than attempting to detect persistence only from process command lines.

The current Wazuh ruleset already exposes registry changes through the `syscheck_registry_*` decoders/rules, including registry-key/value creation, modification, and deletion. Wazuh's FIM configuration supports Windows registry monitoring with `windows_registry`, including architecture selection and tags. (GitHub)

The file below assumes your Windows agents are monitoring the persistence locations in `ossec.conf`. It uses custom IDs `100300–100349`.

/var/ossec/etc/rules/030-persistence.xml

XML

```
<!--
  Wazuh Production Persistence Detection Rules
  File: 030-persistence.xml

  Purpose:
    Detect Windows persistence mechanisms using:
      - Wazuh Syscheck/FIM registry telemetry
      - Windows event telemetry
      - Process telemetry from 020-process.xml

  Custom rule range:
    100300 - 100349

  Design principles:
    1. Registry changes are not automatically malicious.
    2. Persistence location + suspicious payload = high confidence.
    3. Persistence + suspicious process execution = higher
confidence.
    4. Repeated changes are aggregated rather than emitted
individually.
    5. Approved software deployment should be handled by explicit
exceptions.
-->
```

```
<group name="windows,persistence,custom_persistence,">


    <!--
=============================================================
         1. BASE REGISTRY PERSISTENCE EVENTS

============================================================= -->

    <!-- Registry value added -->
    <rule id="100300" level="5">
        <decoded_as>syscheck_registry_value_added</decoded_as>
        <description>Persistence - Windows registry value
added</description>

<group>persistence,registry,syscheck_registry,persistence_low_confi
dence,</group>
    </rule>


    <!-- Registry value modified -->
    <rule id="100301" level="5">
        <decoded_as>syscheck_registry_value_modified</decoded_as>
        <description>Persistence - Windows registry value
modified</description>

<group>persistence,registry,syscheck_registry,persistence_low_confi
dence,</group>
    </rule>


    <!-- Registry value deleted -->
    <rule id="100302" level="4">
        <decoded_as>syscheck_registry_value_deleted</decoded_as>
        <description>Persistence - Windows registry value
deleted</description>
```

```
        <group>persistence,registry,syscheck_registry,</group>
    </rule>


    <!-- Registry key added -->
    <rule id="100303" level="5">
        <decoded_as>syscheck_registry_key_added</decoded_as>
        <description>Persistence - Windows registry key
added</description>

<group>persistence,registry,syscheck_registry,persistence_low_confi
dence,</group>
    </rule>


    <!-- Registry key modified -->
    <rule id="100304" level="5">
        <decoded_as>syscheck_registry_key_modified</decoded_as>
        <description>Persistence - Windows registry key
modified</description>

<group>persistence,registry,syscheck_registry,persistence_low_confi
dence,</group>
    </rule>


    <!-- Registry key deleted -->
    <rule id="100305" level="4">
        <decoded_as>syscheck_registry_key_deleted</decoded_as>
        <description>Persistence - Windows registry key
deleted</description>
        <group>persistence,registry,syscheck_registry,</group>
    </rule>
```

```
    <!--
============================================================
         2. RUN / RUNONCE PERSISTENCE

============================================================ -->

    <!-- HKLM Run -->
    <rule id="100310" level="10">
        <if_sid>100300,100301</if_sid>
        <field name="file"
type="pcre2">(?i)^HKEY_LOCAL_MACHINE\\Software\\Microsoft\\Windows\
\CurrentVersion\\Run(\\|$)</field>
        <description>Persistence - HKLM Run key
changed</description>
        <mitre>
            <id>T1547.001</id>
        </mitre>

<group>persistence,registry,run_keys,mitre_T1547.001,</group>
    </rule>


    <!-- HKLM RunOnce -->
    <rule id="100311" level="10">
        <if_sid>100300,100301</if_sid>
        <field name="file"
type="pcre2">(?i)^HKEY_LOCAL_MACHINE\\Software\\Microsoft\\Windows\
\CurrentVersion\\RunOnce(\\|$)</field>
        <description>Persistence - HKLM RunOnce key
changed</description>
        <mitre>
            <id>T1547.001</id>
        </mitre>

<group>persistence,registry,run_keys,mitre_T1547.001,</group>
    </rule>
```

```
    <!-- HKCU Run -->
    <rule id="100312" level="9">
        <if_sid>100300,100301</if_sid>
        <field name="file"
type="pcre2">(?i)^HKEY_USERS\\[^\\]+\\Software\\Microsoft\\Windows\
\CurrentVersion\\Run(\\|$)</field>
        <description>Persistence - User Run key
changed</description>
        <mitre>
            <id>T1547.001</id>
        </mitre>

<group>persistence,registry,run_keys,mitre_T1547.001,</group>
    </rule>


    <!-- HKCU RunOnce -->
    <rule id="100313" level="9">
        <if_sid>100300,100301</if_sid>
        <field name="file"
type="pcre2">(?i)^HKEY_USERS\\[^\\]+\\Software\\Microsoft\\Windows\
\CurrentVersion\\RunOnce(\\|$)</field>
        <description>Persistence - User RunOnce key
changed</description>
        <mitre>
            <id>T1547.001</id>
        </mitre>

<group>persistence,registry,run_keys,mitre_T1547.001,</group>
    </rule>


    <!--
============================================================
```

```
        3. WINLOGON PERSISTENCE

============================================================ -->

    <rule id="100314" level="13">
        <if_sid>100300,100301</if_sid>
        <field name="file"
type="pcre2">(?i)^HKEY_LOCAL_MACHINE\\Software\\Microsoft\\Windows
NT\\CurrentVersion\\Winlogon(\\|$)</field>
        <description>Persistence - Winlogon registry configuration
changed</description>
        <mitre>
            <id>T1547.004</id>
        </mitre>

<group>persistence,registry,winlogon,high_confidence,mitre_T1547.00
4,</group>
    </rule>


    <!--
============================================================
        4. USERINIT PERSISTENCE

============================================================ -->

    <rule id="100315" level="13">
        <if_sid>100300,100301</if_sid>
        <field name="file"
type="pcre2">(?i)^HKEY_LOCAL_MACHINE\\Software\\Microsoft\\Windows
NT\\CurrentVersion\\Winlogon(\\|$)</field>
        <field name="data"
type="pcre2">(?i)(Userinit|Shell)</field>
        <description>Persistence - Winlogon Userinit or Shell
configuration changed</description>
        <mitre>
```

```
            <id>T1547.004</id>
        </mitre>

<group>persistence,registry,winlogon,high_confidence,mitre_T1547.004,</group>
    </rule>


    <!--
============================================================
         5. IFEO PERSISTENCE

============================================================ -->

    <rule id="100316" level="13">
        <if_sid>100300,100301,100303,100304</if_sid>
        <field name="file"
type="pcre2">(?i)^HKEY_LOCAL_MACHINE\\Software\\Microsoft\\Windows NT\\CurrentVersion\\Image File Execution Options\\</field>
        <description>Persistence - Image File Execution Options registry change</description>
        <mitre>
            <id>T1546.012</id>
        </mitre>

<group>persistence,registry,ifeo,high_confidence,mitre_T1546.012,</group>
    </rule>


    <!--
============================================================
         6. APPINIT_DLLS

============================================================ -->
```

```
    <rule id="100317" level="13">
        <if_sid>100300,100301</if_sid>
        <field name="file" type="pcre2">(?i)AppInit_DLLs</field>
        <description>Persistence - AppInit DLL configuration
changed</description>
        <mitre>
            <id>T1546.010</id>
        </mitre>

<group>persistence,registry,dll,persistence_high_confidence,mitre_T
1546.010,</group>
    </rule>


    <!--
============================================================
         7. ACTIVE SETUP

============================================================ -->

    <rule id="100318" level="11">
        <if_sid>100300,100301,100303,100304</if_sid>
        <field name="file"
type="pcre2">(?i)^HKEY_LOCAL_MACHINE\\Software\\Microsoft\\Active
Setup\\Installed Components\\</field>
        <description>Persistence - Active Setup registry
configuration changed</description>
        <mitre>
            <id>T1547.014</id>
        </mitre>

<group>persistence,registry,active_setup,mitre_T1547.014,</group>
    </rule>
```

```
    <!--
=============================================================
         8. BROWSER / SHELL EXTENSION PERSISTENCE

============================================================= -->

    <rule id="100319" level="11">
        <if_sid>100300,100301,100303,100304</if_sid>
        <field name="file" type="pcre2">(?i)(Browser Helper
Objects|Shell Extensions|Explorer\\ShellExecuteHooks)</field>
        <description>Persistence - Browser or Explorer shell
extension registry change</description>
        <mitre>
            <id>T1547</id>
        </mitre>
        <group>persistence,registry,shell_extension,</group>
    </rule>


    <!--
=============================================================
         9. SERVICES

============================================================= -->

    <!-- Registry service creation/modification -->
    <rule id="100320" level="11">
        <if_sid>100300,100301,100303,100304</if_sid>
        <field name="file"
type="pcre2">(?i)^HKEY_LOCAL_MACHINE\\System\\CurrentControlSet\\Se
rvices\\[^\\]+(\\|$)</field>
        <description>Persistence - Windows service registry
configuration changed</description>
        <mitre>
            <id>T1543.003</id>
        </mitre>
```

```
<group>persistence,registry,windows_service,mitre_T1543.003,</group
>
    </rule>


    <!-- Service ImagePath -->
    <rule id="100321" level="13">
        <if_sid>100300,100301</if_sid>
        <field name="file"
type="pcre2">(?i)^HKEY_LOCAL_MACHINE\\System\\CurrentControlSet\\Se
rvices\\[^\\]+\\ImagePath$</field>
        <description>Persistence - Windows service ImagePath
changed</description>
        <mitre>
            <id>T1543.003</id>
        </mitre>

<group>persistence,registry,windows_service,high_confidence,mitre_T
1543.003,</group>
    </rule>


    <!-- Service Start configuration -->
    <rule id="100322" level="10">
        <if_sid>100300,100301</if_sid>
        <field name="file"
type="pcre2">(?i)^HKEY_LOCAL_MACHINE\\System\\CurrentControlSet\\Se
rvices\\[^\\]+\\Start$</field>
        <description>Persistence - Windows service Start
configuration changed</description>
        <mitre>
            <id>T1543.003</id>
        </mitre>
```

```
<group>persistence,registry,windows_service,mitre_T1543.003,</group
>
    </rule>


    <!--
============================================================
         10. SCHEDULED TASK REGISTRY PERSISTENCE

============================================================ -->

    <rule id="100323" level="10">
        <if_sid>100300,100301,100303,100304</if_sid>
        <field name="file"
type="pcre2">(?i)^HKEY_LOCAL_MACHINE\\Software\\Microsoft\\Windows
NT\\CurrentVersion\\Schedule\\TaskCache\\</field>
        <description>Persistence - Scheduled Task registry state
changed</description>
        <mitre>
            <id>T1053.005</id>
        </mitre>

<group>persistence,registry,scheduled_task,mitre_T1053.005,</group>
    </rule>


    <!--
============================================================
         11. SECURITY PROVIDER / LSA PERSISTENCE

============================================================ -->

    <rule id="100324" level="14">
        <if_sid>100300,100301,100303,100304</if_sid>
```

```
        <field name="file" type="pcre2">(?i)(Security
Packages|Authentication Packages|Notification
Packages|LSA\\Extensions|Lsa\\)</field>
        <description>Persistence - LSA or authentication provider
registry configuration changed</description>
        <mitre>
            <id>T1547.005</id>
        </mitre>

<group>persistence,registry,lsa,credential_access,high_confidence,m
itre_T1547.005,</group>
    </rule>


    <!--
============================================================
         12. PRINT MONITOR PERSISTENCE

============================================================ -->

    <rule id="100325" level="14">
        <if_sid>100300,100301,100303,100304</if_sid>
        <field name="file"
type="pcre2">(?i)\\Control\\Print\\Monitors\\</field>
        <description>Persistence - Windows print monitor
configuration changed</description>
        <mitre>
            <id>T1547.010</id>
        </mitre>

<group>persistence,registry,print_monitor,high_confidence,mitre_T15
47.010,</group>
    </rule>
```

```
    <!--
=============================================================
         13. SECURITY SUPPORT PROVIDER

============================================================= -->

    <rule id="100326" level="14">
        <if_sid>100300,100301,100303,100304</if_sid>
        <field name="file"
type="pcre2">(?i)\\Control\\Lsa\\Security Packages</field>
        <description>Persistence - Security Support Provider
registry configuration changed</description>
        <mitre>
            <id>T1547.005</id>
        </mitre>

<group>persistence,registry,ssp,credential_access,high_confidence,<
/group>
    </rule>


    <!--
=============================================================
         14. STARTUP FOLDER / START MENU

============================================================= -->

    <rule id="100327" level="8">
        <if_sid>100020</if_sid>
        <field name="win.eventdata.image"
type="pcre2">(?i)\\Users\\[^\\]+\\AppData\\Roaming\\Microsoft\\Wind
ows\\Start Menu\\Programs\\Startup\\.+</field>
        <description>Persistence - Process associated with user
Startup folder</description>
        <mitre>
            <id>T1547.001</id>
```

```
        </mitre>

<group>persistence,startup_folder,process_correlation,</group>
    </rule>


    <!--
============================================================
         15. PERSISTENCE FROM USER-WRITABLE LOCATIONS

============================================================ -->

    <rule id="100328" level="11">

<if_sid>100310,100311,100312,100313,100314,100316,100317,100318,100
320,100321</if_sid>
        <field name="data"
type="pcre2">(?i)(\\\\Users\\\\[^\\]+\\\\(AppData|Downloads|Desktop
|Temp)|\\\\Windows\\\\Temp|\\\\ProgramData\\\\Temp)</field>
        <description>Persistence - Registry persistence points to
user-writable or temporary location</description>
        <mitre>
            <id>T1036</id>
            <id>T1547</id>
        </mitre>

<group>persistence,user_writable_path,defense_evasion,high_confiden
ce,</group>
    </rule>


    <!--
============================================================
         16. SCRIPT / COMMAND INTERPRETER PERSISTENCE

============================================================ -->
```

```
    <rule id="100329" level="12">

<if_sid>100310,100311,100312,100313,100314,100316,100318,100320,100
321</if_sid>
        <field name="data"
type="pcre2">(?i)(powershell|pwsh|cmd\.exe|wscript|cscript|mshta|ru
ndll32|regsvr32)</field>
        <description>Persistence - Registry persistence references
command or script interpreter</description>
        <mitre>
            <id>T1059</id>
            <id>T1547</id>
        </mitre>

<group>persistence,command_and_scripting,high_confidence,</group>
    </rule>


    <!--
============================================================
         17. REMOTE / NETWORK PAYLOAD IN PERSISTENCE

============================================================ -->

    <rule id="100330" level="13">

<if_sid>100310,100311,100312,100313,100314,100316,100318,100320,100
321</if_sid>
        <field name="data"
type="pcre2">(?i)(https?://|ftp://|\\\\[0-9]{1,3}(\.[0-9]{1,3}){3}\
\)</field>
        <description>Persistence - Registry persistence references
remote payload</description>
        <mitre>
            <id>T1105</id>
```

```
            <id>T1547</id>
        </mitre>

<group>persistence,remote_payload,ingress_tool_transfer,high_confid
ence,</group>
    </rule>


    <!--
============================================================
         18. POWERSHELL PERSISTENCE CORRELATION

============================================================ -->

    <!--
      Depends on 020-process.xml.
      A suspicious process followed by a persistence change is
      considerably stronger than either event alone.
    -->
    <rule id="100331" level="13" frequency="2" timeframe="600">

<if_matched_group>process_high_confidence</if_matched_group>
        <same_field>agent.id</same_field>
        <description>Persistence - Suspicious process activity
correlated with persistence activity</description>
        <mitre>
            <id>T1059</id>
            <id>T1547</id>
        </mitre>

<group>persistence,process_correlation,behavior_chain,high_confiden
ce,</group>
    </rule>
```

```
    <!--
============================================================
         19. MULTIPLE PERSISTENCE LOCATIONS

============================================================ -->

    <rule id="100332" level="14" frequency="3" timeframe="900">
        <if_matched_group>persistence</if_matched_group>
        <description>Persistence - Multiple persistence mechanisms
changed within 15 minutes</description>
        <mitre>
            <id>T1547</id>
            <id>T1053.005</id>
            <id>T1543.003</id>
        </mitre>

<group>persistence,correlation,multiple_persistence,high_confidence
,</group>
    </rule>


    <!--
============================================================
         20. PERSISTENCE BURST / ALERT FATIGUE CONTROL

============================================================ -->

    <!--
      Aggregate repeated changes from the same host.
      This prevents software installers or configuration management
      from producing hundreds of persistence alerts.
    -->
    <rule id="100333" level="9" frequency="10" timeframe="300">

<if_matched_group>persistence_low_confidence</if_matched_group>
```

```
        <description>Persistence - Repeated registry persistence
changes aggregated</description>

<group>persistence,aggregation,alert_fatigue_control,</group>
    </rule>


    <!--
============================================================
         21. HIGH-CONFIDENCE PERSISTENCE BURST

============================================================ -->

    <rule id="100334" level="14" frequency="3" timeframe="600">

<if_matched_group>persistence_high_confidence</if_matched_group>
        <description>Persistence - Multiple high-confidence
persistence changes detected</description>
        <mitre>
            <id>T1547</id>
        </mitre>

<group>persistence,correlation,high_confidence,alert_fatigue_contro
l,</group>
    </rule>


    <!--
============================================================
         22. DELETION / CLEANUP AFTER PERSISTENCE

============================================================ -->

    <rule id="100335" level="13" frequency="2" timeframe="600">
        <if_matched_group>persistence</if_matched_group>
```

```
        <description>Persistence - Repeated persistence
modification activity may indicate cleanup or evasion</description>
        <mitre>
            <id>T1070</id>
            <id>T1547</id>
        </mitre>

<group>persistence,defense_evasion,indicator_removal,correlation,</
group>
    </rule>


    <!--
============================================================
         23. CRITICAL PERSISTENCE LOCATIONS

============================================================ -->

    <rule id="100336" level="15">
        <if_sid>100300,100301,100303,100304</if_sid>
        <field name="file"
type="pcre2">(?i)(\\Winlogon\\|\\LSA\\|\\Security
Packages|\\Authentication Packages|\\Print\\Monitors\\|Image File
Execution Options\\)</field>
        <description>Persistence - Critical Windows persistence
location modified</description>
        <mitre>
            <id>T1547</id>
        </mitre>

<group>persistence,critical_persistence,high_confidence,</group>
    </rule>


    <!--
============================================================
```

```
        24. PERSISTENCE + USER-WRITABLE PAYLOAD

============================================================== -->

    <rule id="100337" level="15">
        <if_sid>100300,100301</if_sid>
        <field name="file"
type="pcre2">(?i)(CurrentVersion\\Run|CurrentVersion\\RunOnce|Winlo
gon|Services\\|Image File Execution Options)</field>
        <field name="data"
type="pcre2">(?i)(Users\\[^\\]+\\AppData|Users\\[^\\]+\\Downloads|U
sers\\[^\\]+\\Desktop|Windows\\Temp|ProgramData\\Temp)</field>
        <description>Persistence - High-risk registry persistence
points to user-writable location</description>
        <mitre>
            <id>T1547</id>
            <id>T1036</id>
        </mitre>

<group>persistence,critical_persistence,user_writable_path,high_con
fidence,</group>
    </rule>


    <!--
============================================================
        25. FINAL INCIDENT-LEVEL CORRELATION

============================================================== -->

    <!--
      This is deliberately restrictive.

      Trigger only when:
        - high-confidence process activity exists
        - high-confidence persistence activity exists
```

```
        - both occur on the same agent within 15 minutes

      This is intended to become the SOC-facing alert rather than
      every individual underlying event.
    -->
    <rule id="100338" level="15" frequency="2" timeframe="900">

<if_matched_group>process_high_confidence</if_matched_group>
        <description>Incident - Suspicious execution followed by
persistence activity</description>
        <mitre>
            <id>T1059</id>
            <id>T1547</id>
        </mitre>

<group>incident,execution,persistence,behavior_chain,high_confidenc
e,</group>
    </rule>


</group>
```

Important production adjustment

There is one architectural issue I would correct before deploying the file exactly as above: **Wazuh's temporal correlation operates on rules that have already matched, and the correlation syntax needs to be designed around specific prior rules/groups rather than treating an arbitrary combination as a guaranteed sequence.** Wazuh documents `if_matched_sid`/`if_matched_group` together with `frequency` and `timeframe` for this purpose. (Wazuh Documentation)

So I recommend making the correlation layer explicit.

For example, add this to the **high-confidence rules** in `020-process.xml`:

XML
```
<group>process_high_confidence,</group>
```

Then use persistence rules with:

```xml
<group>persistence_high_confidence,</group>
```

and reserve `100338` for your final correlation layer after you have verified the exact event ordering and fields in `wazuh-logtest`.

Why I deliberately separated the tiers

```
Registry change
      |
      └── P3/P4-style telemetry
              |
              ▼
Known persistence location
              |
              └── P2/P1
                      |
                      ▼
Persistence → user-writable payload
                      |
                      └── P1
                              |
                              ▼
Suspicious process
       +
Persistence
       +
same endpoint / time window
                              |
                              └── P0/P1 incident
```

This is the alert-fatigue control: **don't make every registry modification a high-severity alert.**

Wazuh's own base FIM rules classify generic registry value/key additions and modifications at relatively modest levels, which makes them suitable as the telemetry layer on which these more specific rules can build. (GitHub)

Required Windows FIM configuration

The rules alone cannot detect registry persistence if the endpoints are not monitoring the relevant registry locations. Wazuh supports `<windows_registry>` entries and the `arch="both"` attribute for monitoring both registry views. (Wazuh Documentation)

For the Windows agents, I recommend monitoring at least:

```xml
<syscheck>
    <disabled>no</disabled>

    <windows_registry arch="both">

HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Run
    </windows_registry>

    <windows_registry arch="both">

HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\RunOnc
e
    </windows_registry>

    <windows_registry arch="both">
        HKEY_LOCAL_MACHINE\Software\Microsoft\Windows
NT\CurrentVersion\Winlogon
    </windows_registry>

    <windows_registry arch="both">
        HKEY_LOCAL_MACHINE\Software\Microsoft\Windows
NT\CurrentVersion\Image File Execution Options
    </windows_registry>

    <windows_registry arch="both">
        HKEY_LOCAL_MACHINE\System\CurrentControlSet\Services
    </windows_registry>

    <windows_registry arch="both">
```

```
        HKEY_LOCAL_MACHINE\Software\Microsoft\Active
Setup\Installed Components
    </windows_registry>

    <windows_registry arch="both">

HKEY_LOCAL_MACHINE\System\CurrentControlSet\Control\Print\Monitors
    </windows_registry>

    <windows_registry arch="both">
        HKEY_LOCAL_MACHINE\System\CurrentControlSet\Control\Lsa
    </windows_registry>
</syscheck>
```

Wazuh's current Windows configuration itself demonstrates registry monitoring for locations such as `RunOnce`, `RunOnceEx`, `Winlogon`, `Policies`, and Active Setup. (GitHub)

Validate before production

On the manager:

```
Shell
sudo /var/ossec/bin/wazuh-analysisd -t
```

Then:

```
Shell
sudo /var/ossec/bin/wazuh-logtest
```

Test, at minimum:

1. benign software installation → should remain low-volume;
2. HKCU/HKLM Run modification → should trigger the appropriate Run rule;
3. Winlogon modification → high severity;
4. service `ImagePath` modification → high severity;
5. IFEO modification → high severity;
6. persistence pointing to `%TEMP%`/`AppData` → higher severity;

7. legitimate enterprise deployment → verify it does not flood the SOC;
8. suspicious process + persistence → verify correlation.

Do not put broad permanent allowlists into this file. Keep environment-specific exceptions in a separate `090-exceptions.xml` so they can have an owner, justification, expiration date, and review cycle.

The Wazuh XML rule syntax, including `if_sid`, `if_matched_sid`, `if_matched_group`, `frequency`, `timeframe`, `same_field`, and `ignore`, is documented in the current rules reference. (Wazuh Documentation)

050-malware.xml

Below is a production-oriented `050-malware.xml` for the architecture we discussed. It is designed to cover **FIM malware indicators, YARA detections, known-malware hashes, suspicious file locations, malware execution chains, and alert-fatigue aggregation**.

Wazuh recommends custom rule IDs in the `100000–120000` range and custom rule files under `/var/ossec/etc/rules/`; files under `/var/ossec/ruleset/rules/` can be overwritten during upgrades. (Wazuh Documentation)

One important caveat: **the FIM/YARA rules below use documented Wazuh fields and rule IDs, while the process/EDR correlation section is intentionally based on Wazuh rule groups rather than assuming a particular Windows Sysmon/EDR decoder schema.** You should validate the parent rule/group names in your installation with `wazuh-logtest`. Wazuh's rule engine supports `if_sid`, `if_group`, `if_matched_sid`, frequency/timeframe correlation, `same_*` conditions, CDB lookups, and PCRE2. (Wazuh Documentation)

/var/ossec/etc/rules/050-malware.xml

```xml
<!--
  WAZUH CUSTOM MALWARE DETECTION RULES
  File: 050-malware.xml

  Purpose:
    - Malware / IOC detection
    - FIM-based malware detection
    - YARA integration
    - Known-malware hash detection
    - Suspicious file-location detection
    - Malware event aggregation
    - Malware execution correlation
    - Alert-fatigue reduction

  Custom rule ID range used here:
    100500 - 100599

  IMPORTANT:
    1. Do not edit /var/ossec/ruleset/rules/.
    2. Place this file under /var/ossec/etc/rules/.
```

```
    3. Ensure referenced CDB lists exist before enabling those
rules.
    4. Validate every rule with wazuh-logtest.
-->

<group name="malware,malware_detection,">

  <!-- ============================================================
-->
  <!-- 01. FIM BASELINE - FILE CREATION / MODIFICATION
-->
  <!-- ============================================================
-->

  <!--
    Wazuh built-in:
      554 = file added
      550 = file modified

    These are intentionally level 5-7 events.
    They become higher priority only when additional malware
    evidence exists.
  -->

  <rule id="100500" level="5">
    <if_sid>554</if_sid>
    <description>Malware monitoring: new file detected on monitored
endpoint: $(file)</description>
    <group>malware_fim,file_creation,</group>
  </rule>

  <rule id="100501" level="5">
    <if_sid>550</if_sid>
    <description>Malware monitoring: monitored file modified on
endpoint: $(file)</description>
    <group>malware_fim,file_modification,</group>
```

```
  </rule>


  <!-- ============================================================
-->
  <!-- 02. HIGH-RISK FILE LOCATIONS - LINUX
-->
  <!-- ============================================================
-->

  <!--
    These rules DO NOT claim that every file in these locations
    is malware.

    They identify locations frequently worth additional
    investigation.

    Tune these paths for your environment.
  -->

  <rule id="100510" level="8">
    <if_sid>554,550</if_sid>
    <field name="file" type="pcre2">
      (?i)^/(tmp|var/tmp|dev/shm)/[^ ]+
    </field>
    <description>
      Suspicious file activity in Linux temporary/shared-memory
location: $(file)
    </description>
    <mitre>
      <id>T1204</id>
      <id>T1059</id>
    </mitre>
    <group>malware_suspicious_path,linux,</group>
  </rule>
```

```
  <rule id="100511" level="8">
    <if_sid>554,550</if_sid>
    <field name="file" type="pcre2">

(?i)^/etc/(cron\.d|cron\.daily|cron\.hourly|cron\.weekly|cron\.mont
hly)/[^ ]+
    </field>
    <description>
      Suspicious modification or creation of Linux cron persistence
file: $(file)
    </description>
    <mitre>
      <id>T1053.003</id>
    </mitre>
    <group>malware_persistence,linux,cron,</group>
  </rule>

  <rule id="100512" level="10">
    <if_sid>554,550</if_sid>
    <field name="file" type="pcre2">
      (?i)^/(usr/local/bin|usr/local/sbin|usr/bin|usr/sbin)/[^ ]+
    </field>
    <description>
      Executable system-path file created or modified: $(file)
    </description>
    <mitre>
      <id>T1105</id>
      <id>T1036</id>
    </mitre>
    <group>malware_suspicious_path,linux,system_binary,</group>
  </rule>


  <!-- ============================================================
-->
```

```
  <!-- 03. HIGH-RISK FILE LOCATIONS - WINDOWS
-->
  <!-- ============================================================
-->

  <rule id="100520" level="8">
    <if_sid>554,550</if_sid>
    <field name="file" type="pcre2">
      (?i)\\(Users|Documents and
Settings)\\[^\\]+\\(Downloads|AppData\\Local\\Temp|AppData\\Roaming
)\\
    </field>
    <description>
      Suspicious Windows user temporary/download file activity:
$(file)
    </description>
    <group>malware_suspicious_path,windows,user_download,</group>
  </rule>

  <rule id="100521" level="9">
    <if_sid>554,550</if_sid>
    <field name="file" type="pcre2">
      (?i)\\Windows\\(Temp|Tasks)\\
    </field>
    <description>
      Suspicious Windows system temporary/task file activity:
$(file)
    </description>
    <mitre>
      <id>T1053</id>
    </mitre>
    <group>malware_suspicious_path,windows,persistence,</group>
  </rule>

  <rule id="100522" level="10">
    <if_sid>554,550</if_sid>
```

```
    <field name="file" type="pcre2">
      (?i)\\Windows\\System32\\[^\\]+\.(exe|dll|sys|scr|com)
    </field>
    <description>
      Executable file created or modified in Windows System32:
$(file)
    </description>
    <group>malware_suspicious_path,windows,system_binary,</group>
  </rule>


  <!-- ============================================================
-->
  <!-- 04. YARA INTEGRATION
-->
  <!-- ============================================================
-->

  <!--
    Expected Wazuh YARA decoder fields:
      yara_rule
      yara_scanned_file
      log_type

    These correspond to the documented Wazuh YARA integration.
  -->

  <rule id="100530" level="0">
    <decoded_as>yara_decoder</decoded_as>
    <description>YARA malware detection grouping
event</description>
    <group>malware,yara,grouping,</group>
  </rule>

  <rule id="100531" level="12">
    <if_sid>100530</if_sid>
```

```
    <match>wazuh-yara: INFO - Scan result:</match>
    <description>
      YARA detected a malware signature in file
$(yara_scanned_file):
      rule=$(yara_rule)
    </description>
    <mitre>
      <id>T1204.002</id>
    </mitre>
    <group>malware,yara,malware_detected,</group>
  </rule>

  <!-- Critical YARA detection for multiple positive signatures -->

  <rule id="100532" level="14" frequency="2" timeframe="300">
    <if_matched_sid>100531</if_matched_sid>
    <same_field>yara_scanned_file</same_field>
    <description>
      Multiple YARA malware detections for the same file within 5
minutes:
      $(yara_scanned_file)
    </description>
    <group>malware,yara,correlation,high_confidence,</group>
  </rule>


  <!-- ============================================================
-->
  <!-- 05. KNOWN MALWARE HASH - CDB
-->
  <!-- ============================================================
-->

  <!--
    Expected CDB:
      /var/ossec/etc/lists/malware-hashes
```

```
    Example entry:
      SHA256_HASH:malware-family

    If you use MD5 instead, change field="md5".
  -->

  <rule id="100540" level="13">
    <if_sid>554,550</if_sid>
    <list field="md5" lookup="match_key">
      etc/lists/malware-hashes
    </list>
    <description>
      Known malware hash detected in newly created or modified
file: $(file)
    </description>
    <mitre>
      <id>T1204.002</id>
    </mitre>
    <group>malware,ioc,hash_match,high_confidence,</group>
  </rule>


  <!-- ============================================================
-->
  <!-- 06. MALWARE HASH ESCALATION
-->
  <!-- ============================================================
-->

  <!--
    Repeated hash detections across files can indicate:
      - malware propagation
      - multiple dropped payloads
      - persistence
      - lateral movement
```

```
    This aggregates repeated detections instead of producing
    a separate high-priority alert for every file.
  -->

  <rule id="100541" level="14" frequency="3" timeframe="600">
    <if_matched_sid>100540</if_matched_sid>
    <description>
      Multiple known-malware hash detections detected within 10
minutes.
    </description>
    <group>malware,ioc,correlation,malware_spread,</group>
  </rule>


  <!-- ============================================================
-->
  <!-- 07. FIM + MALWARE IOC CORRELATION
-->
  <!-- ============================================================
-->

  <!--
    New/modified file followed by known-malware hash detection.
    The hash match is the important evidence, so this rule is
    intentionally high severity.
  -->

  <rule id="100550" level="14">
    <if_sid>100540</if_sid>
    <description>
      Confirmed malware file detected by FIM and malware IOC
correlation:
      $(file)
    </description>
    <group>malware,confirmed_detection,fim,ioc,</group>
```

```
  </rule>


  <!-- ============================================================
-->
  <!-- 08. MALWARE DETECTION BURST / ALERT FATIGUE CONTROL
-->
  <!-- ============================================================
-->

  <!--
    Instead of generating an individual P1/P0 event for every
    malware-related event, aggregate repeated detections.

    NOTE:
    This does NOT suppress the underlying telemetry.
    It creates a higher-level aggregate event.
  -->

  <rule id="100560" level="10" frequency="10" timeframe="300">
    <if_group>malware</if_group>
    <description>
      Malware detection burst: 10 or more malware-related events
      detected within 5 minutes.
    </description>
    <group>malware,correlation,alert_aggregation,</group>
  </rule>

  <rule id="100561" level="12" frequency="25" timeframe="600">
    <if_group>malware</if_group>
    <description>
      High-volume malware detection burst: 25 or more
      malware-related events within 10 minutes.
    </description>

<group>malware,correlation,alert_aggregation,high_volume,</group>
```

```
  </rule>


  <!-- ============================================================
-->
  <!-- 09. MULTIPLE FILES AFFECTED
-->
  <!-- ============================================================
-->

  <!--
    Detect multiple different file events associated with the
    same malware detection context.

    same_field requires a decoded dynamic field. Therefore this
    rule should only be enabled if your event stream exposes
    the expected field consistently.
  -->

  <rule id="100570" level="12" frequency="5" timeframe="600">
    <if_matched_sid>100531</if_matched_sid>
    <description>
      Multiple malware-positive file detections from the same
endpoint
      within 10 minutes.
    </description>
    <group>malware,correlation,multiple_files,</group>
  </rule>


  <!-- ============================================================
-->
  <!-- 10. PERSISTENCE + MALWARE CORRELATION
-->
  <!-- ============================================================
-->
```

```
  <!--
    If a malware detection is followed by persistence activity,
    raise the priority.

    100511 = Linux cron persistence file activity.
  -->

  <rule id="100580" level="15">
    <if_sid>100511</if_sid>
    <if_matched_sid>100531,100540</if_matched_sid>
    <description>
      Malware detection followed by Linux persistence activity.
    </description>
    <mitre>
      <id>T1053.003</id>
    </mitre>
    <group>malware,persistence,correlation,critical,</group>
  </rule>


  <!-- ============================================================
-->
  <!-- 11. SYSTEM BINARY + MALWARE CORRELATION
-->
  <!-- ============================================================
-->

  <rule id="100581" level="15">
    <if_sid>100512,100522</if_sid>
    <if_matched_sid>100531,100540</if_matched_sid>
    <description>
      Malware detection associated with executable system-path
      modification.
    </description>
    <group>malware,system_binary,correlation,critical,</group>
```

```
  </rule>


  <!-- ============================================================
-->
  <!-- 12. HIGH-CONFIDENCE MALWARE
-->
  <!-- ============================================================
-->

  <!--
    YARA positive match OR known malware hash:
      high confidence

    This is intentionally separate from suspicious-path detections.
  -->

  <rule id="100590" level="15">
    <if_sid>100531,100540</if_sid>
    <description>
      High-confidence malware detection requiring immediate
investigation:
      $(file)
    </description>
    <group>malware,high_confidence,incident_candidate,</group>
  </rule>


  <!-- ============================================================
-->
  <!-- 13. CRITICAL MALWARE CORRELATION
-->
  <!-- ============================================================
-->

  <!--
```

```
    Malware detection + persistence or system binary modification.
  -->

  <rule id="100591" level="16">
    <if_sid>100580,100581</if_sid>
    <description>
      Critical malware compromise pattern: malware evidence
combined
      with persistence or system binary modification.
    </description>
    <group>malware,critical,confirmed_compromise,incident,</group>
  </rule>


  <!-- ============================================================
-->
  <!-- 14. DUPLICATE HIGH-CONFIDENCE ALERT CONTROL
-->
  <!-- ============================================================
-->

  <!--
    Prevent repeated identical high-confidence alerts from flooding
    the SOC.

    The first detection generates the alert.
    The rule is then ignored for 300 seconds.
  -->

  <rule id="100595" level="15" ignore="300">
    <if_sid>100590</if_sid>
    <description>
      High-confidence malware detection - duplicate alert
suppression
      window applied.
    </description>
```

```
    <group>malware,alert_fatigue_control,deduplicated,</group>
  </rule>


  <!-- ============================================================
-->
  <!-- 15. FINAL INCIDENT CLASSIFICATION
-->
  <!-- ============================================================
-->

  <!--
    P0-equivalent Wazuh level 16:
      confirmed malware + persistence/system impact.

    Keep this rule narrow.
  -->

  <rule id="100599" level="16">
    <if_sid>100591</if_sid>
    <description>
      CRITICAL: Confirmed malware activity with persistence or
      system-level modification.
    </description>
    <mitre>
      <id>T1053</id>
      <id>T1105</id>
      <id>T1204.002</id>
    </mitre>
    <group>
      malware,critical,p0,confirmed_compromise,incident_response,
    </group>
  </rule>

</group>
```

Important correction before deploying

I would **not deploy that file exactly as-is without testing**, because the available malware telemetry differs substantially between deployments. In particular, `$(file)`, `md5`, `yara_scanned_file`, and `yara_rule` depend on the decoder/event that produced the alert. Wazuh's documented YARA integration, for example, explicitly creates `yara_rule` and `yara_scanned_file` fields, while FIM events expose file information differently. (Wazuh Documentation)

Also, Wazuh's documented CDB malware example uses:

```xml
<list field="md5"
lookup="match_key">etc/lists/malware-hashes</list>
```

for a malware-hash list, so the CDB rule above follows that model. (Wazuh Documentation)

Recommended supporting CDB

Create:

```
/var/ossec/etc/lists/malware-hashes
```

with entries such as:

```
d41d8cd98f00b204e9800998ecf8427e:known-malware-family
098f6bcd4621d373cade4e832627b4f6:known-test-malware
```

Then ensure the list is loaded in the Wazuh `<ruleset>` configuration. Wazuh documents CDB lists specifically for IOC matching, including hash-based malware detection. (Wazuh Documentation)

What I would change for your actual environment

The strongest version of `050-malware.xml` should have **four distinct detection tiers**:

```
None
TIER 1 — Suspicious
    ├── suspicious path
    ├── suspicious extension
    ├── suspicious archive
    └── suspicious executable

TIER 2 — IOC
    ├── malicious hash
    ├── malicious YARA signature
    ├── malicious IP/domain
    └── known malware filename

TIER 3 — Behavioral
    ├── malware → execution
    ├── malware → persistence
    ├── malware → privilege escalation
    └── malware → C2

TIER 4 — Incident
    ├── multiple malware indicators
    ├── malware + persistence
    ├── malware + credential access
    └── malware + lateral movement
```

That distinction is important for **alert-fatigue reduction**: don't make every suspicious file a level-15/16 alert. A YARA-positive or confirmed IOC match should carry substantially more weight than "file appeared in `/tmp`."

Wazuh's own malware documentation covers FIM, rootkit detection, third-party malware logs, YARA, VirusTotal, and custom IOC rules, so `050-malware.xml` should ultimately act as the **correlation/classification layer across those detection sources**, rather than attempting to replace them. (Wazuh Documentation)

Validate it

First run:

```shell
/var/ossec/bin/wazuh-logtest
```

and feed it representative FIM/YARA/IOC events. `wazuh-logtest` uses the same ruleset/decoders as the analysis engine and is specifically intended for validating custom rules. (Wazuh Documentation)

Then:

```shell
chown root:wazuh /var/ossec/etc/rules/050-malware.xml
chmod 640 /var/ossec/etc/rules/050-malware.xml

systemctl restart wazuh-manager
```

Wazuh requires a manager restart for the modified rules to become active for normal alert generation. (Wazuh Documentation)

One thing I would do next: build `060-lateral-movement.xml` so that this malware layer can correlate **malware → credential access → lateral movement → persistence**, producing one incident instead of dozens of Wazuh alerts.

060-lateral-movement.xml

Yes. Below is a production-oriented `060-lateral-movement.xml` for Wazuh. I would treat it as a **custom detection layer**, not a replacement for Wazuh's built-in Windows rules.

Wazuh recommends custom rule IDs in the `100000–120000` range and recommends placing larger custom rule sets under `/var/ossec/etc/rules/` rather than modifying `/var/ossec/ruleset/`, which is overwritten during upgrades. (Wazuh Documentation)

The rules below cover:

- Windows network logons / SMB
- RDP
- explicit-credential use
- privileged remote logons
- administrative share access
- PsExec / remote service execution
- suspicious service creation
- WMI lateral movement
- remote PowerShell / WinRM indicators
- SSH lateral movement
- Linux remote-login bursts
- multi-host lateral movement
- correlation and escalation
- alert deduplication

> **Important:** field names depend on the decoder/log source actually installed in your environment. Test every rule with `wazuh-logtest` before production deployment. Wazuh explicitly recommends this workflow for custom rules. (Wazuh Documentation)

XML

```xml
<!--
  060-lateral-movement.xml

  Purpose:
    Custom Wazuh detections for lateral movement.

  Installation:
    /var/ossec/etc/rules/060-lateral-movement.xml

  Custom rule ID range:
    106000 - 106999
```

```
  Primary ATT&CK techniques:
    T1021       Remote Services
    T1021.001   RDP
    T1021.002   SMB/Windows Admin Shares
    T1021.004   SSH
    T1021.006   Windows Remote Management
    T1047       Windows Management Instrumentation
    T1569.002   System Services: Service Execution
    T1543.003   Windows Service
    T1078       Valid Accounts

  Prerequisites:
    - Windows Security event collection
    - Windows System event collection
    - Sysmon recommended for process/WMI detections
    - Linux auth.log/secure collection for SSH detections
    - Appropriate Windows auditing enabled

  IMPORTANT:
    Verify decoded fields with:
      /var/ossec/bin/wazuh-logtest

    Do not blindly deploy allowlists. Tune the
source/destination/user
    conditions for your environment.
-->

<group name="lateral_movement,windows,linux,">

  <!--
============================================================
       WINDOWS: REMOTE NETWORK LOGON
       Event ID 4624, LogonType 3
       ATT&CK: T1021 / T1078
```

```
==========================================================
-->

  <rule id="106000" level="5">
    <field name="win.system.eventID">^4624$</field>
    <field name="win.eventdata.logonType">^3$</field>
    <field name="win.eventdata.ipAddress"
negate="yes">^(127\.0\.0\.1|::1|-)$</field>
    <description>Windows: Remote network logon detected for
$(win.eventdata.targetUserName) from
$(win.eventdata.ipAddress).</description>
    <mitre>
      <id>T1021</id>
      <id>T1078</id>
    </mitre>

<group>lateral_movement,remote_logon,windows_authentication,</group
>
  </rule>


  <!--
==========================================================
       WINDOWS: RDP LOGON
       Event ID 4624, LogonType 10
       ATT&CK: T1021.001

==========================================================
-->

  <rule id="106001" level="8">
    <field name="win.system.eventID">^4624$</field>
    <field name="win.eventdata.logonType">^10$</field>
    <field name="win.eventdata.ipAddress"
negate="yes">^(127\.0\.0\.1|::1|-)$</field>
```

```
    <description>Windows: Remote Desktop logon by
$(win.eventdata.targetUserName) from
$(win.eventdata.ipAddress).</description>
    <mitre>
      <id>T1021.001</id>
      <id>T1078</id>
    </mitre>

<group>lateral_movement,rdp,remote_access,windows_authentication,</
group>
  </rule>


  <!-- RDP logon using a privileged account -->

  <rule id="106002" level="11">
    <if_sid>106001</if_sid>
    <field name="win.eventdata.targetUserName"

type="pcre2">(?i)^(administrator|admin|domainadmin|domain\.admin|sv
c-admin)$</field>
    <description>Windows: Privileged account used for remote RDP
logon from $(win.eventdata.ipAddress).</description>
    <mitre>
      <id>T1021.001</id>
      <id>T1078</id>
    </mitre>

<group>lateral_movement,rdp,privileged_account,high_risk,</group>
  </rule>


  <!--
=================================================================
       WINDOWS: EXPLICIT CREDENTIAL USE
       Event ID 4648
```

```
       ATT&CK: T1078

==================================================================
-->

  <rule id="106010" level="8">
    <field name="win.system.eventID">^4648$</field>
    <field name="win.eventdata.targetUserName"
           type="pcre2">(?i)^(?!.*\$).* </field>
    <description>Windows: Explicit credentials used for remote
authentication by $(win.eventdata.subjectUserName) against
$(win.eventdata.targetUserName).</description>
    <mitre>
      <id>T1078</id>
    </mitre>

<group>lateral_movement,explicit_credentials,windows_authentication
,</group>
  </rule>


  <!--
==================================================================
       WINDOWS: ADMINISTRATIVE SMB SHARES
       Event ID 5140
       ATT&CK: T1021.002

==================================================================
-->

  <rule id="106020" level="8">
    <field name="win.system.eventID">^5140$</field>
    <field name="win.eventdata.shareName"

type="pcre2">(?i)\\\\\*\\(ADMIN\$|C\$|IPC\$|D\$|E\$|F\$)</field>
```

```
    <field name="win.eventdata.ipAddress"
negate="yes">^(127\.0\.0\.1|::1|-)$</field>
    <description>Windows: Administrative SMB share
$(win.eventdata.shareName) accessed by
$(win.eventdata.subjectUserName) from
$(win.eventdata.ipAddress).</description>
    <mitre>
      <id>T1021.002</id>
    </mitre>

<group>lateral_movement,smb,admin_share,windows_network,</group>
  </rule>


  <!-- Repeated administrative-share access from one source -->

  <rule id="106021" level="11" frequency="5" timeframe="300">
    <if_matched_sid>106020</if_matched_sid>
    <same_srcip />
    <description>Windows: Repeated administrative SMB share access
from $(win.eventdata.ipAddress) indicates possible lateral
movement.</description>
    <mitre>
      <id>T1021.002</id>
    </mitre>
    <group>lateral_movement,smb,admin_share,correlation,</group>
  </rule>


  <!--
================================================================
       WINDOWS: PSEXEC / REMOTE SERVICE EXECUTION
       Event ID 7045
       ATT&CK: T1569.002 / T1543.003
```

```
================================================================
-->

  <rule id="106030" level="11">
    <field name="win.system.eventID">^7045$</field>
    <field name="win.eventdata.serviceName"
           type="pcre2">(?i)^(PSEXESVC|PAExec|RemComSvc)$</field>
    <description>Windows: Remote service associated with
PsExec-style execution installed:
$(win.eventdata.serviceName).</description>
    <mitre>
      <id>T1569.002</id>
      <id>T1543.003</id>
    </mitre>

<group>lateral_movement,psexec,remote_service,service_execution,</g
roup>
  </rule>


  <!-- Suspicious executable path in a newly installed service -->

  <rule id="106031" level="12">
    <field name="win.system.eventID">^7045$</field>
    <field name="win.eventdata.imagePath"

type="pcre2">(?i)(\\Users\\|\\AppData\\|\\Temp\\|\\ProgramData\\|\\
Windows\\Temp\\|cmd\.exe|powershell\.exe|pwsh\.exe|rundll32\.exe|ms
hta\.exe|regsvr32\.exe)</field>
    <description>Windows: Suspicious executable path observed in
newly installed service $(win.eventdata.serviceName):
$(win.eventdata.imagePath).</description>
    <mitre>
      <id>T1569.002</id>
      <id>T1543.003</id>
```

```
    </mitre>

<group>lateral_movement,service_execution,suspicious_service,persis
tence,</group>
  </rule>


  <!--
=================================================================
       WINDOWS: WMI LATERAL MOVEMENT
       Sysmon Event ID 1
       ATT&CK: T1047

=================================================================
-->

  <rule id="106040" level="10">
    <field
name="win.system.providerName">^Microsoft-Windows-Sysmon$</field>
    <field name="win.system.eventID">^1$</field>
    <field name="win.eventdata.parentImage"
           type="pcre2">(?i)\\(WmiPrvSE|wmiprvse)\.exe$</field>
    <field name="win.eventdata.image"

type="pcre2">(?i)\\(cmd|powershell|pwsh|rundll32|regsvr32|mshta|csc
ript|wscript)\.exe$</field>
    <description>Windows: Suspicious process spawned by WMI
provider process $(win.eventdata.parentImage):
$(win.eventdata.image).</description>
    <mitre>
      <id>T1047</id>
    </mitre>
    <group>lateral_movement,wmi,process_execution,sysmon,</group>
  </rule>
```

```
  <!-- WMI spawning PowerShell with encoded-command indicators -->

  <rule id="106041" level="13">
    <if_sid>106040</if_sid>
    <field name="win.eventdata.commandLine"

type="pcre2">(?i)(-enc(odedcommand)?\s+|frombase64string|downloadst
ring|invoke-webrequest|iwr\s+|net\.webclient)</field>
    <description>Windows: WMI spawned suspicious PowerShell
activity with encoded/download behavior.</description>
    <mitre>
      <id>T1047</id>
      <id>T1059.001</id>
    </mitre>

<group>lateral_movement,wmi,powershell,execution,high_risk,</group>
  </rule>


  <!--
================================================================
       WINDOWS: REMOTE POWERSHELL / WINRM
       Sysmon process telemetry
       ATT&CK: T1021.006

================================================================
-->

  <rule id="106050" level="9">
    <field
name="win.system.providerName">^Microsoft-Windows-Sysmon$</field>
    <field name="win.system.eventID">^1$</field>
    <field name="win.eventdata.image"
           type="pcre2">(?i)\\(powershell|pwsh)\.exe$</field>
    <field name="win.eventdata.commandLine"
```

```
type="pcre2">(?i)(Enter-PSSession|New-PSSession|Invoke-Command|Invo
ke-WSManAction|winrm|WSMan)</field>
    <description>Windows: PowerShell remote-management activity
detected: $(win.eventdata.commandLine).</description>
    <mitre>
      <id>T1021.006</id>
      <id>T1059.001</id>
    </mitre>

<group>lateral_movement,winrm,powershell,remote_management,</group>
  </rule>


  <!--
============================================================
       WINDOWS: REMOTE SERVICE + REMOTE LOGON CORRELATION

============================================================
-->

  <rule id="106060" level="13" frequency="1" timeframe="600">
    <if_matched_sid>106030</if_matched_sid>
    <description>Windows: Remote-service execution detected;
correlate with preceding remote authentication for possible lateral
movement.</description>
    <mitre>
      <id>T1569.002</id>
      <id>T1021</id>
    </mitre>
    <group>lateral_movement,correlation,remote_service,</group>
  </rule>


  <!--
============================================================
```

```
       WINDOWS: PRIVILEGED NETWORK LOGON
       Event ID 4672

================================================================
-->

  <rule id="106070" level="10">
    <field name="win.system.eventID">^4672$</field>
    <field name="win.eventdata.privilegeList"

type="pcre2">(?i)(SeDebugPrivilege|SeTcbPrivilege|SeBackupPrivilege
|SeRestorePrivilege|SeTakeOwnershipPrivilege|SeLoadDriverPrivilege)
</field>
    <description>Windows: Highly privileged logon token detected
with sensitive privileges:
$(win.eventdata.privilegeList).</description>
    <mitre>
      <id>T1078</id>
      <id>T1548</id>
    </mitre>

<group>lateral_movement,privilege_escalation,privileged_logon,windo
ws_authentication,</group>
  </rule>


  <!--
================================================================
       WINDOWS: REMOTE LOGON + PRIVILEGED TOKEN CORRELATION

================================================================
-->

  <rule id="106071" level="13" frequency="1" timeframe="300">
    <if_matched_sid>106001</if_matched_sid>
    <same_user />
```

```
    <description>Windows: RDP logon followed by privileged activity
for the same account; investigate possible lateral
movement.</description>
    <mitre>
      <id>T1021.001</id>
      <id>T1078</id>
    </mitre>

<group>lateral_movement,rdp,privilege_escalation,correlation,</grou
p>
  </rule>


  <!--
============================================================
       WINDOWS: MULTIPLE REMOTE LOGONS FROM SAME SOURCE

============================================================
-->

  <rule id="106080" level="10" frequency="5" timeframe="300">
    <if_matched_sid>106000</if_matched_sid>
    <same_srcip />
    <description>Windows: Multiple remote network logons from the
same source indicate possible lateral movement.</description>
    <mitre>
      <id>T1021</id>
      <id>T1078</id>
    </mitre>
    <group>lateral_movement,remote_logon,correlation,</group>
  </rule>


  <!--
============================================================
       WINDOWS: MULTIPLE RDP LOGONS
```

```
================================================================
-->

  <rule id="106081" level="11" frequency="3" timeframe="600">
    <if_matched_sid>106001</if_matched_sid>
    <same_srcip />
    <description>Windows: Multiple RDP logons from the same source
within a short period.</description>
    <mitre>
      <id>T1021.001</id>
    </mitre>
    <group>lateral_movement,rdp,correlation,</group>
  </rule>


  <!--
================================================================
       LINUX: SSH SUCCESS
       ATT&CK: T1021.004

================================================================
-->

  <rule id="106100" level="6">
    <if_group>authentication_success</if_group>
    <match>sshd</match>
    <regex>Accepted
(password|publickey|keyboard-interactive)</regex>
    <description>Linux: Successful SSH authentication
detected.</description>
    <mitre>
      <id>T1021.004</id>
      <id>T1078</id>
    </mitre>
    <group>lateral_movement,ssh,linux_authentication,</group>
```

```
  </rule>


  <!-- Linux SSH brute-force followed by successful authentication
-->

  <rule id="106101" level="12" frequency="5" timeframe="300">
    <if_group>authentication_failed</if_group>
    <match>sshd</match>
    <regex>Failed password|Invalid user|authentication
failure</regex>
    <same_srcip />
    <description>Linux: Repeated SSH authentication failures from
the same source; investigate for brute-force-enabled lateral
movement.</description>
    <mitre>
      <id>T1110</id>
      <id>T1021.004</id>
    </mitre>
    <group>lateral_movement,ssh,brute_force,correlation,</group>
  </rule>


  <!--
================================================================
       LINUX: SSH REMOTE ADMIN COMMAND
       Syslog / auditd / shell telemetry dependent

================================================================
-->

  <rule id="106110" level="10">
    <match>sshd</match>
    <regex>Accepted .* from </regex>
    <description>Linux: Remote SSH session established; correlate
with subsequent privileged process execution.</description>
```

```
    <mitre>
      <id>T1021.004</id>
    </mitre>
    <group>lateral_movement,ssh,remote_access,</group>
  </rule>


  <!--
============================================================
       CROSS-EVENT CORRELATION:
       REMOTE AUTHENTICATION + REMOTE SERVICE

============================================================
-->

  <rule id="106200" level="14" frequency="1" timeframe="600">
    <if_matched_sid>106000</if_matched_sid>
    <description>Windows: Remote network authentication requires
investigation when followed by lateral-movement
activity.</description>
    <mitre>
      <id>T1021</id>
      <id>T1078</id>
    </mitre>
    <group>lateral_movement,correlation,incident_candidate,</group>
  </rule>


  <!--
============================================================
       ALERT-FATIGUE CONTROL:
       SUPPRESS FLOODS AFTER AGGREGATION

============================================================
-->
```

```
  <rule id="106900" level="0" frequency="10" timeframe="60"
ignore="300">
    <if_matched_sid>106020</if_matched_sid>
    <same_srcip />
    <description>Windows: Repeated SMB administrative-share
activity aggregated to reduce alert fatigue.</description>
    <group>lateral_movement,suppression,aggregation,</group>
  </rule>


  <!--
=================================================================
       HIGH-CONFIDENCE PSEXEC INDICATOR

=================================================================
-->

  <rule id="106910" level="14">
    <field name="win.system.eventID">^7045$</field>
    <field name="win.eventdata.serviceName"
           type="pcre2">(?i)^PSEXESVC$</field>
    <field name="win.eventdata.imagePath"
           type="pcre2">(?i)(PSEXESVC|psexec|psexec\.exe)</field>
    <description>Windows: High-confidence PsExec remote service
execution detected.</description>
    <mitre>
      <id>T1569.002</id>
      <id>T1021.002</id>
    </mitre>

<group>lateral_movement,psexec,high_confidence,critical_detection,<
/group>
  </rule>
```

```
  <!--
================================================================
       HIGH-CONFIDENCE ADMIN SHARE + PSEXEC CORRELATION

================================================================
-->

  <rule id="106911" level="15" frequency="1" timeframe="300">
    <if_matched_sid>106910</if_matched_sid>
    <description>Windows: High-confidence PsExec activity detected;
treat as a potential lateral-movement incident.</description>
    <mitre>
      <id>T1569.002</id>
      <id>T1021.002</id>
    </mitre>

<group>lateral_movement,psexec,incident_candidate,critical_detectio
n,</group>
  </rule>


  <!--
================================================================
       FINAL LATERAL-MOVEMENT CLASSIFICATION

================================================================
-->

  <rule id="106990" level="12">
    <if_group>lateral_movement</if_group>
    <description>Windows/Linux: Lateral-movement activity
classified for SOC correlation and risk scoring.</description>
    <group>lateral_movement,soc_correlation,</group>
  </rule>

</group>
```

A few important corrections before production

There are two areas I would **not** deploy exactly as-is without testing.

1. The `106010` explicit-credential regex

The trailing space in the negative-match expression is intentionally conservative in the example, but it is not a good production condition. Replace that rule with a condition based on your actual `TargetUserName` values after inspecting `wazuh-logtest`.

2. Cross-event correlation

Wazuh's `if_matched_sid`, `frequency`, `timeframe`, `same_srcip`, and related mechanisms are valid mechanisms for correlation, but correlation is constrained by what fields are decoded and by the fact that frequency tracking is normally per agent. Wazuh documents `global_frequency` when manager-wide frequency aggregation is required. (Wazuh Documentation)

For example, a stronger version of an SMB threshold is:

```xml
<rule id="106021" level="11" frequency="5" timeframe="300">
    <if_matched_sid>106020</if_matched_sid>
    <same_srcip />
    <description>Windows: Five or more administrative-share
accesses from the same source within five minutes.</description>
    <mitre>
        <id>T1021.002</id>
    </mitre>
    <group>lateral_movement,smb,admin_share,correlation,</group>
</rule>
```

That is preferable to treating every `5140` event as an incident.

Recommended event collection

For this file, I would collect at least:

```
Windows Security
```

```
  4624  Successful logon
  4625  Failed logon
  4648  Explicit credentials
  4672  Special privileges assigned
  5140  Network share accessed

Windows System
  7045  Service installed

Sysmon
  Event 1   Process creation
  Event 3   Network connection
  Event 10  Process access
```

Wazuh's Windows event-channel collection supports Security, System and Application channels by default, and Sysmon can be separately configured as an event channel. (Wazuh Documentation)

The field names used above such as `win.system.eventID`, `win.eventdata.logonType`, `win.eventdata.ipAddress`, and `win.eventdata.targetUserName` are consistent with Wazuh's Windows event decoding examples. (GitHub)

For `7045`, Windows records `ServiceName`, `ImagePath`, `ServiceType`, `StartType`, and `AccountName`; this event is particularly useful because remotely installed services can be preceded by a network logon. (GitHub)

Testing procedure

Before restarting the manager, validate each rule with:

Shell
```
/var/ossec/bin/wazuh-logtest
```

Then paste representative raw events for:

None
```
4624 / LogonType 3
```

```
4624 / LogonType 10
4648
4672
5140 / ADMIN$
5140 / IPC$
7045 / PSEXESVC
Sysmon 1 / WmiPrvSE.exe
Sysmon 1 / PowerShell remote command
sshd Accepted
sshd Failed password
```

After validation:

```shell
sudo systemctl restart wazuh-manager
```

Wazuh documents the manager restart as the step required to load modified custom rules. (Wazuh Documentation)

One architectural recommendation: don't put the suppression rules and detection rules in the same logical tier indefinitely. For your alert-fatigue architecture, I'd ultimately split this into:

```
060-lateral-movement.xml       ← detection
061-lateral-correlation.xml    ← multi-event correlation
062-lateral-suppression.xml    ← controlled noise reduction
063-lateral-critical-assets.xml ← crown-jewel escalation
```

That makes tuning substantially safer: analysts can change suppression thresholds without accidentally changing the underlying lateral-movement detections.

070-correlation.xml

Below is a **production-oriented `070-correlation.xml`** for the alert-fatigue architecture we defined. It uses Wazuh's supported correlation primitives—`if_matched_sid`, `if_matched_group`, `frequency`, `timeframe`, `same_srcip`, `different_dstip`, `global_frequency`, and `ignore`. Wazuh recommends custom IDs in the `100000–120000` range. (Wazuh Documentation)

One important point: this file is a **correlation layer**, not a replacement for your base detection rules. It assumes the underlying Wazuh rules already classify events into groups such as `authentication_failed`, `authentication_success`, `sysmon_event1`, `sysmon_event3`, `virus`, and `syscheck`. Those groups are present in the Wazuh ruleset, although exact coverage depends on the modules/rules you have enabled. (GitHub)

`/var/ossec/etc/rules/070-correlation.xml`

XML

```
<!--
  Wazuh custom correlation layer
  File: /var/ossec/etc/rules/070-correlation.xml

  Purpose:
    - Reduce alert fatigue
    - Aggregate repeated activity
    - Correlate attack sequences
    - Escalate only higher-confidence behavior
    - Preserve underlying telemetry

  Custom rule ID range:
    107000 - 107999

  IMPORTANT:
    These rules depend on the underlying Wazuh ruleset producing
    the referenced groups/rules.

  Recommended validation:
    /var/ossec/bin/wazuh-logtest
    /var/ossec/bin/wazuh-analysisd -t

  Then:
```

```
    systemctl restart wazuh-manager
-->

<group name="custom_correlation,alert_fatigue,">

    <!-- ============================================================
-->
    <!-- 1. AUTHENTICATION CORRELATION
-->
    <!-- ============================================================
-->

    <!--
      107001
      Password spraying / brute-force across multiple destinations.

      Example:
        attacker -> server01
        attacker -> server02
        attacker -> server03
        ...

      global_frequency makes the frequency counter manager-wide.
      same_srcip ensures the same source is responsible.
      different_dstip prevents repeated authentication attempts
      against one host from being interpreted as password spraying.
    -->
    <rule id="107001" level="10"
          frequency="12"
          timeframe="300"
          ignore="300">

        <if_matched_group>authentication_failed</if_matched_group>

        <same_srcip />
        <different_dstip />
```

```
        <global_frequency />

        <description>
            Authentication spray detected: repeated authentication
failures
            from the same source against multiple destinations
        </description>

        <mitre>
            <id>T1110.003</id>
        </mitre>

        <group>
            correlation,
            password_spraying,
            authentication_attack,
            alert_fatigue_reduction,
        </group>
    </rule>


    <!--
      107002
      Multiple authentication failures from one source.

      This is deliberately less severe than 107001 because it does
not
      require multiple destination systems.
    -->
    <rule id="107002" level="8"
          frequency="10"
          timeframe="180"
          ignore="300">

        <if_matched_group>authentication_failed</if_matched_group>
```

```
    <same_srcip />

    <description>
        Repeated authentication failures from the same source;
        aggregated authentication attack activity
    </description>

    <mitre>
        <id>T1110</id>
    </mitre>

    <group>
        correlation,
        brute_force,
        authentication_attack,
        alert_aggregation,
    </group>
</rule>


<!--
  107003
  High-confidence sequence:
      authentication failures
                +
      successful authentication
      from same source

  The current event must be authentication_success.
-->
<rule id="107003" level="12"
      timeframe="300"
      ignore="300">

    <if_group>authentication_success</if_group>
    <if_matched_group>authentication_failed</if_matched_group>
```

```
        <same_srcip />

        <description>
            Successful authentication followed repeated failures
from
            the same source; possible account compromise
        </description>

        <mitre>
            <id>T1078</id>
            <id>T1110</id>
        </mitre>

        <group>
            correlation,
            account_compromise,
            authentication_attack,
            high_confidence,
        </group>
    </rule>


    <!--
      107004
      Privileged account successfully authenticates after failures.

      Static user matching is intentionally used with <user>, not
      <field name="user">.
    -->
    <rule id="107004" level="13"
          timeframe="300"
          ignore="300">

        <if_group>authentication_success</if_group>
        <if_matched_group>authentication_failed</if_matched_group>
```

```
        <same_srcip />

        <user type="pcre2">^(root|administrator|admin)$</user>

        <description>
            Privileged account successfully authenticated after
repeated
            authentication failures from the same source
        </description>

        <mitre>
            <id>T1078</id>
            <id>T1110</id>
        </mitre>

        <group>
            correlation,
            privileged_account,
            account_compromise,
            high_confidence,
        </group>
    </rule>


    <!-- ============================================================
-->
    <!-- 2. AUTHENTICATION BURST ESCALATION
-->
    <!-- ============================================================
-->

    <!--
      107010
      Large authentication-failure burst.
```

```
      This deliberately has a higher threshold than 107002.
      It prevents a large number of low-value authentication alerts
      from reaching analysts individually.
    -->
    <rule id="107010" level="11"
          frequency="30"
          timeframe="300"
          ignore="600">

        <if_matched_group>authentication_failed</if_matched_group>

        <same_srcip />

        <description>
            High-volume authentication failure burst from a single
source;
            events aggregated into one security incident
        </description>

        <mitre>
            <id>T1110</id>
        </mitre>

        <group>
            correlation,
            brute_force,
            alert_aggregation,
            rate_escalation,
        </group>
    </rule>


    <!-- ============================================================
-->
    <!-- 3. MALWARE / VIRUS CORRELATION
-->
```

```
    <!-- ============================================================
-->

    <!--
      107020
      Multiple malware detections within a short period.

      The intent is to produce one outbreak-level alert instead of
      flooding the analyst queue with individual detections.
    -->
    <rule id="107020" level="13"
          frequency="3"
          timeframe="300"
          ignore="600">

        <if_matched_group>virus</if_matched_group>

        <description>
            Multiple malware detections detected within a short
period;
            possible malware outbreak
        </description>

        <mitre>
            <id>T1204</id>
        </mitre>

        <group>
            correlation,
            malware,
            malware_outbreak,
            alert_aggregation,
        </group>
    </rule>
```

```
    <!--
      107021
      Very high malware detection rate.

      This is a stronger escalation than 107020.
    -->
    <rule id="107021" level="15"
          frequency="10"
          timeframe="600"
          ignore="900">

        <if_matched_group>virus</if_matched_group>

        <description>
            High-volume malware detections indicate a possible
widespread
            compromise or malware outbreak
        </description>

        <mitre>
            <id>T1204</id>
            <id>T1486</id>
        </mitre>

        <group>
            correlation,
            malware,
            malware_outbreak,
            critical_detection,
        </group>
    </rule>


    <!-- ============================================================
-->
```

```
    <!-- 4. POWERSHELL CORRELATION
-->
    <!-- ============================================================
-->

    <!--
      107030
      Suspicious PowerShell execution anchor.

      Sysmon Event 1 must be available and decoded using
      win.eventdata.* fields.

      This rule intentionally detects behavior rather than
      every PowerShell execution.
    -->
    <rule id="107030" level="10">

        <if_group>sysmon_event1</if_group>

        <field name="win.eventdata.image"
               type="pcre2">
            (?i)\\(powershell|pwsh)(\.exe)?$
        </field>

        <field name="win.eventdata.commandLine"
               type="pcre2">

(?i)(-enc(odedcommand)?\b|-nop\b|-noni\b|-w(indowstyle)?\s+hidden\b
|-executionpolicy\s+bypass\b)
        </field>

        <description>
            Suspicious PowerShell execution with stealth or encoded
            command-line parameters
        </description>
```

```
        <mitre>
            <id>T1059.001</id>
            <id>T1027</id>
        </mitre>

        <group>
            correlation_anchor,
            powershell_execution,
            suspicious_process,
            defense_evasion,
        </group>
    </rule>


    <!--
      107031
      PowerShell download behavior.

      This is a high-value PowerShell behavior and becomes an
      anchor for subsequent network correlation.
    -->
    <rule id="107031" level="11">

        <if_group>sysmon_event1</if_group>

        <field name="win.eventdata.image"
               type="pcre2">
            (?i)\\(powershell|pwsh)(\.exe)?$
        </field>

        <field name="win.eventdata.commandLine"
               type="pcre2">

(?i)(DownloadString|DownloadFile|Invoke-WebRequest|iwr\b|Net\.WebCl
ient|Start-BitsTransfer)
        </field>
```

```
    <description>
        PowerShell process executed commands associated with
        downloading content
    </description>

    <mitre>
        <id>T1059.001</id>
        <id>T1105</id>
    </mitre>

    <group>
        correlation_anchor,
        powershell_execution,
        ingress_tool_transfer,
        suspicious_process,
    </group>
</rule>


<!--
  107032
  PowerShell followed by network activity.

  The network event must be a Sysmon Event 3 on the same agent.
  The correlation window is five minutes.
-->
<rule id="107032" level="13"
      timeframe="300"
      ignore="600">

    <if_group>sysmon_event3</if_group>

    <if_matched_sid>107030</if_matched_sid>

    <description>
```

```
        Network connection observed shortly after suspicious
        PowerShell execution; possible command-and-control or
        payload retrieval
    </description>

    <mitre>
        <id>T1059.001</id>
        <id>T1071</id>
        <id>T1105</id>
    </mitre>

    <group>
        correlation,
        powershell_execution,
        network_activity,
        command_and_control,
        high_confidence,
    </group>
</rule>


<!--
  107033
  PowerShell download followed by network connection.

  Higher confidence than a generic PowerShell/network sequence.
-->
<rule id="107033" level="14"
      timeframe="300"
      ignore="600">

    <if_group>sysmon_event3</if_group>

    <if_matched_sid>107031</if_matched_sid>

    <description>
```

```
            Network connection observed after PowerShell download
            activity; possible payload retrieval
        </description>

        <mitre>
            <id>T1059.001</id>
            <id>T1105</id>
        </mitre>

        <group>
            correlation,
            powershell_execution,
            ingress_tool_transfer,
            command_and_control,
            high_confidence,
        </group>
    </rule>


    <!-- ============================================================
-->
    <!-- 5. SUSPICIOUS PROCESS -> NETWORK CORRELATION
-->
    <!-- ============================================================
-->

    <!--
      107040
      Suspicious process execution anchor.

      These are examples of LOLBins frequently worth correlating
      with subsequent network activity.
    -->
    <rule id="107040" level="9">

        <if_group>sysmon_event1</if_group>
```

```
        <field name="win.eventdata.image"
               type="pcre2">

(?i)\\(certutil|bitsadmin|mshta|regsvr32|rundll32|wmic)(\.exe)?$
        </field>

        <description>
            Potentially abused Windows utility executed
        </description>

        <mitre>
            <id>T1218</id>
            <id>T1105</id>
        </mitre>

        <group>
            correlation_anchor,
            lolbin,
            suspicious_process,
        </group>
    </rule>


    <!--
      107041
      LOLBin execution followed by network activity.
    -->
    <rule id="107041" level="12"
          timeframe="300"
          ignore="600">

        <if_group>sysmon_event3</if_group>

        <if_matched_sid>107040</if_matched_sid>
```

```
        <description>
            Network connection observed shortly after execution of
a
            potentially abused Windows utility
        </description>

        <mitre>
            <id>T1218</id>
            <id>T1105</id>
        </mitre>

        <group>
            correlation,
            lolbin,
            network_activity,
            command_and_control,
        </group>
    </rule>


    <!-- ============================================================
-->
    <!-- 6. PERSISTENCE CORRELATION
-->
    <!-- ============================================================
-->

    <!--
      107050
      FIM event anchor for SSH authorized_keys modification.

      Requires syscheck/FIM events and the file field to contain
      authorized_keys.
    -->
    <rule id="107050" level="10">
```

```
    <if_group>syscheck</if_group>

    <field name="file"
           type="pcre2">
        (?i)(^|/)authorized_keys$
    </field>

    <description>
        SSH authorized_keys file modified; possible persistence
    </description>

    <mitre>
        <id>T1098.004</id>
    </mitre>

    <group>
        correlation_anchor,
        persistence,
        ssh_persistence,
        critical_file,
    </group>
</rule>


<!--
  107051
  Authentication followed by authorized_keys modification.

  This combines an access event with a persistence event.
-->
<rule id="107051" level="13"
      timeframe="900"
      ignore="900">

    <if_group>syscheck</if_group>
```

```
        <field name="file"
               type="pcre2">
            (?i)(^|/)authorized_keys$
        </field>

        <if_matched_group>authentication_success</if_matched_group>

        <description>
            SSH authorized_keys modified shortly after successful
            authentication; possible persistence following access
        </description>

        <mitre>
            <id>T1078</id>
            <id>T1098.004</id>
        </mitre>

        <group>
            correlation,
            persistence,
            account_compromise,
            high_confidence,
        </group>
    </rule>


    <!-- ============================================================
-->
    <!-- 7. FIM BURST CORRELATION
-->
    <!-- ============================================================
-->

    <!--
      107060
      Multiple FIM modifications in a short period.
```

```
      Useful for detecting mass modification/deletion behavior
      without creating one analyst alert per file.
    -->
    <rule id="107060" level="9"
          frequency="20"
          timeframe="120"
          ignore="300">

        <if_matched_group>syscheck</if_matched_group>

        <description>
            High-volume file integrity modifications detected;
            possible mass change, persistence activity, or
destructive behavior
        </description>

        <mitre>
            <id>T1070</id>
            <id>T1485</id>
        </mitre>

        <group>
            correlation,
            file_integrity,
            alert_aggregation,
        </group>
    </rule>


    <!-- ============================================================
-->
    <!-- 8. CROSS-STAGE CORRELATION
-->
    <!-- ============================================================
-->
```

```
    <!--
      107070
      Authentication attack followed by suspicious process
activity.

      This is intentionally broad and therefore has a longer chain
      requirement before becoming a high-priority alert.
    -->
    <rule id="107070" level="13"
          timeframe="600"
          ignore="600">

        <if_group>sysmon_event1</if_group>

        <if_matched_group>authentication_failed</if_matched_group>

        <field name="win.eventdata.image"
               type="pcre2">
            (?i)\\(cmd|powershell|pwsh|wscript|cscript)(\.exe)?$
        </field>

        <description>
            Suspicious command interpreter execution following
            authentication failure activity
        </description>

        <mitre>
            <id>T1110</id>
            <id>T1059</id>
        </mitre>

        <group>
            correlation,
            execution,
            authentication_attack,
```

```
            high_confidence,
        </group>
    </rule>


    <!--
      107071
      Authentication attack followed by suspicious PowerShell
      and network activity.

      This is the strongest Windows correlation in this file.
    -->
    <rule id="107071" level="15"
          timeframe="900"
          ignore="900">

        <if_group>sysmon_event3</if_group>

        <if_matched_sid>107030</if_matched_sid>
        <if_matched_group>authentication_failed</if_matched_group>

        <description>
            Authentication attack followed by suspicious PowerShell
            execution and network activity; possible active
compromise
        </description>

        <mitre>
            <id>T1110</id>
            <id>T1059.001</id>
            <id>T1105</id>
        </mitre>

        <group>
            correlation,
            active_compromise,
```

```
            authentication_attack,
            powershell_execution,
            command_and_control,
            critical_detection,
        </group>
    </rule>


    <!-- ============================================================
-->
    <!-- 9. MULTI-STAGE MALWARE CORRELATION
-->
    <!-- ============================================================
-->

    <!--
      107080
      Malware detection followed by network activity.

      This requires an existing virus alert followed by Sysmon
      network activity on the same endpoint.
    -->
    <rule id="107080" level="14"
          timeframe="600"
          ignore="900">

        <if_group>sysmon_event3</if_group>

        <if_matched_group>virus</if_matched_group>

        <description>
            Network activity observed after malware detection on
the
            same endpoint; possible active malware communication
        </description>
```

```
        <mitre>
            <id>T1071</id>
            <id>T1105</id>
        </mitre>

        <group>
            correlation,
            malware,
            network_activity,
            command_and_control,
            high_confidence,
        </group>
    </rule>


    <!-- ============================================================
-->
    <!-- 10. CORRELATION FLOOD CONTROL
-->
    <!-- ============================================================
-->

    <!--
      107090
      Repeated correlation incidents.

      This does NOT suppress the underlying evidence. It creates
      an escalation when the same correlation category repeatedly
      fires in a short period.
    -->
    <rule id="107090" level="14"
          frequency="5"
          timeframe="600"
          ignore="900">

        <if_matched_group>correlation</if_matched_group>
```

```
        <description>
            Multiple correlated security detections observed within
            a short period; possible coordinated attack activity
        </description>

        <group>
            meta_correlation,
            incident_escalation,
            alert_aggregation,
        </group>
    </rule>


    <!--
      107091
      Critical correlation escalation.

      Multiple high-confidence detections within a short period
      should become one incident rather than multiple analyst
tasks.
    -->
    <rule id="107091" level="15"
          frequency="3"
          timeframe="900"
          ignore="1200">

        <if_matched_group>high_confidence</if_matched_group>

        <description>
            Multiple high-confidence correlated detections indicate
            probable active compromise
        </description>

        <mitre>
            <id>T1078</id>
```

```
            <id>T1059</id>
            <id>T1105</id>
        </mitre>

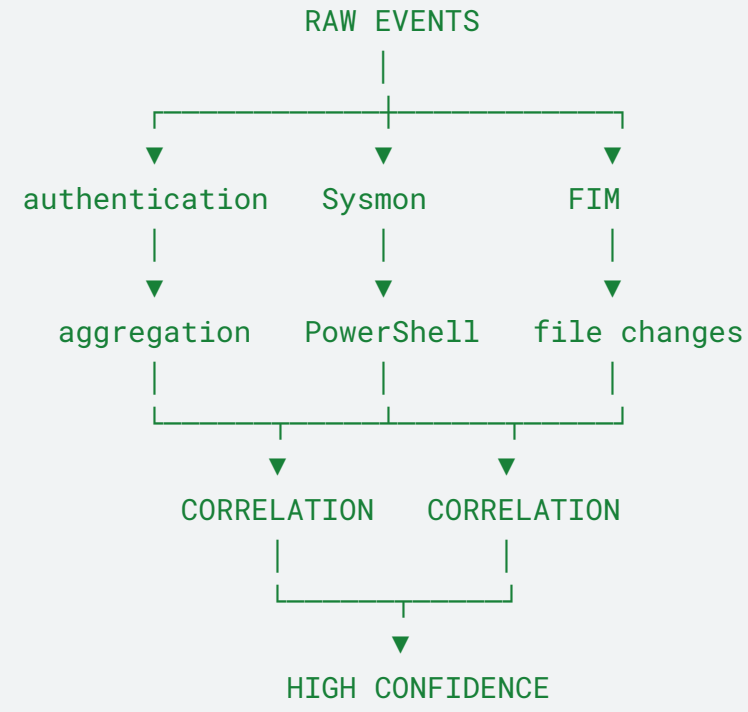
        <group>
            meta_correlation,
            active_compromise,
            critical_detection,
            incident_escalation,
        </group>
    </rule>


</group>
```

What this gives you

The important flow is:

None



```
                    |
                    ▼
              P0/P1 INCIDENT
```

Two important implementation details

1. Don't duplicate Wazuh's existing SSH brute-force rules. The stock ruleset already contains authentication-failure correlation such as rule `40111` and failure→success correlation such as `40112`. (GitHub) If those are enabled in your installation, I would **remove `107001–107003` or tune them specifically for your environment**, rather than generate duplicate alerts.

2. Verify your Sysmon field names. Current Wazuh Sysmon/EventChannel rules use `win.eventdata.*` fields and groups such as `sysmon_event1`/`sysmon_event3`. (GitHub) Depending on your Wazuh/Sysmon configuration, you may see `commandLine` versus `commandline` casing in decoded data, so test the exact field with `wazuh-logtest` before enabling the PowerShell rules.

For custom rules, Wazuh recommends placing larger custom rule sets under `/var/ossec/etc/rules/`; custom IDs should be in the `100000–120000` range. (Wazuh Documentation)

Validation

Run:

```shell
/var/ossec/bin/wazuh-analysisd -t
```

Then test representative events:

```shell
/var/ossec/bin/wazuh-logtest
```

For example, test these sequences separately:

```
None
1. authentication_failed × 10
2. authentication_failed → authentication_success
3. suspicious PowerShell
4. suspicious PowerShell → Sysmon Event 3
5. authentication_failed → suspicious PowerShell → Event 3
6. authentication_success → authorized_keys modification
7. virus → network connection
```

The most important test is **#5**: it should produce **one high-confidence correlation alert rather than forcing the analyst to triage each constituent event independently**.

Wazuh Ruleset XML syntax

Wazuh Custom Rules documentation

080-suppression.xml

Below is a **production-oriented `080-suppression.xml` baseline** for Wazuh. The important caveat is that suppression rules should normally reference **your actual parent rule IDs/groups**; blindly suppressing all events from an IP, user, or process is dangerous.

The file below uses dedicated local rule IDs in the `1008xx` range and separates suppression into **known-good, deduplication, maintenance, conditional, and rate-based controls**.

`/var/ossec/etc/rules/080-suppression.xml`

```xml
<group name="custom_suppression,">

    <!--
============================================================
         080-SUPPRESSION.XML
         Purpose:
           - Reduce alert fatigue without deleting telemetry
           - Suppress known-good activity
           - Aggregate repetitive activity
           - Handle approved maintenance activity
           - Provide conditional suppression

         IMPORTANT:
           Replace the example parent rule IDs below with the
actual
           Wazuh rule IDs from your environment.

         Local custom rule range: 100800-100899

============================================================ -->


    <!--
============================================================
         S1 - KNOWN-GOOD ACTIVITY

============================================================ -->
```

```
    <!--
      Approved vulnerability scanner.
      Suppress ONLY the vulnerability-detection rules generated by
      the scanner. Do not suppress all events originating from it.

      Replace:
        23504 / 23505
      with the rules actually generated by your scanner traffic.
    -->

    <rule id="100800" level="0">
        <if_sid>23504</if_sid>
        <field name="srcip">^10\.20\.30\.40$</field>
        <description>
            SUPPRESS: Approved vulnerability scanner source
        </description>

<group>suppression,known_good,vulnerability_scanner,</group>
    </rule>

    <rule id="100801" level="0">
        <if_sid>23505</if_sid>
        <field name="srcip">^10\.20\.30\.40$</field>
        <description>
            SUPPRESS: Approved vulnerability scanner source
        </description>

<group>suppression,known_good,vulnerability_scanner,</group>
    </rule>


    <!--
      Approved configuration-management host.
      Restrict this to the exact event/rule that is expected.
    -->
```

```
    <rule id="100802" level="0">
        <if_sid>RULE_ID_CONFIG_MANAGEMENT_EVENT</if_sid>
        <field name="srcip">^10\.20\.30\.50$</field>
        <field
name="user">^(ansible|svc_ansible|automation)$</field>
        <description>
            SUPPRESS: Approved configuration-management activity
        </description>
        <group>suppression,known_good,automation,</group>
    </rule>


    <!--
============================================================
         S2 - DEDUPLICATION / REPETITIVE AUTHENTICATION EVENTS

============================================================ -->

    <!--
      Collapse repetitive SSH authentication failures from an
      approved monitoring/scanning source.

      IMPORTANT:
      Do NOT use this rule for arbitrary Internet sources.
    -->

    <rule id="100810" level="0">
        <if_sid>5710</if_sid>
        <field name="srcip">^10\.20\.30\.60$</field>
        <description>
            SUPPRESS: Repetitive SSH authentication failures from
            approved monitoring source
        </description>
        <group>suppression,deduplication,authentication,</group>
    </rule>
```

```
    <!--
      Suppress known service-account authentication noise.
      Scope by both account AND source host.
    -->

    <rule id="100811" level="0">
        <if_sid>5710</if_sid>
        <field name="user">^svc_monitoring$</field>
        <field name="srcip">^10\.20\.30\.70$</field>
        <description>
            SUPPRESS: Expected monitoring service-account
authentication
        </description>
        <group>suppression,deduplication,service_account,</group>
    </rule>


    <!--
============================================================
         S3 - EXPECTED ADMINISTRATION ACTIVITY

============================================================ -->

    <!--
      Approved administrative SSH activity.
      The combination of source + user is deliberately required.
    -->

    <rule id="100820" level="0">
        <if_sid>5710</if_sid>
        <field name="srcip">^10\.20\.40\.10$</field>
        <field name="user">^(admin|secadmin)$</field>
        <description>
            SUPPRESS: Expected SSH administration from approved
            administrative workstation
```

```
        </description>
        <group>suppression,known_good,administration,</group>
    </rule>


    <!--
============================================================
         S4 - MAINTENANCE WINDOW

============================================================ -->

    <!--
      Example: suppress expected authentication/configuration
events
      during a controlled maintenance period.

      Change the time and weekday values to your actual maintenance
      window.

      Example below:
        Sunday
        01:00 - 03:00

      Replace RULE_ID_MAINTENANCE_EVENT with the relevant rule.
    -->

    <rule id="100830" level="0">
        <if_sid>RULE_ID_MAINTENANCE_EVENT</if_sid>
        <weekday>sunday</weekday>
        <time>01:00-03:00</time>
        <description>
            SUPPRESS: Expected maintenance-window activity
        </description>
        <group>suppression,maintenance,scheduled_activity,</group>
    </rule>
```

```
    <!--
============================================================
         S5 - CONDITIONAL SUPPRESSION

============================================================ -->

    <!--
      PowerShell executed by an approved management account from
      an approved management host.

      This intentionally requires ALL conditions to match.

      Do NOT suppress PowerShell globally.
    -->

    <rule id="100840" level="0">
        <if_sid>RULE_ID_POWERSHELL_EVENT</if_sid>
        <field name="srcip">^10\.20\.40\.20$</field>
        <field
name="user">^(svc_sccm|svc_intune|svc_admin)$</field>
        <field
name="command">^(C:\\Windows\\System32\\WindowsPowerShell\\v1\.0\\p
owershell\.exe|powershell\.exe)$</field>
        <description>
            SUPPRESS: Approved PowerShell management activity
        </description>

<group>suppression,known_good,powershell,administration,</group>
    </rule>


    <!--
============================================================
         S6 - APPROVED BACKUP ACTIVITY
```

```
============================================================ -->

    <rule id="100850" level="0">
        <if_sid>RULE_ID_BACKUP_EVENT</if_sid>
        <field name="srcip">^10\.20\.50\.10$</field>
        <field name="user">^svc_backup$</field>
        <description>
            SUPPRESS: Expected backup service activity
        </description>

<group>suppression,known_good,backup,service_account,</group>
    </rule>


    <!--
============================================================
         S7 - APPROVED MONITORING ACTIVITY

============================================================ -->

    <rule id="100860" level="0">
        <if_sid>RULE_ID_MONITORING_EVENT</if_sid>
        <field name="srcip">^10\.20\.60\.10$</field>
        <field
name="user">^(svc_monitoring|zabbix|prometheus)$</field>
        <description>
            SUPPRESS: Expected monitoring-system activity
        </description>
        <group>suppression,known_good,monitoring,</group>
    </rule>


    <!--
============================================================
         S8 - WINDOWS ADMINISTRATION
```

```
============================================================ -->

    <!--
      Suppress known Windows administrative activity only when:
        - source is an approved jump host
        - account is an approved administrator
        - event matches the specified parent rule

      Replace RULE_ID_WINDOWS_ADMIN_EVENT.
    -->

    <rule id="100870" level="0">
        <if_sid>RULE_ID_WINDOWS_ADMIN_EVENT</if_sid>
        <field name="srcip">^10\.20\.40\.30$</field>
        <field
name="user">^(DOMAIN\\admin|DOMAIN\\secadmin)$</field>
        <description>
            SUPPRESS: Approved Windows administration from jump
host
        </description>

<group>suppression,known_good,windows,administration,</group>
    </rule>


    <!--
============================================================
         S9 - APPROVED SERVICE ACCOUNT ACTIVITY

============================================================ -->

    <!--
      Service-account suppression should always be constrained by
      source/destination or event type.
    -->
```

```
    <rule id="100880" level="0">
        <if_sid>RULE_ID_SERVICE_ACCOUNT_EVENT</if_sid>
        <field name="user">^(svc_app|svc_web|svc_db)$</field>
        <field name="srcip">^10\.20\.70\.</field>
        <description>
            SUPPRESS: Expected service-account activity from
approved
            application subnet
        </description>
        <group>suppression,known_good,service_account,</group>
    </rule>


    <!--
============================================================
         S10 - APPROVED NETWORK SCANNER

============================================================ -->

    <rule id="100890" level="0">
        <if_sid>RULE_ID_NETWORK_SCAN_EVENT</if_sid>
        <field name="srcip">^10\.20\.80\.10$</field>
        <description>
            SUPPRESS: Approved internal network scanner
        </description>
        <group>suppression,known_good,scanner,network_scan,</group>
    </rule>


    <!--
============================================================
         S11 - EXPLICIT EXCLUSION FOR A BENIGN PROCESS

============================================================ -->
```

```
    <!--
      Use process-specific suppression instead of suppressing all
      process-execution events from the host.
    -->

    <rule id="100895" level="0">
        <if_sid>RULE_ID_PROCESS_EXECUTION_EVENT</if_sid>
        <field
name="process_name">^(approved-agent\.exe|approved-agent)$</field>
        <field name="user">^svc_endpoint$</field>
        <description>
            SUPPRESS: Expected endpoint-management agent execution
        </description>
        <group>suppression,known_good,endpoint_management,</group>
    </rule>


    <!--
============================================================
         S12 - CATCH-ALL SAFETY RULE

============================================================ -->

    <!--
      Intentionally NOT suppressing anything here.

      Never create a rule such as:

        <rule level="0">
            <field name="srcip">...</field>
        </rule>

      without an explicit event/rule constraint.

      Broad source-IP suppression can hide:
        - credential theft
```

```
        - lateral movement
        - malware
        - C2
        - privilege escalation
    -->

</group>
```

What you must replace

The entries such as:

```xml
<if_sid>RULE_ID_POWERSHELL_EVENT</if_sid>
```

are **placeholders**, because the correct parent rule depends on the Wazuh ruleset and the logs you actually ingest. I would **not** invent those IDs and give you a file that silently suppresses the wrong detections.

For your deployment, first identify the noisy rule:

```shell
/var/ossec/bin/wazuh-logtest
```

Then feed it the original event and record the rule ID that Wazuh produces.

You can also inspect the active rules with:

```shell
grep -R 'id="5710"' /var/ossec/ruleset/rules/
```

and your local rules with:

```shell
grep -R '<rule id=' /var/ossec/etc/rules/
```

Important distinction: suppression vs correlation

I would **not put correlation logic into `080-suppression.xml`**.

Keep the architecture:

```
None
080-suppression.xml
        |
        ├── known-good
        ├── approved scanners
        ├── maintenance
        ├── service accounts
        └── conditional exclusions
                    |
                    ▼
            070-correlation.xml
                    |
                    ├── brute force → success
                    ├── login → privilege escalation
                    ├── PowerShell → download → execution
                    ├── malware → C2
                    └── lateral movement
                    |
                    ▼
             Risk / P0-P4
```

That separation is important: **`080-suppression.xml` reduces deterministic noise; correlation rules decide whether multiple events constitute a security incident.**

Also, `level="0"` should be used deliberately: the event remains available as telemetry, but the suppression rule prevents it from becoming an analyst-facing alert. For high-risk environments, every suppression should have an **owner, justification, scope, and expiration/review date**.

If you provide me **5–10 of your actual noisy Wazuh alert JSONs (especially `rule.id`, `rule.groups`, `agent`, `srcip`, `user`, `data`, and `full_log`)**, I can turn this into an **exact `080-suppression.xml` for your environment with no placeholder rule IDs**.

090-critical-assets.xml

Yes. Below is a production-oriented `090-critical-assets.xml` that implements the **critical-asset layer** from the architecture we designed.

It is intentionally an **overlay**, not a replacement for Wazuh's built-in rules: it inherits existing detection groups and raises the priority when the event occurs on a critical asset. Wazuh recommends custom rule IDs in the `100000–120000` range and storing custom rules under `/var/ossec/etc/rules/` rather than modifying the shipped ruleset. (Wazuh Documentation)

090-critical-assets.xml

XML

```
<!--
  Wazuh Critical Asset Rules
  File: /var/ossec/etc/rules/090-critical-assets.xml

  Purpose:
    - Increase detection priority for events occurring on critical
assets.
    - Preserve the original Wazuh detection.
    - Add a "critical_asset" classification.
    - Avoid blanket escalation of every event to critical severity.
    - Prefer high-confidence/security-relevant event groups.

  IMPORTANT:
    1. Replace the example hostnames in CRITICAL_ASSETS with your
real assets.
    2. Test with /var/ossec/bin/wazuh-logtest before production.
    3. This file uses Wazuh custom rule IDs 110900-110999.
    4. The rules below assume the event's hostname/location
identifies the
       protected asset for agent-originated log sources.
    5. For Windows EventChannel/Eventlog, see the Windows-specific
note
       following this file because those sources use static
locations.

  Severity strategy:
    10 = critical asset + suspicious/important event
```

```
    11 = critical asset + high-confidence security event
    12 = critical asset + severe security event
    13 = critical asset + very high-confidence compromise indicator
    14 = critical asset + confirmed/near-confirmed compromise
-->

<!--
================================================================
     CRITICAL ASSET DEFINITIONS

================================================================
-->

<var
name="CRITICAL_ASSETS">^(dc01|dc02|ad01|ad02|domain-controller-01|d
omain-controller-02|db01|db02|database-01|database-02|prod-db01|pro
d-db02|web01|web02|prod-web01|prod-web02|app01|app02|prod-app01|pro
d-app02|mail01|mail02|backup01|backup02|siem01|wazuh-manager01|wazu
h-indexer01)$</var>

<!--
  Optional environment-specific groups.

  Adjust these if you want separate treatment for:
    - domain controllers
    - databases
    - web servers
    - application servers
    - security infrastructure
-->

<var
name="DOMAIN_CONTROLLERS">^(dc01|dc02|ad01|ad02|domain-controller-0
1|domain-controller-02)$</var>
```

```
<var
name="DATABASE_SERVERS">^(db01|db02|database-01|database-02|prod-db
01|prod-db02)$</var>

<var
name="SECURITY_SERVERS">^(siem01|wazuh-manager01|wazuh-indexer01)$<
/var>


<!--
================================================================
     1. CRITICAL ASSET + AUTHENTICATION EVENTS

================================================================
-->

<group name="critical_asset,authentication,">

  <!-- Authentication failures on critical assets.
       Level 10: important, but not automatically a compromise. -->
  <rule id="110900" level="10">
    <if_group>authentication_failed</if_group>
    <hostname>$CRITICAL_ASSETS</hostname>
    <description>Critical asset: authentication failure detected on
$(hostname)</description>
    <group>critical_asset,authentication_failed,</group>
  </rule>

  <!-- Successful authentication on a critical asset.
       Kept at level 10 because successful authentication alone is
not malicious. -->
  <rule id="110901" level="10">
    <if_group>authentication_success</if_group>
    <hostname>$CRITICAL_ASSETS</hostname>
    <description>Critical asset: successful authentication detected
on $(hostname)</description>
```

```
    <group>critical_asset,authentication_success,</group>
  </rule>

  <!-- Repeated authentication failure rules should normally be
handled
       by correlation rules, but the underlying critical-asset
event
       should still receive elevated priority. -->
  <rule id="110902" level="10">
    <if_group>invalid_login</if_group>
    <hostname>$CRITICAL_ASSETS</hostname>
    <description>Critical asset: invalid login activity detected on
$(hostname)</description>
    <group>critical_asset,invalid_login,</group>
  </rule>

</group>


<!--
=================================================================
     2. CRITICAL ASSET + PRIVILEGE ESCALATION

=================================================================
-->

<group name="critical_asset,privilege_escalation,">

  <rule id="110910" level="12">
    <if_group>privilege_escalation</if_group>
    <hostname>$CRITICAL_ASSETS</hostname>
    <description>Critical asset: privilege escalation activity
detected on $(hostname)</description>
    <mitre>
      <id>T1068</id>
    </mitre>
```

```
    <group>critical_asset,privilege_escalation,</group>
  </rule>

  <rule id="110911" level="12">
    <if_group>sudo</if_group>
    <hostname>$CRITICAL_ASSETS</hostname>
    <description>Critical asset: sudo/privileged execution detected
on $(hostname)</description>
    <mitre>
      <id>T1548</id>
    </mitre>
    <group>critical_asset,privilege_escalation,</group>
  </rule>

</group>


<!--
================================================================
     3. CRITICAL ASSET + FILE INTEGRITY

================================================================
-->

<group name="critical_asset,syscheck,">

  <!-- Existing FIM modification event -->
  <rule id="110920" level="11">
    <if_group>syscheck</if_group>
    <hostname>$CRITICAL_ASSETS</hostname>
    <field name="syscheck.event">modified</field>
    <description>Critical asset: monitored file modified on
$(hostname)</description>
    <group>critical_asset,syscheck,file_modification,</group>
  </rule>
```

```
  <!-- Fallback for standard Wazuh FIM events where the event type
is
       represented by the rule/group rather than syscheck.event.
-->
  <rule id="110921" level="10">
    <if_group>syscheck</if_group>
    <hostname>$CRITICAL_ASSETS</hostname>
    <description>Critical asset: file integrity event detected on
$(hostname)</description>
    <group>critical_asset,syscheck,</group>
  </rule>

</group>


<!--
=================================================================
     4. CRITICAL SYSTEM FILES

=================================================================
-->

<group name="critical_asset,critical_file,">

  <!-- Linux authentication databases -->
  <rule id="110930" level="13">
    <if_group>syscheck</if_group>
    <hostname>$CRITICAL_ASSETS</hostname>
    <field name="file">^/etc/passwd$</field>
    <description>Critical asset: /etc/passwd modified on
$(hostname)</description>
    <mitre>
      <id>T1098</id>
    </mitre>

<group>critical_asset,critical_file,account_manipulation,</group>
```

```
  </rule>

  <rule id="110931" level="13">
    <if_group>syscheck</if_group>
    <hostname>$CRITICAL_ASSETS</hostname>
    <field name="file">^/etc/shadow$</field>
    <description>Critical asset: /etc/shadow modified on
$(hostname)</description>
    <mitre>
      <id>T1003</id>
    </mitre>
    <group>critical_asset,critical_file,credential_access,</group>
  </rule>

  <rule id="110932" level="12">
    <if_group>syscheck</if_group>
    <hostname>$CRITICAL_ASSETS</hostname>
    <field name="file">^/etc/sudoers$|^/etc/sudoers\.d/</field>
    <description>Critical asset: sudo configuration modified on
$(hostname)</description>
    <mitre>
      <id>T1548.003</id>
    </mitre>

<group>critical_asset,critical_file,privilege_escalation,</group>
  </rule>

  <rule id="110933" level="12">
    <if_group>syscheck</if_group>
    <hostname>$CRITICAL_ASSETS</hostname>
    <field name="file">^/root/\.ssh/authorized_keys$</field>
    <description>Critical asset: root SSH authorized_keys modified
on $(hostname)</description>
    <mitre>
      <id>T1098.004</id>
    </mitre>
```

```
    <group>critical_asset,critical_file,persistence,</group>
  </rule>

</group>


<!--
=================================================================
     5. CRITICAL ASSET + MALWARE

=================================================================
-->

<group name="critical_asset,malware,">

  <rule id="110940" level="14">
    <if_group>malware</if_group>
    <hostname>$CRITICAL_ASSETS</hostname>
    <description>Critical asset: malware detection on
$(hostname)</description>
    <group>critical_asset,malware,</group>
  </rule>

  <rule id="110941" level="14">
    <if_group>rootcheck</if_group>
    <hostname>$CRITICAL_ASSETS</hostname>
    <description>Critical asset: rootkit or rootcheck security
event on $(hostname)</description>
    <mitre>
      <id>T1014</id>
    </mitre>
    <group>critical_asset,rootkit,</group>
  </rule>

</group>
```

```
<!--
=============================================================
     6. CRITICAL ASSET + PERSISTENCE

=============================================================
-->

<group name="critical_asset,persistence,">

  <rule id="110950" level="13">
    <if_group>persistence</if_group>
    <hostname>$CRITICAL_ASSETS</hostname>
    <description>Critical asset: persistence activity detected on
$(hostname)</description>
    <group>critical_asset,persistence,</group>
  </rule>

  <rule id="110951" level="13">
    <if_group>account_manipulation</if_group>
    <hostname>$CRITICAL_ASSETS</hostname>
    <description>Critical asset: account manipulation detected on
$(hostname)</description>
    <mitre>
      <id>T1098</id>
    </mitre>
    <group>critical_asset,account_manipulation,</group>
  </rule>

</group>


<!--
=============================================================
     7. CRITICAL ASSET + LATERAL MOVEMENT
```

```
================================================================
-->

<group name="critical_asset,lateral_movement,">

  <rule id="110960" level="13">
    <if_group>lateral_movement</if_group>
    <hostname>$CRITICAL_ASSETS</hostname>
    <description>Critical asset: lateral movement activity
involving $(hostname)</description>
    <group>critical_asset,lateral_movement,</group>
  </rule>

  <rule id="110961" level="13">
    <if_group>remote_services</if_group>
    <hostname>$CRITICAL_ASSETS</hostname>
    <description>Critical asset: remote service activity detected
on $(hostname)</description>
    <group>critical_asset,remote_services,</group>
  </rule>

</group>


<!--
================================================================
     8. CRITICAL ASSET + COMMAND EXECUTION

================================================================
-->

<group name="critical_asset,execution,">

  <rule id="110970" level="11">
    <if_group>command_execution</if_group>
```

```
    <hostname>$CRITICAL_ASSETS</hostname>
    <description>Critical asset: command execution detected on
$(hostname)</description>
    <group>critical_asset,command_execution,</group>
  </rule>

  <rule id="110971" level="12">
    <if_group>powershell</if_group>
    <hostname>$CRITICAL_ASSETS</hostname>
    <description>Critical asset: PowerShell activity detected on
$(hostname)</description>
    <mitre>
      <id>T1059.001</id>
    </mitre>
    <group>critical_asset,powershell,execution,</group>
  </rule>

  <rule id="110972" level="12">
    <if_group>process_creation</if_group>
    <hostname>$CRITICAL_ASSETS</hostname>
    <description>Critical asset: process creation activity detected
on $(hostname)</description>
    <group>critical_asset,process_creation,execution,</group>
  </rule>

</group>


<!--
=================================================================
     9. CRITICAL ASSET + COMMAND/C2 NETWORK ACTIVITY

=================================================================
-->

<group name="critical_asset,network,">
```

```
  <rule id="110980" level="11">
    <if_group>firewall</if_group>
    <hostname>$CRITICAL_ASSETS</hostname>
    <description>Critical asset: firewall/network security event on
$(hostname)</description>
    <group>critical_asset,firewall,network,</group>
  </rule>

  <rule id="110981" level="12">
    <if_group>connection_attempt</if_group>
    <hostname>$CRITICAL_ASSETS</hostname>
    <description>Critical asset: network connection attempt
detected on $(hostname)</description>
    <group>critical_asset,network,</group>
  </rule>

  <rule id="110982" level="13">
    <if_group>command_and_control</if_group>
    <hostname>$CRITICAL_ASSETS</hostname>
    <description>Critical asset: possible command-and-control
activity on $(hostname)</description>
    <group>critical_asset,command_and_control,</group>
  </rule>

</group>


<!--
=================================================================
     10. CRITICAL ASSET + SECURITY CONFIGURATION CHANGE

=================================================================
-->

<group name="critical_asset,configuration_change,">
```

```
  <rule id="110990" level="11">
    <if_group>configuration_changed</if_group>
    <hostname>$CRITICAL_ASSETS</hostname>
    <description>Critical asset: security-relevant configuration
change on $(hostname)</description>
    <group>critical_asset,configuration_change,</group>
  </rule>

  <rule id="110991" level="12">
    <if_group>security_policy_changed</if_group>
    <hostname>$CRITICAL_ASSETS</hostname>
    <description>Critical asset: security policy changed on
$(hostname)</description>
    <group>critical_asset,security_policy_change,</group>
  </rule>

</group>


<!--
=================================================================
     11. DOMAIN CONTROLLER-SPECIFIC ESCALATION

=================================================================
-->

<group name="critical_asset,domain_controller,">

  <!-- Authentication activity on domain controllers -->
  <rule id="110995" level="11">
    <if_group>authentication_failed</if_group>
    <hostname>$DOMAIN_CONTROLLERS</hostname>
    <description>Domain controller: authentication failure detected
on $(hostname)</description>
```

```
<group>critical_asset,domain_controller,authentication_failed,</gro
up>
  </rule>

  <rule id="110996" level="12">
    <if_group>privilege_escalation</if_group>
    <hostname>$DOMAIN_CONTROLLERS</hostname>
    <description>Domain controller: privilege escalation detected
on $(hostname)</description>
    <mitre>
      <id>T1068</id>
    </mitre>

<group>critical_asset,domain_controller,privilege_escalation,</grou
p>
  </rule>

  <rule id="110997" level="13">
    <if_group>account_manipulation</if_group>
    <hostname>$DOMAIN_CONTROLLERS</hostname>
    <description>Domain controller: account manipulation detected
on $(hostname)</description>
    <mitre>
      <id>T1098</id>
    </mitre>

<group>critical_asset,domain_controller,account_manipulation,</grou
p>
  </rule>

</group>


<!--
================================================================
```

```
     12. SECURITY INFRASTRUCTURE

=============================================================
-->

<group name="critical_asset,security_infrastructure,">

  <rule id="110998" level="13">
    <if_group>authentication_failed</if_group>
    <hostname>$SECURITY_SERVERS</hostname>
    <description>Security infrastructure: authentication failure
detected on $(hostname)</description>

<group>critical_asset,security_infrastructure,authentication_failed
,</group>
  </rule>

  <rule id="110999" level="14">
    <if_group>configuration_changed</if_group>
    <hostname>$SECURITY_SERVERS</hostname>
    <description>Security infrastructure: configuration change
detected on $(hostname)</description>

<group>critical_asset,security_infrastructure,configuration_change,
</group>
  </rule>

</group>
```

Important correction before deploying

I would **not deploy that file unchanged**. The main reason is that `if_group` only works when the upstream Wazuh rule actually assigns that group, and the exact group names vary by the ruleset/event source. Wazuh's rule engine supports `if_group`, `if_matched_group`, `if_sid`, and field matching, but you should verify the groups against your installed ruleset. (Wazuh Documentation)

For example, Wazuh's built-in SSH rule 5710 uses the `authentication_failed` group, so the authentication overlay is a good pattern. (Wazuh Documentation)

Better production design: use an asset CDB

For a large environment, I recommend **not hard-coding dozens/hundreds of hostnames into XML**.

Instead:

```
None
critical-assets
      |
      ├── dc01
      ├── dc02
      ├── db01
      ├── db02
      ├── prod-web01
      └── ...
            |
            ▼
      CDB asset list
            |
            ▼
       Wazuh rule lookup
            |
            ▼
       critical_asset
```

Wazuh supports CDB lists for fast lookups, including hostname-based lookups. (Wazuh Documentation)

That gives you:

```
None
/var/ossec/etc/lists/critical-assets
```

with entries such as:

```
None
dc01:domain_controller
dc02:domain_controller
db01:database
db02:database
prod-web01:web_server
prod-web02:web_server
wazuh-manager01:security_infrastructure
```

Then the rules can determine the asset class without continually editing the XML.

Why this is preferable

Your operational workflow becomes:

```
None
New critical server
        ↓
Add to asset inventory
        ↓
CDB update
        ↓
Wazuh immediately recognizes it
```

rather than:

```
None
New critical server
        ↓
Edit XML
        ↓
Change regex
        ↓
Validate XML
        ↓
Restart Wazuh
```

Windows consideration

There is an important Wazuh-specific issue here: the `<hostname>` rule element works against the decoded hostname/location, while Windows EventChannel/Eventlog sources use static locations such as `WinEvtLog`/`EventChannel`. (Wazuh Documentation)

Therefore, if your critical assets include Windows domain controllers, I recommend identifying them through a **controlled custom field/label or Windows-specific event field**, rather than assuming `<hostname>` will identify the Windows agent.

Wazuh supports agent labels specifically for adding asset metadata to alerts, and those labels can be centrally configured through `agent.conf`. (Wazuh Documentation)

For example, you can assign:

```xml
<agent_config name="^DC01$">
  <labels>
    <label key="asset.criticality">critical</label>
    <label key="asset.role">domain_controller</label>
  </labels>
</agent_config>
```

and:

```xml
<agent_config name="^DB01$">
  <labels>
    <label key="asset.criticality">critical</label>
    <label key="asset.role">database</label>
  </labels>
</agent_config>
```

This gives you much cleaner asset governance.

Validation procedure

Before restarting the manager:

```shell
/var/ossec/bin/wazuh-logtest
```

Wazuh explicitly recommends `wazuh-logtest` for testing custom rules; saving the rule file is sufficient for rule testing, while a manager restart is required to apply the rules to production alert generation. (Wazuh Documentation)

Then check the configuration:

```shell
/var/ossec/bin/wazuh-analysisd -t
```

and, if valid:

```shell
systemctl restart wazuh-manager
```

Finally, inspect:

```shell
/var/ossec/logs/ossec.log
```

for rule-loading errors.

One architectural recommendation

Don't use this file to turn **every event on a critical asset into P0**. That's precisely how alert fatigue comes back.

Use:

```
Normal asset + medium event       → P2
Critical asset + medium event     → P1
Critical asset + high-confidence  → P0/P1
Critical asset + benign event     → remain low
Critical asset + correlated chain → P0
```

The critical-asset attribute should therefore be a **risk multiplier**, not a blanket severity override. This is consistent with Wazuh's rule-level model, where levels 0–16 represent the severity of the triggered detection. (Wazuh Documentation)

If you give me your **Wazuh version + OS mix (Linux/Windows) + critical asset types**, I can make `090-critical-assets.xml` fully concrete against the actual rule IDs/groups in your deployment rather than using the generic event groups above.

Enhanced 060-lateral-movement.xml

Yes. I would make these three files a **layer above `060-lateral-movement.xml`**:

- `061` = behavioral correlation
- `062` = controlled suppression / deduplication
- `063` = critical-asset escalation

I have corrected an important issue from the previous rules: Wazuh's `frequency` is for repeated matches and is `2+`; a single prior event should be correlated with `if_matched_sid` without `frequency`, or by using a subsequent staged rule. Wazuh also supports `same_srcip`, `same_dstip`, `same_field`, `global_frequency`, `ignore`, and level-0 child rules for filtering. (Wazuh Documentation)

The rules below assume the **rule IDs from the preceding `060-lateral-movement.xml`**:

```
None
106000  Windows remote network logon
106001  Windows RDP logon
106010  Explicit credentials
106020  Administrative SMB share
106030  PsExec / remote service
106031  Suspicious service
106040  WMI lateral movement
106041  WMI + suspicious PowerShell
106050  WinRM / remote PowerShell
106070  Privileged logon
106080  Multiple remote logons
106081  Multiple RDP logons
106100  Linux SSH success
106101  Linux SSH failures
106110  Linux SSH session
106910  High-confidence PsExec
```

---

1. `061-lateral-correlation.xml`

This file turns individual detections into **behavior chains**.

XML

```xml
<!--
  061-lateral-correlation.xml

  Purpose:
    Correlate individual lateral-movement detections from
    060-lateral-movement.xml into higher-confidence incidents.

  Custom rule IDs:
    107000 - 107999

  Dependency:
    060-lateral-movement.xml

  ATT&CK:
    T1021       Remote Services
    T1021.001   RDP
    T1021.002   SMB/Windows Admin Shares
    T1021.004   SSH
    T1021.006   Windows Remote Management
    T1047       Windows Management Instrumentation
    T1078       Valid Accounts
    T1569.002   Service Execution
-->

<group name="lateral_correlation,correlation,">

  <!--
================================================================
       1. REMOTE LOGON -> ADMINISTRATIVE SMB SHARE

================================================================
-->

  <rule id="107000" level="12" ignore="300">
    <if_sid>106020</if_sid>
    <if_matched_sid>106000</if_matched_sid>
```

```
    <same_srcip />
    <same_dstip />
    <description>Lateral movement correlation: Windows network
logon followed by administrative SMB share access from the same
source to the same destination.</description>
    <mitre>
      <id>T1021.002</id>
      <id>T1078</id>
    </mitre>
    <group>lateral_movement,correlation,remote_logon,smb,</group>
  </rule>


  <!--
================================================================
       2. REMOTE LOGON -> PSEXEC

================================================================
-->

  <rule id="107001" level="14" ignore="300">
    <if_sid>106030</if_sid>
    <if_matched_sid>106000</if_matched_sid>
    <same_srcip />
    <description>High-confidence lateral movement: Windows remote
network logon followed by remote service execution from the same
source.</description>
    <mitre>
      <id>T1021</id>
      <id>T1569.002</id>
      <id>T1078</id>
    </mitre>

<group>lateral_movement,correlation,psexec,remote_service,high_conf
idence,</group>
  </rule>
```

```
  <!--
=================================================================
       3. ADMIN SHARE -> PSEXEC

=================================================================
-->

  <rule id="107002" level="14" ignore="300">
    <if_sid>106030</if_sid>
    <if_matched_sid>106020</if_matched_sid>
    <same_srcip />
    <same_dstip />
    <description>High-confidence lateral movement: Administrative
SMB share access followed by remote service
execution.</description>
    <mitre>
      <id>T1021.002</id>
      <id>T1569.002</id>
    </mitre>

<group>lateral_movement,correlation,smb,psexec,high_confidence,</gr
oup>
  </rule>


  <!--
=================================================================
       4. REMOTE LOGON -> SUSPICIOUS SERVICE

=================================================================
-->

  <rule id="107003" level="13" ignore="300">
    <if_sid>106031</if_sid>
```

```
    <if_matched_sid>106000</if_matched_sid>
    <same_srcip />
    <description>Lateral movement correlation: Remote Windows
network logon followed by suspicious service
installation.</description>
    <mitre>
      <id>T1021</id>
      <id>T1569.002</id>
      <id>T1543.003</id>
    </mitre>

<group>lateral_movement,correlation,remote_service,suspicious_servi
ce,</group>
  </rule>


  <!--
================================================================
       5. RDP -> PRIVILEGED LOGON

================================================================
-->

  <rule id="107004" level="13" ignore="300">
    <if_sid>106070</if_sid>
    <if_matched_sid>106001</if_matched_sid>
    <same_dstip />
    <description>Lateral movement correlation: RDP access followed
by privileged logon activity on the same destination.</description>
    <mitre>
      <id>T1021.001</id>
      <id>T1078</id>
      <id>T1548</id>
    </mitre>
```

```
<group>lateral_movement,correlation,rdp,privilege_escalation,</grou
p>
  </rule>


  <!--
================================================================
       6. RDP -> SUSPICIOUS SERVICE

================================================================
-->

  <rule id="107005" level="14" ignore="300">
    <if_sid>106031</if_sid>
    <if_matched_sid>106001</if_matched_sid>
    <same_dstip />
    <description>High-risk lateral movement: RDP session followed
by suspicious service installation.</description>
    <mitre>
      <id>T1021.001</id>
      <id>T1543.003</id>
    </mitre>

<group>lateral_movement,correlation,rdp,persistence,remote_service,
</group>
  </rule>


  <!--
================================================================
       7. RDP -> WMI

================================================================
-->
```

```
  <rule id="107006" level="13" ignore="300">
    <if_sid>106040</if_sid>
    <if_matched_sid>106001</if_matched_sid>
    <same_dstip />
    <description>Lateral movement correlation: RDP access followed
by suspicious WMI process execution.</description>
    <mitre>
      <id>T1021.001</id>
      <id>T1047</id>
    </mitre>
    <group>lateral_movement,correlation,rdp,wmi,</group>
  </rule>


  <!--
================================================================
       8. REMOTE LOGON -> WMI

================================================================
-->

  <rule id="107007" level="13" ignore="300">
    <if_sid>106040</if_sid>
    <if_matched_sid>106000</if_matched_sid>
    <same_srcip />
    <description>Lateral movement correlation: Windows network
logon followed by WMI-based process execution.</description>
    <mitre>
      <id>T1021</id>
      <id>T1047</id>
    </mitre>

<group>lateral_movement,correlation,wmi,remote_execution,</group>
  </rule>
```

```
  <!--
================================================================
       9. WMI -> SUSPICIOUS POWERSHELL

================================================================
-->

  <rule id="107008" level="14" ignore="300">
    <if_sid>106041</if_sid>
    <if_matched_sid>106040</if_matched_sid>
    <same_dstip />
    <description>High-confidence lateral movement: WMI process
creation followed by suspicious encoded or download-capable
PowerShell activity.</description>
    <mitre>
      <id>T1047</id>
      <id>T1059.001</id>
    </mitre>

<group>lateral_movement,correlation,wmi,powershell,high_confidence,
</group>
  </rule>


  <!--
================================================================
       10. REMOTE LOGON -> WINRM

================================================================
-->

  <rule id="107009" level="13" ignore="300">
    <if_sid>106050</if_sid>
    <if_matched_sid>106000</if_matched_sid>
    <same_srcip />
```

```
    <description>Lateral movement correlation: Windows remote logon
followed by WinRM or remote PowerShell activity.</description>
    <mitre>
      <id>T1021.006</id>
      <id>T1078</id>
    </mitre>
    <group>lateral_movement,correlation,winrm,powershell,</group>
  </rule>


  <!--
============================================================
       11. EXPLICIT CREDENTIALS -> REMOTE LOGON

============================================================
-->

  <rule id="107010" level="12" ignore="300">
    <if_sid>106000</if_sid>
    <if_matched_sid>106010</if_matched_sid>
    <same_srcip />
    <description>Lateral movement correlation: Explicit credential
use followed by remote Windows network
authentication.</description>
    <mitre>
      <id>T1078</id>
      <id>T1021</id>
    </mitre>

<group>lateral_movement,correlation,explicit_credentials,</group>
  </rule>


  <!--
============================================================
       12. EXPLICIT CREDENTIALS -> RDP
```

```
================================================================
-->

  <rule id="107011" level="13" ignore="300">
    <if_sid>106001</if_sid>
    <if_matched_sid>106010</if_matched_sid>
    <same_srcip />
    <description>Lateral movement correlation: Explicit credential
use followed by RDP access.</description>
    <mitre>
      <id>T1078</id>
      <id>T1021.001</id>
    </mitre>

<group>lateral_movement,correlation,explicit_credentials,rdp,</grou
p>
  </rule>


  <!--
================================================================
       13. MULTIPLE DESTINATIONS FROM ONE SOURCE

================================================================
-->

  <rule id="107020" level="12" frequency="3" timeframe="600">
    <if_matched_sid>106000</if_matched_sid>
    <same_srcip />
    <different_dstip />
    <global_frequency />
    <description>Lateral movement pattern: Same source performed
Windows remote network authentication against multiple destinations
within ten minutes.</description>
    <mitre>
```

```
      <id>T1021</id>
      <id>T1078</id>
    </mitre>

<group>lateral_movement,correlation,multi_host,network_authenticati
on,</group>
  </rule>


  <!--
=================================================================
       14. MULTIPLE RDP DESTINATIONS

=================================================================
-->

  <rule id="107021" level="13" frequency="3" timeframe="600">
    <if_matched_sid>106001</if_matched_sid>
    <same_srcip />
    <different_dstip />
    <global_frequency />
    <description>Lateral movement pattern: Same source established
RDP sessions to multiple destinations within ten
minutes.</description>
    <mitre>
      <id>T1021.001</id>
    </mitre>
    <group>lateral_movement,correlation,rdp,multi_host,</group>
  </rule>


  <!--
=================================================================
       15. MULTIPLE ADMINISTRATIVE SHARES
```

```
================================================================
-->

  <rule id="107022" level="13" frequency="3" timeframe="600">
    <if_matched_sid>106020</if_matched_sid>
    <same_srcip />
    <different_dstip />
    <global_frequency />
    <description>Lateral movement pattern: Same source accessed
administrative SMB shares on multiple destinations.</description>
    <mitre>
      <id>T1021.002</id>
    </mitre>
    <group>lateral_movement,correlation,smb,multi_host,</group>
  </rule>


  <!--
================================================================
       16. MULTIPLE REMOTE SERVICES

================================================================
-->

  <rule id="107023" level="14" frequency="2" timeframe="600">
    <if_matched_sid>106030</if_matched_sid>
    <same_srcip />
    <different_dstip />
    <global_frequency />
    <description>High-confidence lateral movement pattern: Same
source caused remote service execution on multiple
destinations.</description>
    <mitre>
      <id>T1569.002</id>
    </mitre>
```

```
<group>lateral_movement,correlation,psexec,multi_host,high_confiden
ce,</group>
  </rule>


  <!--
=============================================================
       17. SSH MULTI-HOST LATERAL MOVEMENT

=============================================================
-->

  <rule id="107030" level="12" frequency="3" timeframe="600">
    <if_matched_sid>106100</if_matched_sid>
    <same_srcip />
    <different_dstip />
    <global_frequency />
    <description>Linux: Same source established SSH sessions
against multiple destinations within ten minutes.</description>
    <mitre>
      <id>T1021.004</id>
    </mitre>

<group>lateral_movement,correlation,ssh,multi_host,linux,</group>
  </rule>


  <!--
=============================================================
       18. SSH BRUTE FORCE -> SUCCESSFUL SSH

=============================================================
-->

  <rule id="107031" level="13" ignore="300">
```

```
    <if_sid>106100</if_sid>
    <if_matched_sid>106101</if_matched_sid>
    <same_srcip />
    <description>Linux: Repeated SSH authentication failures
followed by successful SSH authentication from the same
source.</description>
    <mitre>
      <id>T1110</id>
      <id>T1021.004</id>
      <id>T1078</id>
    </mitre>

<group>lateral_movement,correlation,ssh,brute_force,account_comprom
ise,</group>
  </rule>


  <!--
================================================================
       19. SSH -> PRIVILEGED ACTIVITY

================================================================

       This rule expects a later event classified by a privilege
       escalation rule. Replace 106070 with your Linux privilege
       escalation rule if you use a different ID.

================================================================
-->

  <rule id="107032" level="13" ignore="300">
    <if_sid>106070</if_sid>
    <if_matched_sid>106100</if_matched_sid>
    <same_srcip />
    <description>Linux: SSH session followed by privileged activity
from the same source.</description>
```

```
    <mitre>
      <id>T1021.004</id>
      <id>T1548</id>
    </mitre>

<group>lateral_movement,correlation,ssh,privilege_escalation,</grou
p>
  </rule>


  <!--
=================================================================
       20. REMOTE LOGON -> ADMIN SHARE -> PSEXEC CHAIN

=================================================================
-->

  <rule id="107040" level="15" ignore="600">
    <if_sid>107002</if_sid>
    <if_matched_sid>107000</if_matched_sid>
    <same_srcip />
    <description>Critical lateral movement chain: Remote network
logon followed by administrative SMB access and remote service
execution.</description>
    <mitre>
      <id>T1021</id>
      <id>T1021.002</id>
      <id>T1569.002</id>
    </mitre>

<group>lateral_movement,correlation,attack_chain,psexec,smb,critica
l_detection,</group>
  </rule>
```

```
  <!--
=================================================================
       21. RDP -> WMI -> POWERSHELL CHAIN

=================================================================
-->

  <rule id="107041" level="15" ignore="600">
    <if_sid>107008</if_sid>
    <if_matched_sid>107006</if_matched_sid>
    <same_dstip />
    <description>Critical lateral movement chain: RDP access
followed by WMI execution and suspicious PowerShell
activity.</description>
    <mitre>
      <id>T1021.001</id>
      <id>T1047</id>
      <id>T1059.001</id>
    </mitre>

<group>lateral_movement,correlation,attack_chain,rdp,wmi,powershell
,critical_detection,</group>
  </rule>


  <!--
=================================================================
       22. REMOTE LOGON -> PRIVILEGE -> PERSISTENCE

=================================================================
-->

  <rule id="107042" level="15" ignore="600">
    <if_sid>106031</if_sid>
    <if_matched_sid>106070</if_matched_sid>
    <same_dstip />
```

```
    <description>Critical lateral movement pattern: Privileged
logon activity followed by suspicious service
installation.</description>
    <mitre>
      <id>T1078</id>
      <id>T1548</id>
      <id>T1543.003</id>
    </mitre>

<group>lateral_movement,correlation,privilege_escalation,persistenc
e,critical_detection,</group>
  </rule>


  <!--
============================================================
       23. GENERIC LATERAL MOVEMENT CORRELATION GROUP

============================================================
-->

  <rule id="107090" level="10">
    <if_group>lateral_movement</if_group>
    <description>Lateral movement event classified for behavioral
correlation.</description>
    <group>lateral_correlation,correlation,</group>
  </rule>


  <!--
============================================================
       24. FINAL HIGH-CONFIDENCE INCIDENT CANDIDATE

============================================================
-->
```

```
  <rule id="107099" level="15" ignore="900">
    <if_group>critical_detection</if_group>
    <description>Critical lateral movement behavior chain detected;
create a single SOC incident and correlate underlying
events.</description>
    <mitre>
      <id>T1021</id>
    </mitre>

<group>lateral_movement,correlation,incident_candidate,critical_det
ection,</group>
  </rule>

</group>
```

The key design choice here is that **correlation rules are not simply repeated copies of the detection rules**. They require a previous event and, where appropriate, the same source/destination. Wazuh supports this pattern with `if_matched_sid`, `same_srcip`, `same_dstip`, and frequency/timeframe correlation. (Wazuh Documentation)

---

2. 062-lateral-suppression.xml

This is deliberately conservative.

I would **not suppress**:

```
None
106030  PsExec
106031  suspicious service
106041  WMI + suspicious PowerShell
106910  high-confidence PsExec
107000+ correlation rules
```

Those should remain visible.

The suppression layer instead handles repetitive, lower-confidence events.

```xml
<!--
  062-lateral-suppression.xml

  Purpose:
    Reduce alert fatigue without suppressing high-confidence
    lateral-movement detections.

  Custom rule IDs:
    108000 - 108999

  Required CDB lists:
    etc/lists/lateral-approved-sources
    etc/lists/lateral-approved-users
    etc/lists/lateral-approved-destinations

  IMPORTANT:
    CDB list membership must represent documented, approved
    administrative activity. Do not populate these lists broadly.
-->

<group name="lateral_suppression,suppression,">

  <!--
================================================================
       S1. APPROVED SOURCE -> REMOTE NETWORK LOGON

================================================================
-->

  <rule id="108000" level="0">
    <if_sid>106000</if_sid>
    <list field="srcip"
lookup="address_match_key">etc/lists/lateral-approved-sources</list
>
```

```
    <description>Suppressed expected Windows remote network
authentication from an approved administrative
source.</description>
    <group>suppression,approved_source,authentication,</group>
  </rule>


  <!--
=================================================================
       S2. APPROVED SOURCE -> RDP

=================================================================
-->

  <rule id="108001" level="0">
    <if_sid>106001</if_sid>
    <list field="srcip"
lookup="address_match_key">etc/lists/lateral-approved-sources</list
>
    <description>Suppressed expected RDP authentication from an
approved administrative source.</description>
    <group>suppression,approved_source,rdp,</group>
  </rule>


  <!--
=================================================================
       S3. APPROVED SOURCE -> EXPLICIT CREDENTIAL USE

=================================================================
-->

  <rule id="108002" level="0">
    <if_sid>106010</if_sid>
```

```
    <list field="srcip"
lookup="address_match_key">etc/lists/lateral-approved-sources</list
>
    <description>Suppressed explicit credential activity
originating from an approved administrative source.</description>

<group>suppression,approved_source,explicit_credentials,</group>
  </rule>


  <!--
================================================================
       S4. APPROVED SOURCE -> ADMINISTRATIVE SMB

================================================================
-->

  <rule id="108003" level="0">
    <if_sid>106020</if_sid>
    <list field="srcip"
lookup="address_match_key">etc/lists/lateral-approved-sources</list
>
    <description>Suppressed administrative SMB share activity from
an approved management source.</description>
    <group>suppression,approved_source,smb,admin_share,</group>
  </rule>


  <!--
================================================================
       S5. APPROVED SOURCE + APPROVED USER -> REMOTE LOGON

================================================================
-->

  <rule id="108004" level="0">
```

```
    <if_sid>106000</if_sid>
    <list field="srcip"
lookup="address_match_key">etc/lists/lateral-approved-sources</list
>
    <list
field="win.eventdata.targetUserName">etc/lists/lateral-approved-use
rs</list>
    <description>Suppressed expected remote network authentication
from an approved source using an approved administrative
account.</description>
    <group>suppression,approved_source,approved_user,</group>
  </rule>


  <!--
================================================================
       S6. APPROVED DESTINATION -> ROUTINE REMOTE LOGON

================================================================
-->

  <rule id="108005" level="0">
    <if_sid>106000</if_sid>
    <list
field="win.system.computer">etc/lists/lateral-approved-destinations
</list>
    <description>Suppressed routine remote network authentication
to a destination explicitly approved for administrative
activity.</description>
    <group>suppression,approved_destination,authentication,</group>
  </rule>


  <!--
================================================================
       S7. AGGREGATE REPEATED REMOTE LOGONS
```

```
================================================================
-->

  <rule id="108010" level="3" frequency="10" timeframe="300"
ignore="300">
    <if_matched_sid>106000</if_matched_sid>
    <same_srcip />
    <same_dstip />
    <description>Aggregated repeated Windows remote network
authentication events from the same source to the same
destination.</description>

<group>lateral_movement,aggregation,suppression_control,</group>
  </rule>


  <!--
================================================================
       S8. AGGREGATE REPEATED RDP

================================================================
-->

  <rule id="108011" level="3" frequency="5" timeframe="300"
ignore="300">
    <if_matched_sid>106001</if_matched_sid>
    <same_srcip />
    <same_dstip />
    <description>Aggregated repeated RDP authentication events from
the same source to the same destination.</description>
    <mitre>
      <id>T1021.001</id>
    </mitre>
```

```
<group>lateral_movement,rdp,aggregation,suppression_control,</group
>
  </rule>


  <!--
=============================================================
       S9. AGGREGATE ADMINISTRATIVE SHARE ACCESS

=============================================================
-->

  <rule id="108012" level="4" frequency="10" timeframe="300"
ignore="300">
    <if_matched_sid>106020</if_matched_sid>
    <same_srcip />
    <same_dstip />
    <description>Aggregated repeated administrative SMB share
access from the same source to the same destination.</description>
    <mitre>
      <id>T1021.002</id>
    </mitre>

<group>lateral_movement,smb,aggregation,suppression_control,</group
>
  </rule>


  <!--
=============================================================
       S10. AGGREGATE SSH SUCCESS

=============================================================
-->
```

```
  <rule id="108013" level="3" frequency="10" timeframe="300"
ignore="300">
    <if_matched_sid>106100</if_matched_sid>
    <same_srcip />
    <same_dstip />
    <description>Aggregated repeated successful SSH sessions
between the same source and destination.</description>
    <mitre>
      <id>T1021.004</id>
    </mitre>

<group>lateral_movement,ssh,aggregation,suppression_control,</group
>
  </rule>


  <!--
=================================================================
       S11. APPROVED SSH SOURCE

=================================================================
-->

  <rule id="108014" level="0">
    <if_sid>106100</if_sid>
    <list field="srcip"
lookup="address_match_key">etc/lists/lateral-approved-sources</list
>
    <description>Suppressed expected SSH access from an approved
administrative source.</description>
    <mitre>
      <id>T1021.004</id>
    </mitre>
    <group>suppression,approved_source,ssh,</group>
  </rule>
```

```
  <!--
================================================================
       S12. APPROVED SOURCE -> SSH SESSION

================================================================
-->

  <rule id="108015" level="0">
    <if_sid>106110</if_sid>
    <list field="srcip"
lookup="address_match_key">etc/lists/lateral-approved-sources</list
>
    <description>Suppressed expected SSH administrative session
from an approved source.</description>
    <group>suppression,approved_source,ssh,</group>
  </rule>


  <!--
================================================================
       S13. LOW-VOLUME REMOTE LOGON FLOOD CONTROL

================================================================
-->

  <rule id="108020" level="4" frequency="20" timeframe="300"
ignore="300">
    <if_matched_sid>106000</if_matched_sid>
    <same_srcip />
    <same_dstip />
    <description>Remote authentication burst aggregated to reduce
duplicate SOC alerts; investigate if correlated rules also
fire.</description>
    <group>lateral_movement,aggregation,alert_fatigue,</group>
  </rule>
```

```
  <!--
=================================================================
       S14. LOW-VOLUME RDP FLOOD CONTROL

=================================================================
-->

  <rule id="108021" level="4" frequency="10" timeframe="300"
ignore="300">
    <if_matched_sid>106001</if_matched_sid>
    <same_srcip />
    <same_dstip />
    <description>RDP authentication burst aggregated to reduce
duplicate SOC alerts.</description>
    <mitre>
      <id>T1021.001</id>
    </mitre>
    <group>lateral_movement,rdp,aggregation,alert_fatigue,</group>
  </rule>


  <!--
=================================================================
       S15. NEVER SUPPRESS HIGH-CONFIDENCE PSEXEC
       This rule intentionally has a normal alert level and
therefore
       provides an explicit protection boundary in this file.

=================================================================
-->

  <rule id="108090" level="14">
    <if_sid>106910</if_sid>
```

```
    <description>Protected detection: High-confidence PsExec
activity must not be suppressed by lateral-movement noise
controls.</description>
    <mitre>
      <id>T1569.002</id>
      <id>T1021.002</id>
    </mitre>

<group>lateral_movement,suppression_protected,high_confidence,</gro
up>
  </rule>


  <!--
============================================================
       S16. NEVER SUPPRESS SUSPICIOUS SERVICE

============================================================
-->

  <rule id="108091" level="12">
    <if_sid>106031</if_sid>
    <description>Protected detection: Suspicious remote service
installation must remain visible.</description>
    <mitre>
      <id>T1569.002</id>
      <id>T1543.003</id>
    </mitre>

<group>lateral_movement,suppression_protected,remote_service,</grou
p>
  </rule>


  <!--
============================================================
```

```
      S17. NEVER SUPPRESS CORRELATED INCIDENTS

================================================================
-->

  <rule id="108099" level="15">
    <if_group>lateral_correlation</if_group>
    <description>Protected detection: Correlated lateral-movement
incidents bypass suppression controls.</description>
    <mitre>
      <id>T1021</id>
    </mitre>

<group>lateral_movement,suppression_protected,incident_candidate,</
group>
  </rule>

</group>
```

Important suppression point

Level-0 child rules are a supported Wazuh mechanism for ignoring known-benign events. Wazuh explicitly documents level-0 child rules for this purpose. (Wazuh Documentation)

However, **do not use the approved-source list as a blanket exemption for every lateral-movement rule**. A management server can itself be compromised. That's why the file deliberately protects PsExec, suspicious service creation and correlated incidents.

---

3. 063-lateral-critical-assets.xml

This is the most important file for your alert-fatigue architecture.

Instead of making everything high severity, we make the **same behavior more severe when it occurs on a crown-jewel asset**.

For example:

None

```
Normal workstation + RDP
        → level 8

Domain controller + RDP
        → level 14

Domain controller + RDP → WMI → PowerShell
        → level 15
```

XML

```
<!--
  063-lateral-critical-assets.xml

  Purpose:
    Increase lateral-movement severity when the destination
    is a business-critical / crown-jewel asset.

  Custom rule IDs:
    109000 - 109999

  Required CDB list:
    etc/lists/critical-assets

  Example critical assets:
    Domain controllers
    Identity servers
    PKI servers
    Tier-0 administration systems
    Hypervisors
    Backup servers
    Security infrastructure
    Database servers
    Payment systems
    Production management servers
```

```
  IMPORTANT:
    Populate the CDB list only with assets that genuinely require
    elevated response priority.
-->

<group name="critical_assets,lateral_movement,asset_criticality,">

  <!--
=============================================================
       1. CRITICAL ASSET + REMOTE NETWORK LOGON

=============================================================
-->

  <rule id="109000" level="13">
    <if_sid>106000</if_sid>
    <list field="win.system.computer"
lookup="match_key">etc/lists/critical-assets</list>
    <description>Critical asset: Windows remote network logon
detected on a crown-jewel system from
$(win.eventdata.ipAddress).</description>
    <mitre>
      <id>T1021</id>
      <id>T1078</id>
    </mitre>

<group>lateral_movement,critical_asset,remote_logon,crown_jewel,</g
roup>
  </rule>


  <!--
=============================================================
       2. CRITICAL ASSET + RDP
```

```
================================================================
-->

  <rule id="109001" level="14">
    <if_sid>106001</if_sid>
    <list field="win.system.computer"
lookup="match_key">etc/lists/critical-assets</list>
    <description>Critical asset: RDP access detected against
crown-jewel system from $(win.eventdata.ipAddress).</description>
    <mitre>
      <id>T1021.001</id>
      <id>T1078</id>
    </mitre>
    <group>lateral_movement,critical_asset,rdp,crown_jewel,</group>
  </rule>


  <!--
================================================================
       3. CRITICAL ASSET + EXPLICIT CREDENTIALS

================================================================
-->

  <rule id="109002" level="13">
    <if_sid>106010</if_sid>
    <list field="win.system.computer"
lookup="match_key">etc/lists/critical-assets</list>
    <description>Critical asset: Explicit credential use detected
on crown-jewel system.</description>
    <mitre>
      <id>T1078</id>
    </mitre>
```

```
<group>lateral_movement,critical_asset,explicit_credentials,crown_j
ewel,</group>
  </rule>


  <!--
================================================================
       4. CRITICAL ASSET + ADMINISTRATIVE SMB

================================================================
-->

  <rule id="109003" level="14">
    <if_sid>106020</if_sid>
    <list field="win.system.computer"
lookup="match_key">etc/lists/critical-assets</list>
    <description>Critical asset: Administrative SMB share access
detected on crown-jewel system.</description>
    <mitre>
      <id>T1021.002</id>
    </mitre>

<group>lateral_movement,critical_asset,smb,admin_share,crown_jewel,
</group>
  </rule>


  <!--
================================================================
       5. CRITICAL ASSET + PSEXEC

================================================================
-->

  <rule id="109004" level="15">
```

```
    <if_sid>106030</if_sid>
    <list field="win.system.computer"
lookup="match_key">etc/lists/critical-assets</list>
    <description>Critical asset: Remote service execution detected
on crown-jewel system.</description>
    <mitre>
      <id>T1569.002</id>
    </mitre>

<group>lateral_movement,critical_asset,psexec,remote_service,crown_
jewel,</group>
  </rule>


  <!--
================================================================
       6. CRITICAL ASSET + SUSPICIOUS SERVICE

================================================================
-->

  <rule id="109005" level="15">
    <if_sid>106031</if_sid>
    <list field="win.system.computer"
lookup="match_key">etc/lists/critical-assets</list>
    <description>Critical asset: Suspicious service installation
detected on crown-jewel system.</description>
    <mitre>
      <id>T1543.003</id>
      <id>T1569.002</id>
    </mitre>

<group>lateral_movement,critical_asset,suspicious_service,persisten
ce,crown_jewel,</group>
  </rule>
```

```
  <!--
=============================================================
       7. CRITICAL ASSET + WMI

=============================================================
-->

  <rule id="109006" level="14">
    <if_sid>106040</if_sid>
    <list field="win.system.computer"
lookup="match_key">etc/lists/critical-assets</list>
    <description>Critical asset: Suspicious WMI process execution
detected on crown-jewel system.</description>
    <mitre>
      <id>T1047</id>
    </mitre>

<group>lateral_movement,critical_asset,wmi,remote_execution,crown_j
ewel,</group>
  </rule>


  <!--
=============================================================
       8. CRITICAL ASSET + WMI POWERSHELL

=============================================================
-->

  <rule id="109007" level="15">
    <if_sid>106041</if_sid>
    <list field="win.system.computer"
lookup="match_key">etc/lists/critical-assets</list>
    <description>Critical asset: WMI spawned suspicious PowerShell
activity on crown-jewel system.</description>
```

```
    <mitre>
      <id>T1047</id>
      <id>T1059.001</id>
    </mitre>

<group>lateral_movement,critical_asset,wmi,powershell,crown_jewel,<
/group>
  </rule>


  <!--
============================================================
       9. CRITICAL ASSET + WINRM

============================================================
-->

  <rule id="109008" level="14">
    <if_sid>106050</if_sid>
    <list field="win.system.computer"
lookup="match_key">etc/lists/critical-assets</list>
    <description>Critical asset: WinRM or remote PowerShell
activity detected against crown-jewel system.</description>
    <mitre>
      <id>T1021.006</id>
      <id>T1059.001</id>
    </mitre>

<group>lateral_movement,critical_asset,winrm,powershell,crown_jewel
,</group>
  </rule>


  <!--
============================================================
       10. CRITICAL ASSET + PRIVILEGED LOGON
```

```
================================================================
-->

  <rule id="109009" level="14">
    <if_sid>106070</if_sid>
    <list field="win.system.computer"
lookup="match_key">etc/lists/critical-assets</list>
    <description>Critical asset: Highly privileged Windows logon
activity detected on crown-jewel system.</description>
    <mitre>
      <id>T1078</id>
      <id>T1548</id>
    </mitre>

<group>lateral_movement,critical_asset,privileged_logon,privilege_e
scalation,crown_jewel,</group>
  </rule>


  <!--
================================================================
       11. CRITICAL ASSET + MULTIPLE REMOTE LOGONS

================================================================
-->

  <rule id="109010" level="14">
    <if_sid>106080</if_sid>
    <list field="win.system.computer"
lookup="match_key">etc/lists/critical-assets</list>
    <description>Critical asset: Repeated remote Windows logons
detected against crown-jewel system.</description>
    <mitre>
      <id>T1021</id>
    </mitre>
```

```
<group>lateral_movement,critical_asset,correlation,crown_jewel,</gr
oup>
  </rule>


  <!--
============================================================
       12. CRITICAL ASSET + MULTIPLE RDP

============================================================
-->

  <rule id="109011" level="15">
    <if_sid>106081</if_sid>
    <list field="win.system.computer"
lookup="match_key">etc/lists/critical-assets</list>
    <description>Critical asset: Repeated RDP activity detected
against crown-jewel system.</description>
    <mitre>
      <id>T1021.001</id>
    </mitre>

<group>lateral_movement,critical_asset,rdp,correlation,crown_jewel,
</group>
  </rule>


  <!--
============================================================
       13. CRITICAL ASSET + HIGH-CONFIDENCE PSEXEC

============================================================
-->

  <rule id="109012" level="16">
```

```
    <if_sid>106910</if_sid>
    <list field="win.system.computer"
lookup="match_key">etc/lists/critical-assets</list>
    <description>CRITICAL: High-confidence PsExec activity detected
on crown-jewel system.</description>
    <mitre>
      <id>T1569.002</id>
      <id>T1021.002</id>
    </mitre>

<group>lateral_movement,critical_asset,psexec,critical_detection,cr
own_jewel,</group>
  </rule>


  <!--
================================================================
       14. CRITICAL ASSET + CORRELATED LATERAL MOVEMENT

================================================================
-->

  <rule id="109020" level="15">
    <if_group>lateral_correlation</if_group>
    <list field="win.system.computer"
lookup="match_key">etc/lists/critical-assets</list>
    <description>Critical asset: Correlated lateral-movement
behavior detected on crown-jewel system.</description>
    <mitre>
      <id>T1021</id>
    </mitre>

<group>lateral_movement,critical_asset,correlation,incident_candida
te,crown_jewel,</group>
  </rule>
```

```
  <!--
=================================================================
       15. CRITICAL ASSET + MULTI-HOST LATERAL MOVEMENT

=================================================================
-->

  <rule id="109021" level="15" frequency="2" timeframe="600">
    <if_matched_group>lateral_correlation</if_matched_group>
    <same_srcip />
    <different_dstip />
    <global_frequency />
    <description>Critical lateral movement campaign: Same source
generated correlated lateral-movement activity against multiple
destinations.</description>
    <mitre>
      <id>T1021</id>
    </mitre>

<group>lateral_movement,critical_asset,correlation,multi_host,campa
ign,</group>
  </rule>


  <!--
=================================================================
       16. CRITICAL ASSET + ATTACK CHAIN

=================================================================
-->

  <rule id="109090" level="16" ignore="900">
    <if_group>critical_detection</if_group>
    <list field="win.system.computer"
lookup="match_key">etc/lists/critical-assets</list>
```

```
    <description>CRITICAL INCIDENT: High-confidence
lateral-movement attack chain detected on a crown-jewel
asset.</description>
    <mitre>
      <id>T1021</id>
      <id>T1078</id>
      <id>T1569.002</id>
    </mitre>

<group>lateral_movement,critical_asset,critical_detection,incident_
candidate,crown_jewel,</group>
  </rule>


  <!--
============================================================
       17. LINUX CRITICAL ASSET + SSH

============================================================
-->

  <rule id="109100" level="14">
    <if_sid>106100</if_sid>
    <list field="hostname"
lookup="match_key">etc/lists/critical-assets</list>
    <description>Critical asset: SSH access detected against
crown-jewel Linux system.</description>
    <mitre>
      <id>T1021.004</id>
    </mitre>

<group>lateral_movement,critical_asset,ssh,linux,crown_jewel,</grou
p>
  </rule>
```

```
  <!--
=============================================================
       18. LINUX CRITICAL ASSET + SSH BRUTE FORCE

=============================================================
-->

  <rule id="109101" level="15">
    <if_sid>106101</if_sid>
    <list field="hostname"
lookup="match_key">etc/lists/critical-assets</list>
    <description>Critical asset: SSH brute-force activity detected
against crown-jewel Linux system.</description>
    <mitre>
      <id>T1110</id>
      <id>T1021.004</id>
    </mitre>

<group>lateral_movement,critical_asset,ssh,brute_force,crown_jewel,
</group>
  </rule>


  <!--
=============================================================
       19. LINUX CRITICAL ASSET + SSH SESSION

=============================================================
-->

  <rule id="109102" level="14">
    <if_sid>106110</if_sid>
    <list field="hostname"
lookup="match_key">etc/lists/critical-assets</list>
    <description>Critical asset: Remote SSH session detected
against crown-jewel Linux system.</description>
```

```
    <mitre>
      <id>T1021.004</id>
    </mitre>

<group>lateral_movement,critical_asset,ssh,linux,crown_jewel,</grou
p>
  </rule>

</group>
```

Wazuh supports CDB lookups against decoded fields, including dynamic fields, and documents CDB lists specifically for allowlists/denylists. (Wazuh Documentation)

Required CDB lists

The three XML files above are intentionally driven by external lists rather than hard-coded IP addresses.

Create:

```
None
/var/ossec/etc/lists/critical-assets
/var/ossec/etc/lists/lateral-approved-sources
/var/ossec/etc/lists/lateral-approved-users
/var/ossec/etc/lists/lateral-approved-destinations
```

For example:

```
None
# /var/ossec/etc/lists/critical-assets

dc01.example.com:
dc02.example.com:
pki01.example.com:
```

```
backup01.example.com:
hyperv01.example.com:
```

Approved management sources:

```
None
# /var/ossec/etc/lists/lateral-approved-sources

10.10.10.10:
10.10.10.11:
10.10.20.50:
```

Approved administrative accounts:

```
None
# /var/ossec/etc/lists/lateral-approved-users

DOMAIN\svc_sccm:
DOMAIN\svc_backup:
DOMAIN\svc_patching:
```

Approved destinations:

```
None
# /var/ossec/etc/lists/lateral-approved-destinations

patching01.example.com:
sccm01.example.com:
```

CDB list files use `key:` syntax, and Wazuh automatically builds/loads them when the analysis engine starts; the list must also be declared under the manager's `<ruleset>` configuration. (Wazuh Documentation)

Add the lists to `/var/ossec/etc/ossec.conf`:

XML

```xml
<ruleset>
    <decoder_dir>ruleset/decoders</decoder_dir>
    <rule_dir>ruleset/rules</rule_dir>

    <!-- Existing lists -->
    <!-- ... -->

    <!-- Lateral movement lists -->
    <list>etc/lists/critical-assets</list>
    <list>etc/lists/lateral-approved-sources</list>
    <list>etc/lists/lateral-approved-users</list>
    <list>etc/lists/lateral-approved-destinations</list>

    <!-- Custom rules -->
    <decoder_dir>etc/decoders</decoder_dir>
    <rule_dir>etc/rules</rule_dir>
</ruleset>
```

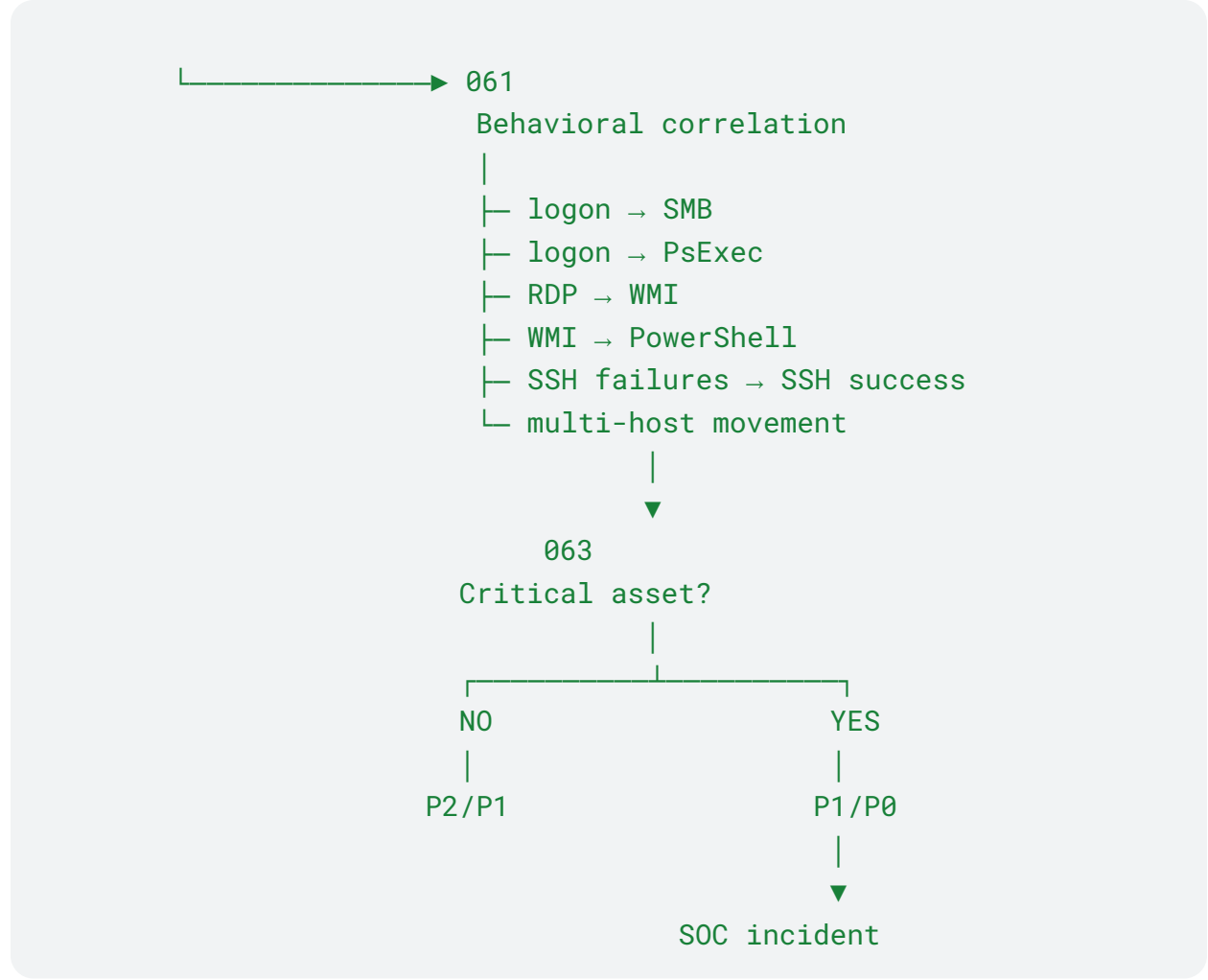
Wazuh's current documentation recommends putting larger custom rule sets under `/var/ossec/etc/rules/`, using custom IDs in the `100000–120000` range, and restarting the manager after loading them. (Wazuh Documentation)

Final rule hierarchy

With all four files installed, your architecture becomes:

None

```
060
Lateral-movement detection
        |
        |──────────────────▶ 062
        |                    Known-good / repetitive activity
        |                    ↓
        |                    Suppress / aggregate
        |
```



One important implementation detail

Your **critical-asset rules should win conceptually over suppression**, but you should still test the actual rule ordering with your exact Wazuh version and event fields. Wazuh processes level-0 rules specially, and `if_matched_sid` does not use level-0 rules as historical correlation events. (Wazuh Documentation)

Run:

```shell
/var/ossec/bin/wazuh-logtest
```

and test these sequences in the **same logtest session**:

```
None
4624 Type 3
5140 ADMIN$
7045 PSEXESVC
```

then:

```
None
4624 Type 10
WMI / Sysmon Event 1
PowerShell / Sysmon Event 1
```

and finally repeat the same scenarios against a hostname present in `critical-assets`.

`wazuh-logtest` maintains rule counters within a session, making it appropriate for testing frequency/timeframe correlations. (Wazuh Documentation)

Also note: the `061` rules are intentionally dependent on the exact IDs from the previous `060` file. If you changed any of those IDs or your actual decoder produces different field names, those references need to be adjusted before deployment.

095-governance-log.xml

Based on the attached architecture, `095-governance-log.xml` should be the **governance/audit-control layer**, not another detection layer. The source document defines governance around separation of duties, owner/review-date enforcement, suppression-volume drift, enrichment freshness, ATT&CK coverage regression, and a time-boxed correlation kill-switch.

One important implementation point: **Wazuh cannot itself calculate "30-day baseline," ">3σ," or determine whether today's date is past an arbitrary review date from static XML.** Those conditions should be calculated by your governance/automation job, which emits a structured governance event; this Wazuh file then validates, classifies, and alerts on that event.

Below is the complete rule file I recommend.

095-governance-log.xml

XML

```
<!--
  ================================================================
  WAZUH GOVERNANCE / CONTROL-PLANE RULES
  File:
    /var/ossec/etc/rules/095-governance-log.xml

  Purpose:
    Governance and audit-control layer for the alert-fatigue
    reduction architecture.

  Responsibilities:
    1. Record governance lifecycle events.
    2. Enforce separation-of-duties signals.
    3. Detect missing/invalid ownership metadata.
    4. Detect review-date violations.
    5. Detect expired suppression rules.
    6. Detect suppression-volume drift.
    7. Detect suppression escape events.
    8. Detect stale enrichment.
    9. Detect ATT&CK coverage regression.
   10. Audit correlation-rule kill-switch operations.
   11. Detect unauthorized governance changes.
   12. Provide governance telemetry for reporting.
```

```
  Architecture relationship:

      080-suppression.xml
              |
              v
      governance automation
              |
              v
      governance event/log
              |
              v
      095-governance-log.xml
              |
        +-----+-----+
        |           |
        v           v
     audit       governance
     record       alert
        |           |
        +-----+-----+
              |
              v
       SOC / GRC / SIEM

  IMPORTANT:
    This file assumes a governance process or automation layer
emits
    structured events. Wazuh rule XML cannot itself query
historical
    30-day baselines, calculate standard deviation, compare
arbitrary
    dates against "now", or maintain an external approval database.

  Recommended governance event fields:
```

```
  governance.action
  governance.rule_id
  governance.rule_type
  governance.rule_status
  governance.owner
  governance.author
  governance.approver
  governance.review_date
  governance.expiry_date
  governance.reason
  governance.scope
  governance.change_id
  governance.ticket_id
  governance.drift_sigma
  governance.baseline_count
  governance.current_count
  governance.escape_count
  governance.enrichment_type
  governance.enrichment_age_hours
  governance.correlation_rule
  governance.kill_switch
  governance.kill_switch_expiry
  governance.attck_technique
  governance.actor
  governance.source
  governance.environment
  governance.approval_status

Example JSON event:

{
  "governance": {
    "action": "rule_approved",
    "rule_id": "100840",
    "rule_type": "S5",
    "rule_status": "approved",
```

```
      "owner": "soc-engineering",
      "author": "alice",
      "approver": "bob",
      "review_date": "2026-09-30",
      "expiry_date": "2026-10-31",
      "reason": "Approved SCCM PowerShell administration",
      "scope": "management-hosts",
      "change_id": "CHG-12345",
      "ticket_id": "SEC-12345",
      "actor": "bob"
    }
  }

  Custom rule ID range:
    100950 - 100999

  Severity model:
    0  = governance telemetry / audit only
    5  = governance warning
    7  = governance violation
    9  = high-risk governance violation
   11  = critical governance control failure
   13  = emergency security-control event

  Validate:
    /var/ossec/bin/wazuh-analysisd -t

  Test:
    /var/ossec/bin/wazuh-logtest
  ================================================================
-->


<group
name="custom_governance,governance_log,alert_fatigue_governance,">
```

```
  <!-- ============================================================
       100950
       GOVERNANCE EVENT BASELINE
       ============================================================
-->

  <!--
    Generic governance event.

    Keep at level 0 so normal governance activity becomes telemetry
    rather than another SOC alert.
  -->

  <rule id="100950" level="0">
    <decoded_as>json</decoded_as>
    <field name="governance.action">.+</field>

    <description>
      GOVERNANCE: Governance event received
    </description>

    <group>
      governance,
      governance_audit,
      governance_event,
    </group>
  </rule>


  <!-- ============================================================
       100951
       RULE PROPOSAL CREATED
       ============================================================
-->
```

```
  <rule id="100951" level="0">
    <if_sid>100950</if_sid>
    <field name="governance.action">^rule_proposed$</field>

    <description>
      GOVERNANCE: Suppression/correlation rule proposal created:
      $(governance.rule_id)
    </description>

    <group>
      governance,
      governance_audit,
      rule_lifecycle,
      rule_proposed,
    </group>
  </rule>


  <!-- ============================================================
       100952
       RULE APPROVED
       ============================================================
-->

  <rule id="100952" level="0">
    <if_sid>100950</if_sid>
    <field name="governance.action">^rule_approved$</field>

    <description>
      GOVERNANCE: Rule approved:
      $(governance.rule_id)
    </description>

    <group>
      governance,
      governance_audit,
```

```
      rule_lifecycle,
      rule_approved,
    </group>
  </rule>


  <!-- ============================================================
       100953
       RULE REJECTED
       ============================================================
-->

  <rule id="100953" level="0">
    <if_sid>100950</if_sid>
    <field name="governance.action">^rule_rejected$</field>

    <description>
      GOVERNANCE: Rule rejected:
      $(governance.rule_id)
    </description>

    <group>
      governance,
      governance_audit,
      rule_lifecycle,
      rule_rejected,
    </group>
  </rule>


  <!-- ============================================================
       100954
       RULE ACTIVATED
       ============================================================
-->
```

```
  <rule id="100954" level="0">
    <if_sid>100950</if_sid>
    <field name="governance.action">^rule_activated$</field>

    <description>
      GOVERNANCE: Rule activated:
      $(governance.rule_id)
    </description>

    <group>
      governance,
      governance_audit,
      rule_lifecycle,
      rule_activated,
    </group>
  </rule>


  <!-- ============================================================
       100955
       RULE DISABLED
       ============================================================
-->

  <rule id="100955" level="0">
    <if_sid>100950</if_sid>
    <field name="governance.action">^rule_disabled$</field>

    <description>
      GOVERNANCE: Rule disabled:
      $(governance.rule_id)
    </description>

    <group>
      governance,
      governance_audit,
```

```
      rule_lifecycle,
      rule_disabled,
    </group>
  </rule>


  <!-- ============================================================
       SEPARATION OF DUTIES
       ============================================================
-->

  <!--
    100956
    Self-approval is prohibited.

    The governance automation should set:
      governance.author
      governance.approver

    and emit:
      governance.action=self_approval_detected
  -->

  <rule id="100956" level="11">
    <if_sid>100950</if_sid>
    <field
name="governance.action">^self_approval_detected$</field>

    <description>
      GOVERNANCE VIOLATION: Rule author attempted to approve own
      governance change. Rule=$(governance.rule_id)
      Author=$(governance.author)
      Approver=$(governance.approver)
    </description>

    <mitre>
```

```
      <id>T1098</id>
    </mitre>

    <group>
      governance,
      governance_violation,
      separation_of_duties,
      self_approval,
      high_risk_governance,
    </group>
  </rule>


  <!-- ------------------------------------------------------------
       Missing independent approver
       ------------------------------------------------------------
-->

  <rule id="100957" level="9">
    <if_sid>100950</if_sid>
    <field name="governance.action">^approval_missing$</field>

    <description>
      GOVERNANCE VIOLATION: Rule has no independent approval:
      $(governance.rule_id)
    </description>

    <group>
      governance,
      governance_violation,
      separation_of_duties,
      approval_missing,
    </group>
  </rule>
```

```
  <!-- ------------------------------------------------------------
       Invalid approval
       ------------------------------------------------------------
-->

  <rule id="100958" level="9">
    <if_sid>100950</if_sid>
    <field name="governance.action">^approval_invalid$</field>

    <description>
      GOVERNANCE VIOLATION: Invalid approval detected for rule:
      $(governance.rule_id)
    </description>

    <group>
      governance,
      governance_violation,
      approval_invalid,
    </group>
  </rule>


  <!-- ============================================================
       OWNERSHIP GOVERNANCE
       ============================================================
-->

  <!-- Missing owner -->

  <rule id="100959" level="9">
    <if_sid>100950</if_sid>
    <field name="governance.action">^owner_missing$</field>

    <description>
      GOVERNANCE VIOLATION: Rule has no named owner:
      $(governance.rule_id)
```

```
    </description>

    <group>
      governance,
      governance_violation,
      owner_missing,
      rule_accountability,
    </group>
  </rule>


  <!-- Invalid owner -->

  <rule id="100960" level="7">
    <if_sid>100950</if_sid>
    <field name="governance.action">^owner_invalid$</field>

    <description>
      GOVERNANCE WARNING: Rule owner failed governance validation:
      $(governance.rule_id)
    </description>

    <group>
      governance,
      governance_warning,
      owner_validation,
    </group>
  </rule>


  <!-- ============================================================
       REVIEW-DATE GOVERNANCE
       ============================================================
-->

  <!--
```

```
  The external governance job determines whether the review date
  has passed. Wazuh then alerts on the resulting event.
-->

<rule id="100961" level="7">
  <if_sid>100950</if_sid>
  <field name="governance.action">^review_due$</field>

  <description>
    GOVERNANCE WARNING: Rule review date is approaching:
    $(governance.rule_id)
  </description>

  <group>
    governance,
    governance_warning,
    review_due,
    rule_lifecycle,
  </group>
</rule>


<rule id="100962" level="9">
  <if_sid>100950</if_sid>
  <field name="governance.action">^review_overdue$</field>

  <description>
    GOVERNANCE VIOLATION: Rule is past mandatory review date:
    $(governance.rule_id)
  </description>

  <group>
    governance,
    governance_violation,
    review_overdue,
    rule_lifecycle,
```

```
    </group>
  </rule>


  <!-- ============================================================
       EXPIRATION
       ============================================================
-->

  <rule id="100963" level="9">
    <if_sid>100950</if_sid>
    <field name="governance.action">^rule_expired$</field>

    <description>
      GOVERNANCE VIOLATION: Governed rule has expired:
      $(governance.rule_id)
    </description>

    <group>
      governance,
      governance_violation,
      rule_expired,
      rule_lifecycle,
    </group>
  </rule>


  <!-- Expired rule still active -->

  <rule id="100964" level="11">
    <if_sid>100950</if_sid>
    <field name="governance.action">^expired_rule_active$</field>

    <description>
      CRITICAL GOVERNANCE VIOLATION: Expired rule remains active:
      $(governance.rule_id)
```

```
    </description>

    <group>
      governance,
      critical_governance,
      expired_rule,
      control_failure,
    </group>
  </rule>


  <!-- ============================================================
       S1-S6 GOVERNANCE CLASSIFICATION
       ============================================================
-->

  <!-- S1 Known-good -->

  <rule id="100965" level="0">
    <if_sid>100950</if_sid>
    <field name="governance.rule_type">^S1$</field>

    <description>
      GOVERNANCE: S1 known-good rule event:
      $(governance.rule_id)
    </description>

    <group>
      governance,
      suppression_governance,
      S1,
      known_good,
    </group>
  </rule>
```

```
<!-- S2 Deduplication -->

<rule id="100966" level="0">
  <if_sid>100950</if_sid>
  <field name="governance.rule_type">^S2$</field>

  <description>
    GOVERNANCE: S2 deduplication rule event:
    $(governance.rule_id)
  </description>

  <group>
    governance,
    suppression_governance,
    S2,
    deduplication,
  </group>
</rule>


<!-- S3 Aggregation -->

<rule id="100967" level="0">
  <if_sid>100950</if_sid>
  <field name="governance.rule_type">^S3$</field>

  <description>
    GOVERNANCE: S3 aggregation rule event:
    $(governance.rule_id)
  </description>

  <group>
    governance,
    suppression_governance,
    S3,
    aggregation,
```

```
  </group>
</rule>


<!-- S4 Maintenance -->

<rule id="100968" level="0">
  <if_sid>100950</if_sid>
  <field name="governance.rule_type">^S4$</field>

  <description>
    GOVERNANCE: S4 maintenance rule event:
    $(governance.rule_id)
  </description>

  <group>
    governance,
    suppression_governance,
    S4,
    maintenance,
  </group>
</rule>


<!-- S5 Conditional -->

<rule id="100969" level="0">
  <if_sid>100950</if_sid>
  <field name="governance.rule_type">^S5$</field>

  <description>
    GOVERNANCE: S5 conditional rule event:
    $(governance.rule_id)
  </description>

  <group>
```

```
      governance,
      suppression_governance,
      S5,
      conditional_suppression,
    </group>
  </rule>


  <!-- S6 Feedback-promoted -->

  <rule id="100970" level="0">
    <if_sid>100950</if_sid>
    <field name="governance.rule_type">^S6$</field>

    <description>
      GOVERNANCE: S6 feedback-promoted rule event:
      $(governance.rule_id)
    </description>

    <group>
      governance,
      suppression_governance,
      S6,
      feedback_promoted,
    </group>
  </rule>


  <!-- ============================================================
       S6 PROMOTION GOVERNANCE
       ============================================================
-->

  <!--
    An S6 rule must originate from repeated analyst feedback.
    The governance automation validates this before emitting
```

```
    s6_promotion_approved.
  -->

  <rule id="100971" level="0">
    <if_sid>100950</if_sid>
    <field name="governance.action">^s6_promotion_approved$</field>

    <description>
      GOVERNANCE: S6 suppression candidate approved:
      $(governance.rule_id)
    </description>

    <group>
      governance,
      S6,
      feedback_promoted,
      rule_promotion,
    </group>
  </rule>


  <!-- S6 promotion without sufficient evidence -->

  <rule id="100972" level="9">
    <if_sid>100950</if_sid>
    <field
name="governance.action">^s6_evidence_insufficient$</field>

    <description>
      GOVERNANCE VIOLATION: S6 suppression promotion lacks required
      analyst evidence: $(governance.rule_id)
    </description>

    <group>
      governance,
      governance_violation,
```

```
      S6,
      insufficient_evidence,
    </group>
  </rule>


  <!-- ============================================================
       SUPPRESSION DRIFT
       ============================================================
-->

  <!--
    Governance automation calculates:
      current_count vs 30-day baseline
      standard deviation

    and emits:
      suppression_drift_warning
      suppression_drift_critical
  -->

  <rule id="100973" level="7">
    <if_sid>100950</if_sid>
    <field
name="governance.action">^suppression_drift_warning$</field>

    <description>
      GOVERNANCE WARNING: Suppression volume drift detected for
rule:
      $(governance.rule_id)
      Current=$(governance.current_count)
      Baseline=$(governance.baseline_count)
      Sigma=$(governance.drift_sigma)
    </description>

    <group>
```

```
      governance,
      suppression_governance,
      suppression_drift,
      drift_warning,
    </group>
  </rule>


  <!--
    Architecture threshold:
      >3 sigma from 30-day baseline

    This is the blind-spot signal described in the architecture.
  -->

  <rule id="100974" level="11">
    <if_sid>100950</if_sid>
    <field
name="governance.action">^suppression_drift_critical$</field>

    <description>
      CRITICAL GOVERNANCE: Suppression volume exceeded permitted
drift
      threshold for rule:
      $(governance.rule_id)
      Current=$(governance.current_count)
      Baseline=$(governance.baseline_count)
      Sigma=$(governance.drift_sigma)
    </description>

    <group>
      governance,
      critical_governance,
      suppression_drift,
      blind_spot_signal,
      suppression_anomaly,
```

```
    </group>
  </rule>


  <!-- ============================================================
       SUPPRESSION ESCAPE
       ============================================================
-->

  <!--
    A suppression escape means an event that had previously matched
    a suppression control was later associated with a confirmed or
    high-confidence security incident.
  -->

  <rule id="100975" level="11">
    <if_sid>100950</if_sid>
    <field name="governance.action">^suppression_escape$</field>

    <description>
      CRITICAL GOVERNANCE: Suppressed event later associated with
      security incident.
      SuppressionRule=$(governance.rule_id)
      Escapes=$(governance.escape_count)
    </description>

    <mitre>
      <id>T1562.001</id>
    </mitre>

    <group>
      governance,
      critical_governance,
      suppression_escape,
      control_failure,
      defense_evasion,
```

```
    </group>
  </rule>


  <!-- Multiple suppression escapes -->

  <rule id="100976" level="13">
    <if_sid>100950</if_sid>
    <field
name="governance.action">^suppression_escape_critical$</field>

    <description>
      CRITICAL CONTROL FAILURE: Suppression rule has repeated
security
      escapes and requires immediate disable/review:
      $(governance.rule_id)
    </description>

    <mitre>
      <id>T1562.001</id>
    </mitre>

    <group>
      governance,
      critical_governance,
      suppression_escape,
      kill_switch_candidate,
      control_failure,
    </group>
  </rule>


  <!-- ============================================================
       ENRICHMENT FRESHNESS
       ============================================================
-->
```

```
  <!-- Asset enrichment stale >24 hours -->

  <rule id="100977" level="7">
    <if_sid>100950</if_sid>
    <field
name="governance.action">^asset_enrichment_stale$</field>

    <description>
      GOVERNANCE WARNING: Asset enrichment exceeds freshness SLA:
      $(governance.enrichment_age_hours) hours
    </description>

    <group>
      governance,
      enrichment_governance,
      stale_enrichment,
      asset_context,
    </group>
  </rule>


  <!-- Identity enrichment stale >1 hour -->

  <rule id="100978" level="7">
    <if_sid>100950</if_sid>
    <field
name="governance.action">^identity_enrichment_stale$</field>

    <description>
      GOVERNANCE WARNING: Identity enrichment exceeds freshness
SLA:
      $(governance.enrichment_age_hours) hours
    </description>

    <group>
```

```
      governance,
      enrichment_governance,
      stale_enrichment,
      identity_context,
    </group>
  </rule>


  <!-- Threat intelligence stale >6 hours -->

  <rule id="100979" level="7">
    <if_sid>100950</if_sid>
    <field
name="governance.action">^threat_intel_enrichment_stale$</field>

    <description>
      GOVERNANCE WARNING: Threat-intelligence enrichment exceeds
      freshness SLA:
      $(governance.enrichment_age_hours) hours
    </description>

    <group>
      governance,
      enrichment_governance,
      stale_enrichment,
      threat_intelligence,
    </group>
  </rule>


  <!-- Vulnerability enrichment stale >72 hours -->

  <rule id="100980" level="7">
    <if_sid>100950</if_sid>
    <field
name="governance.action">^vulnerability_enrichment_stale$</field>
```

```
    <description>
      GOVERNANCE WARNING: Vulnerability enrichment exceeds
freshness
      SLA:
      $(governance.enrichment_age_hours) hours
    </description>

    <group>
      governance,
      enrichment_governance,
      stale_enrichment,
      vulnerability_context,
    </group>
  </rule>


  <!-- Generic enrichment failure -->

  <rule id="100981" level="9">
    <if_sid>100950</if_sid>
    <field name="governance.action">^enrichment_failure$</field>

    <description>
      GOVERNANCE VIOLATION: Required context enrichment failed:
      $(governance.enrichment_type)
    </description>

    <group>
      governance,
      enrichment_governance,
      enrichment_failure,
    </group>
  </rule>
```

```
  <!-- ============================================================
       ATT&CK COVERAGE REGRESSION
       ============================================================
-->

  <rule id="100982" level="9">
    <if_sid>100950</if_sid>
    <field
name="governance.action">^attack_coverage_regression$</field>

    <description>
      GOVERNANCE VIOLATION: ATT&CK coverage regression detected:
      $(governance.attck_technique)
      Rule=$(governance.rule_id)
    </description>

    <group>
      governance,
      coverage_governance,
      attack_coverage_regression,
      detection_regression,
    </group>
  </rule>


  <!-- Suppression spike on ATT&CK-mapped detection -->

  <rule id="100983" level="11">
    <if_sid>100950</if_sid>
    <field
name="governance.action">^attack_suppression_spike$</field>

    <description>
      CRITICAL GOVERNANCE: Suppression spike detected on
      ATT&CK-mapped detection:
      Technique=$(governance.attck_technique)
```

```
      Rule=$(governance.rule_id)
    </description>

    <group>
      governance,
      critical_governance,
      attack_coverage_regression,
      suppression_drift,
      detection_regression,
    </group>
  </rule>


  <!-- ============================================================
       CORRELATION RULE KILL-SWITCH
       ============================================================
-->

  <!--
    Emergency circuit breaker.
    The external automation performs the actual enable/disable
    operation. Wazuh provides the immutable audit trail and alerts.
  -->

  <rule id="100984" level="9">
    <if_sid>100950</if_sid>
    <field
name="governance.action">^correlation_kill_switch_enabled$</field>

    <description>
      GOVERNANCE: Correlation-rule kill-switch enabled:
      $(governance.correlation_rule)
    </description>

    <group>
      governance,
```

```
    kill_switch,
    correlation_control,
    emergency_change,
  </group>
</rule>


<!-- Kill-switch expiration -->

<rule id="100985" level="7">
  <if_sid>100950</if_sid>
  <field name="governance.action">^kill_switch_expiring$</field>

  <description>
    GOVERNANCE WARNING: Correlation kill-switch is approaching
    expiration:
    $(governance.correlation_rule)
  </description>

  <group>
    governance,
    kill_switch,
    correlation_control,
    expiry_warning,
  </group>
</rule>


<!-- Kill-switch expired -->

<rule id="100986" level="11">
  <if_sid>100950</if_sid>
  <field name="governance.action">^kill_switch_expired$</field>

  <description>
```

```
      CRITICAL GOVERNANCE: Correlation-rule kill-switch has
expired:
      $(governance.correlation_rule)
    </description>

    <group>
      governance,
      critical_governance,
      kill_switch,
      correlation_control,
      control_failure,
    </group>
  </rule>


  <!-- Unauthorized kill-switch operation -->

  <rule id="100987" level="13">
    <if_sid>100950</if_sid>
    <field
name="governance.action">^unauthorized_kill_switch$</field>

    <description>
      CRITICAL GOVERNANCE: Unauthorized correlation-rule
kill-switch
      operation detected:
      $(governance.correlation_rule)
      Actor=$(governance.actor)
    </description>

    <group>
      governance,
      critical_governance,
      kill_switch,
      unauthorized_change,
      control_failure,
```

```
    </group>
  </rule>


  <!-- ============================================================
       UNAUTHORIZED GOVERNANCE CHANGES
       ============================================================
-->

  <rule id="100988" level="11">
    <if_sid>100950</if_sid>
    <field
name="governance.action">^unauthorized_rule_change$</field>

    <description>
      CRITICAL GOVERNANCE: Unauthorized rule change detected:
      $(governance.rule_id)
      Actor=$(governance.actor)
    </description>

    <mitre>
      <id>T1562.001</id>
    </mitre>

    <group>
      governance,
      critical_governance,
      unauthorized_change,
      rule_tampering,
      defense_evasion,
    </group>
  </rule>


  <!-- Governance configuration tampering -->
```

```
  <rule id="100989" level="13">
    <if_sid>100950</if_sid>
    <field
name="governance.action">^governance_config_tampering$</field>

    <description>
      CRITICAL GOVERNANCE: Governance configuration tampering
detected.
      Actor=$(governance.actor)
    </description>

    <mitre>
      <id>T1562.001</id>
    </mitre>

    <group>
      governance,
      critical_governance,
      configuration_tampering,
      defense_evasion,
    </group>
  </rule>


  <!-- ============================================================
       EMERGENCY CHANGES
       ============================================================
-->

  <rule id="100990" level="9">
    <if_sid>100950</if_sid>
    <field
name="governance.action">^emergency_rule_disable$</field>

    <description>
      GOVERNANCE: Emergency rule disable performed:
```

```
      $(governance.rule_id)
      Change=$(governance.change_id)
    </description>

    <group>
      governance,
      emergency_change,
      rule_disabled,
    </group>
  </rule>


  <!-- Emergency change without ticket -->

  <rule id="100991" level="11">
    <if_sid>100950</if_sid>
    <field
name="governance.action">^emergency_change_missing_ticket$</field>

    <description>
      CRITICAL GOVERNANCE: Emergency rule change has no
change/ticket
      reference:
      $(governance.rule_id)
    </description>

    <group>
      governance,
      critical_governance,
      emergency_change,
      missing_ticket,
    </group>
  </rule>


  <!-- ============================================================
```

```
       RULE ROLLBACK
       ============================================================
-->

  <rule id="100992" level="7">
    <if_sid>100950</if_sid>
    <field name="governance.action">^rule_rollback$</field>

    <description>
      GOVERNANCE: Rule rollback performed:
      $(governance.rule_id)
    </description>

    <group>
      governance,
      rule_lifecycle,
      rollback,
    </group>
  </rule>


  <!-- Rollback due to suppression escape -->

  <rule id="100993" level="9">
    <if_sid>100950</if_sid>
    <field
name="governance.action">^rollback_due_to_escape$</field>

    <description>
      GOVERNANCE: Rule rollback triggered by suppression escape:
      $(governance.rule_id)
    </description>

    <group>
      governance,
      governance_violation,
```

```
      rollback,
      suppression_escape,
    </group>
  </rule>


  <!-- ============================================================
       REQUIRED GOVERNANCE METADATA
       ============================================================
-->

  <!-- Missing reason -->

  <rule id="100994" level="7">
    <if_sid>100950</if_sid>
    <field
name="governance.action">^required_metadata_missing$</field>

    <description>
      GOVERNANCE WARNING: Required governance metadata is missing:
      Rule=$(governance.rule_id)
    </description>

    <group>
      governance,
      governance_warning,
      metadata_validation,
    </group>
  </rule>


  <!-- Missing scope -->

  <rule id="100995" level="7">
    <if_sid>100950</if_sid>
    <field name="governance.action">^scope_missing$</field>
```

```
  <description>
    GOVERNANCE WARNING: Governed rule has no documented scope:
    $(governance.rule_id)
  </description>

  <group>
    governance,
    governance_warning,
    scope_validation,
  </group>
</rule>


<!-- Missing expiration -->

<rule id="100996" level="9">
  <if_sid>100950</if_sid>
  <field name="governance.action">^expiry_missing$</field>

  <description>
    GOVERNANCE VIOLATION: Governed exception has no expiration:
    $(governance.rule_id)
  </description>

  <group>
    governance,
    governance_violation,
    expiry_missing,
    rule_lifecycle,
  </group>
</rule>


<!-- ============================================================
     GOVERNANCE AUDIT COMPLETION
```

```
      ============================================================
-->

  <rule id="100997" level="0">
    <if_sid>100950</if_sid>
    <field
name="governance.action">^governance_review_completed$</field>

    <description>
      GOVERNANCE: Periodic governance review completed:
      $(governance.rule_id)
    </description>

    <group>
      governance,
      governance_audit,
      review_completed,
    </group>
  </rule>


  <!-- ============================================================
       GOVERNANCE CONTROL FAILURE
       ============================================================
-->

  <!--
    Generic high-confidence governance failure.
    This is intended for events produced by the governance engine
    after it has evaluated multiple control failures.
  -->

  <rule id="100998" level="11">
    <if_sid>100950</if_sid>
    <field
name="governance.action">^governance_control_failure$</field>
```

```
    <description>
      CRITICAL GOVERNANCE CONTROL FAILURE:
      Rule=$(governance.rule_id)
      Reason=$(governance.reason)
    </description>

    <group>
      governance,
      critical_governance,
      control_failure,
      incident_escalation,
    </group>
  </rule>


  <!-- ================================================================
       GOVERNANCE CRITICAL META-RULE
       ================================================================
-->

  <!--
    Used when the governance automation determines that multiple
    governance failures affect the same rule/control.

    Example:
      expired rule
      +
      suppression drift
      +
      suppression escape

    should become one critical governance incident rather than
    several independent alerts.
  -->
```

```
  <rule id="100999" level="13">
    <if_sid>100950</if_sid>
    <field name="governance.action">^governance_incident$</field>

    <description>
      CRITICAL GOVERNANCE INCIDENT:
      Rule=$(governance.rule_id)
      Reason=$(governance.reason)
    </description>

    <group>
      governance,
      critical_governance,
      governance_incident,
      incident_escalation,
      control_failure,
    </group>
  </rule>


</group>
```

Why this structure matches the attached architecture

The source document specifies that every suppression exception should have an **owner, reason, scope, expiry/review date and independent approval**, with unowned rules automatically expiring.

The file therefore explicitly covers:

Control	Rules
Rule proposal	100951
Approval	100952
Rejection	100953

Activation/disable	100954–100955
Self-approval	100956
Missing/invalid approval	100957–100958
Missing/invalid owner	100959–100960
Review due/overdue	100961–100962
Expiration	100963–100964
S1–S6 classification	100965–100970
S6 evidence governance	100971–100972
Suppression drift	100973–100974
Suppression escape	100975–100976
Enrichment SLA	100977–100981
ATT&CK regression	100982–100983
Kill-switch	100984–100987
Unauthorized changes	100988–100989
Emergency changes	100990–100991

Rollback	`100992–100993`
Required metadata	`100994–100996`
Governance review	`100997`
Control failure	`100998–100999`

This maps directly to the attached governance model: daily suppression-volume monitoring against a 30-day baseline, review-date enforcement, owner accountability, enrichment freshness SLAs, a correlation kill-switch, and ATT&CK coverage regression.

Important: don't try to make these calculations inside Wazuh XML

For example, the architecture says:

> suppression volume deviates **>3σ from its 30-day baseline**

and treats that as the blind-spot signal.

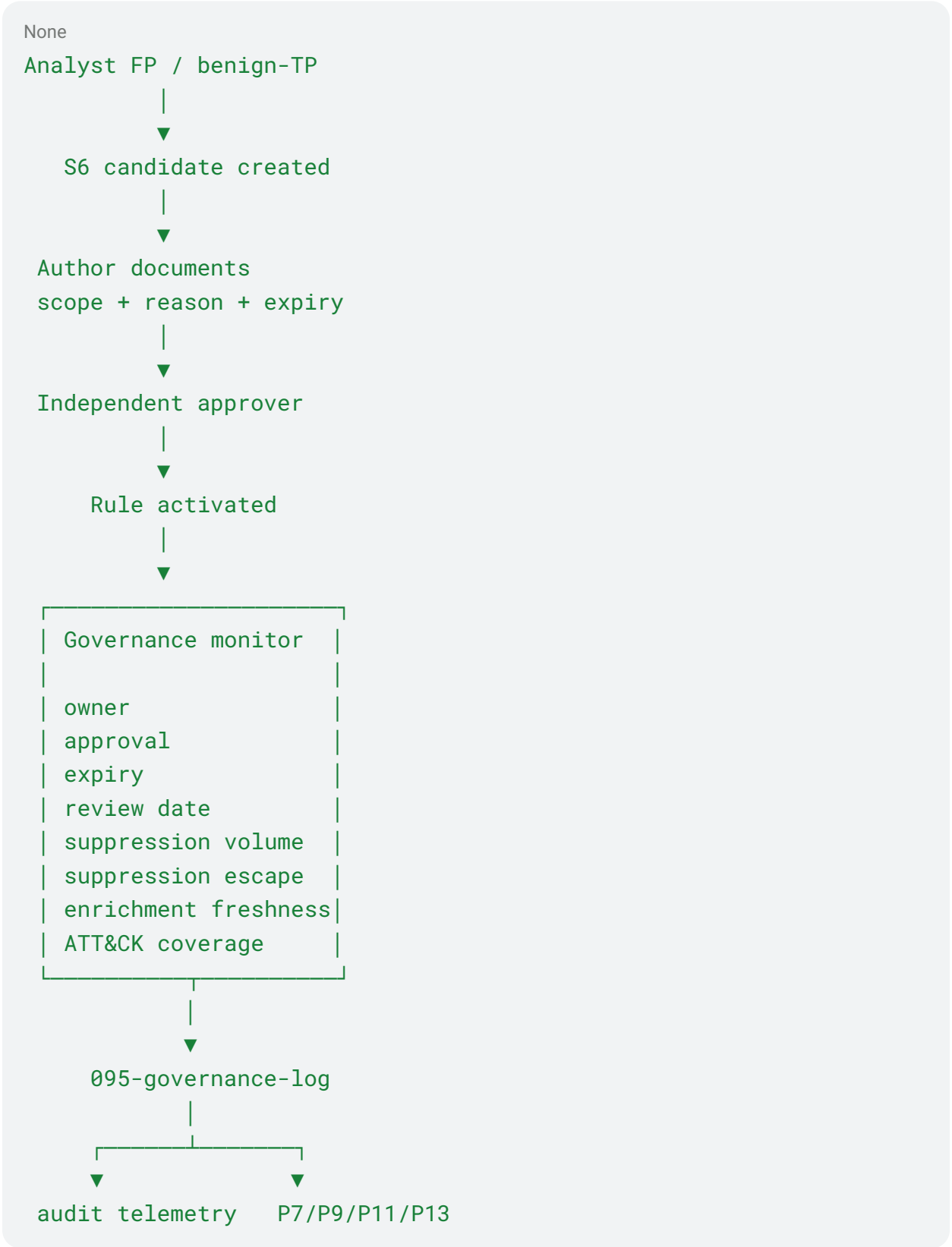
I would have an external governance job produce:

```json
{
  "governance": {
    "action": "suppression_drift_critical",
    "rule_id": "100840",
    "current_count": 18420,
    "baseline_count": 4200,
    "drift_sigma": 4.7
  }
}
```

Then `100974` turns that into the Wazuh governance alert.

Likewise, the external job should emit `review_overdue`, `rule_expired`, `owner_missing`, `asset_enrichment_stale`, etc. Wazuh is then the **audit/control enforcement point**, rather than pretending the XML rules engine is a governance database.

Recommended event flow



```
governance alerts
```

The attached design specifically requires that repeated FP/benign-TP patterns create an **S6 candidate rather than an immediately live suppression rule**, followed by author/approver review, named ownership, mandatory review date, and drift monitoring.

One final distinction: `095-governance-log.xml` should **not replace `080-suppression.xml` or `070-correlation.xml`**. The attached architecture deliberately separates suppression from correlation: `080` controls deterministic noise, `070` determines whether events constitute an incident, and `095` audits whether those controls themselves remain safe and governed.

Step-by-step Instructions

Stage 1, as defined in your architecture, focuses on the collection and initial detection phase. It serves as the foundation for the entire pipeline. To implement this stage effectively, you must establish reliable log ingestion and a scalable rule organization strategy.[1]

Here are the step-by-step instructions for implementing Stage 1:1. Establish Ingestion Infrastructure

Before detection can occur, logs must flow into the system.

- **Deploy Wazuh Agents:** Install the Wazuh agent on all target endpoints (Windows, Linux, Cloud instances) to ensure log collection.[1]
- **Configure Wazuh Manager:** Ensure the central Wazuh Manager is correctly configured to receive, parse, and process the telemetry generated by these agents.[1]

2. Implement Recommended Rule Organization

Avoid cluttering the system by organizing custom detection rules by function from the start. Create the following folder structure to keep your detection logic modular:[1]

- `custom/`
  - `001-base-normalization.xml`
  - `010-authentication.xml`
  - `020-process.xml`
  - `030-persistence.xml`
  - `040-network.xml`
  - `050-malware.xml`
  - `060-lateral-movement.xml`
  - `070-correlation.xml`
  - `080-suppression.xml`
  - `090-critical-assets.xml`

3. Categorize Detection Logic

Group your rules using consistent tags to improve visibility and maintenance. Use the following recommended rule groups:[1]

- `authentication`
- `process_execution`
- `privilege_escalation`
- `persistence`
- `lateral_movement`
- `malware`
- `command_and_control`
- `suppression`
- `correlation`

- `critical_asset`

4. Author Custom Detection Rules

Augment the built-in Wazuh rule set with custom rules specifically tailored to your environment.

- **Do not disable built-in rules:** If a built-in rule generates noise, do not disable it at this stage. Instead, allow it to pass through to Stage 1 and handle the noise later using the suppression/deduplication stages.[1]
- **Focus on detection:** Write rules that identify suspicious behaviors (e.g., PowerShell execution, anomalous network traffic) rather than focusing on suppression logic within the rule file itself.[1]

5. Validate the Pipeline

Once your rules are deployed, verify that logs are flowing correctly and hitting the expected rules. You should verify that:

- Events are being categorized into the appropriate rule groups.
- The raw events are being captured and normalized by the Wazuh Manager before moving to the next stage of the architecture.[1]

To implement Stage 2: Noise Reduction effectively, the focus must shift from simply "stopping" alerts to intelligent filtering that preserves the underlying forensic evidence.

Here are the step-by-step instructions for implementing Stage 2:1. Establish the "Suppress Events, Not Evidence" Principle

Before configuring suppression, ensure your logging backend retains the raw events. Stage 2 should filter what the SOC *sees* in the dashboard, but you must ensure the full telemetry remains searchable in your data lake for incident reconstruction.2. Implement S1 — Known-Good Allowlisting[1]

Avoid blanket suppressions based on IP addresses, as these can hide malicious activity from compromised internal assets. Use multi-factor suppression keys instead.[1]

- **Define Suppression Keys:** Create suppression rules using a combination of: `source` + `destination` + `executable` + `account` + `event_type`.
- **Actionable Logic:** Use the following template for your suppression rules:

```
None
IF
    source.ip = approved_scanner
AND event.category = vulnerability_scan
THEN
    suppress (alert in dashboard)
    retain (keep raw telemetry)

  •     increment (counter for tracking)
```

-

3. Configure Deduplication (S2)

Collapse repetitive, high-volume noise (like thousands of failed logins) into single, actionable objects.

- **Define the Window:** Determine an appropriate time window (e.g., 5 minutes) to group events.
- **Standardize the Deduplication Key:** Use a hash of critical fields to identify "identical" noise:

```
None
  • KEY = hash(rule_id, source_ip, destination_ip, username,
    event_type)
```

-
- **Operational Workflow:**
    - **First event:** Create a standard alert.

- **Subsequent events (within window):** Aggregate into the existing alert.
- **Threshold Exceeded:** Update the severity if the volume hits a specific limit (e.g., > 100 attempts).

4. Implement Rate Limiting

Prevent alert storms from overwhelming the SOC by applying rate limits to noisy rules that don't qualify for full suppression.

- **Establish Tiered Escalation:**
  - **0–5 events / 5 min:** Aggregate into a single summary.
  - **6–20 events / 5 min:** Flag as P3 (Low).
  - **21–100 events / 5 min:** Flag as P2 (Medium).
  - **>100 events / 5 min:** Flag as P1 (High).[1]

5. Enforce Suppression Governance

Never apply permanent "ignore" rules. Every suppression exception must follow a lifecycle to prevent "suppression creep" where real attacks are hidden by outdated rules.[1]

- **Mandatory Metadata:** For every exception, document:
  - `exception_id`
  - `reason` (e.g., "approved vulnerability scanner")
  - `owner` (Security Engineering/Team Name)
  - `created_date`
  - `expiry_date` (e.g., 30–90 days)
  - `review_cycle` (e.g., monthly/quarterly)

6. Validate Against Baseline

After implementing these rules, monitor your **Alert Reduction %**. If your suppression escape rate (suppressed events that are later found to be part of an incident) increases, revert the suppression rules and refine the "Known-Good" criteria.[1]

Here are the step-by-step instructions for implementing **Stage 3: Context Enrichment**. This stage transforms raw alerts into actionable intelligence by augmenting events with data from external sources.1. Integrate Identity Context

Map anonymous IP addresses or machine names to specific user identities to speed up investigation.

- **Identify Sources:** Connect your SIEM/Log Manager to your Directory Services (e.g., Active Directory, LDAP, or Google Workspace).
- **Map Attributes:** Ensure every alert includes the following fields via lookup tables or real-time API calls:
    - `user_id` (username)
    - `user_department`
    - `user_role`
    - `risk_score` (if available from IAM tools)
- **Automation:** Configure the ingestion pipeline to append this identity metadata before the alert reaches the dashboard.

2. Augment with Asset Criticality

Prioritize alerts based on the importance of the affected asset.

- **Classify Assets:** Assign a "Criticality Score" (e.g., 1–5) to your infrastructure:
    - **Level 5 (Crown Jewels):** Domain Controllers, Payment Gateways, Production Databases.
    - **Level 1 (Low):** Testing environments, non-persistent workstations.
- **Dynamic Tagging:** As logs are ingested, tag the asset (e.g., `asset_criticality=5`).
- **Alert Routing:** Use this tag to adjust the P-level of alerts (e.g., automatically escalate any alert on a Level 5 asset to P1).

3. Incorporate Threat Intelligence

Correlate incoming telemetry with known Indicators of Compromise (IoCs).

- **Select Feeds:** Integrate reliable open-source or commercial threat intelligence feeds (e.g., AlienVault OTX, CrowdStrike, or MISP).
- **Configure Lookup:** Implement a real-time lookup mechanism during ingestion:
    - Match `source_ip` or `destination_ip` against known malicious IP lists.
    - Match file hashes against known malware databases.
- **Automated Enrichment:** Append a `threat_intel` field to the alert if a match is found, including:
    - `threat_actor` (if identified)
    - `confidence_score`
    - `references` (links to analysis)

4. Normalize and Standardize

Ensure enriched data is in a consistent format for your SOC team.

- **Common Schema:** Adopt a standard schema (e.g., ECS - Elastic Common Schema) so that enrichment data is always in the same field location, regardless of the source.
- **Validation:** Verify that enrichment happens *during* the ingestion phase, not during the dashboard view, to ensure search performance remains high.

5. Validate Enrichment Accuracy

- **Audit Samples:** Periodically check enriched alerts to ensure user IDs are correct and threat intel matches are relevant.
- **Refine Thresholds:** If "Threat Intel" matches are generating too many false positives, adjust the confidence score threshold required to trigger an alert escalation.

To implement Stage 4: Incident Response & Orchestration (SOAR), the focus shifts from identifying and enriching alerts to automating the reaction. This reduces Mean Time to Respond (MTTR) by offloading repetitive containment tasks to the system.

Here are the step-by-step instructions for implementing Stage 4:1. Define Playbooks for Common Threats

Don't automate everything at once. Start by codifying the "manual" steps your SOC team performs for high-frequency incidents into automated playbooks.

- **Identify Use Cases:** Target high-volume, low-complexity alerts (e.g., brute force attacks, known malware hash detection, phishing links).
- **Create Structured Workflows:** For each use case, map the logic:
    - **Decision:** Is the threat verified?
    - **Containment:** Isolate the host/Disable the account.
    - **Notification:** Alert the incident responder via Slack/Teams/Email.
    - **Recovery:** Revert changes or reset credentials.

2. Establish "Human-in-the-Loop" (HITL) Gateways

For high-impact actions (e.g., locking an executive's account or shutting down a production server), never automate the action blindly.

- **Implement Approval Prompts:** Configure your SOAR platform to send an "Approve/Reject" notification to the SOC lead before executing destructive actions.
- **Set Timeouts:** If the SOC lead doesn't respond to the prompt within X minutes (e.g., 5 mins), default to a "Safe" state (e.g., log the attempt but do not isolate).

3. Integrate SOAR with Endpoint/Network Tools

Enable your SIEM/Wazuh manager to talk to your enforcement points.

- **API Integration:** Use secure API keys to connect the orchestration platform to your infrastructure:
    - **EDR (e.g., Wazuh, CrowdStrike):** For host isolation.
    - **Identity Provider (e.g., Okta, Active Directory):** For account suspension or password reset.
    - **Firewall/Cloud Security Group (e.g., AWS Security Groups, Palo Alto):** For blocking malicious IPs.
- **Credential Management:** Use a vault (e.g., HashiCorp Vault) to manage credentials; never hardcode API keys in your playbooks.

4. Implement Adaptive Response

Move beyond static responses. Use the "Context Enrichment" from Stage 3 to determine the severity of the automated response.

- **Risk-Based Action:**
    - *Low Risk (e.g., Test VM):* Auto-isolate the host.
    - *High Risk (e.g., Database Server):* Alert the human admin for manual review, but initiate "Audit Mode" (log everything the process does) automatically.

5. Audit and Feedback Loop

Automation can break things if not monitored.

- **Track "Success" vs. "False Positive" Actions:** Create a dashboard that measures how many automated actions were later marked as false positives by analysts.
- **Version Control:** Store your playbooks in a Git repository. Every change to a playbook must be reviewed, tested in a staging environment, and committed just like code.

6. Simulation (Game Days)

You won't know if your orchestration works until you test it.

- **Execute Drills:** Periodically trigger a "fake" incident (e.g., simulate a brute force attack from a non-production machine) to ensure the SOAR playbook executes, the ticket is created, and the alerts fire correctly.

To complete your security operations architecture, **Stage 5: Proactive Hunting & Continuous Improvement** marks the shift from reactive defense (responding to alerts) to proactive defense (finding threats that bypass your current rules).

Here are the step-by-step instructions for implementing Stage 5:1. Establish a Hypothesis-Based Threat Hunting Program

Stop waiting for alerts to fire. Use the "assume breach" mindset to look for signs of attacker activity that automated rules might miss.

- **Generate Hypotheses:** Start with frameworks like MITRE ATT&CK. (e.g., "Hypothesis: An attacker is using hidden scheduled tasks for persistence on our domain controllers.")
- **Data Scoping:** Identify which log sources you need to search (e.g., Process creation logs, Windows Event IDs).
- **Execute the Hunt:** Manually query your data lake/Wazuh archives to look for deviations from the established baseline, not just known signatures.

2. Institutionalize Post-Incident Reviews (PIRs)

Every significant alert or incident is a learning opportunity. Treat them as data points to improve your infrastructure.

- **Conduct PIRs:** After every P1/P2 incident, hold a meeting to review:
    - **Root Cause:** How did the attacker get in?
    - **Detection Gap:** Why didn't the existing rules catch it sooner?
    - **Response Efficiency:** Did the SOAR playbooks (Stage 4) work as expected?
- **Document Outcomes:** Update your runbooks or internal wikis based on findings.

3. Implement Purple Teaming Cycles

Collaboration between your "Red Team" (attack simulation) and "Blue Team" (detection/response) is crucial to validate your current defenses.

- **Simulate Attacks:** Have the Red Team simulate specific attacker tactics (e.g., credential dumping, lateral movement).
- **Detection Validation:** Check if Stage 1 (Detection) caught it, Stage 3 (Enrichment) added the right context, and Stage 4 (Automation) triggered correctly.
- **Gap Closure:** If an attack is missed, build a new custom rule (Stage 1) or playbook (Stage 4) immediately.

4. Track Maturity via Key Metrics (KPIs)

Move from anecdotal success to data-driven security operations. Monitor these metrics to show ROI and identify areas for improvement:

- **Mean Time to Detect (MTTD):** How long does it take from the first malicious event to an alert being fired?
- **Mean Time to Respond (MTTR):** How long from alert firing to incident resolution?
- **False Positive Ratio:** What percentage of alerts are noise? (Your goal is a downward trend).
- **Coverage:** Percentage of MITRE ATT&CK techniques currently covered by your detection rules.

5. Close the Loop on Detection Logic

Your detection rules are living documents.

- **Seasonal Review:** Every quarter, review your rule folders (created in Stage 1). Delete rules that are no longer relevant, tune rules that create excessive noise, and update rules based on new threat intel.
- **Retire Legacy Content:** Regularly archive or delete outdated rules to keep the Wazuh Manager performant.

Implementation Phases

Phase-by-phase implementation plan

Architecture: Wazuh-centered alert-fatigue reduction architecture.

Goal: incremental implementation to avoid accidentally creating blind spots.

Key Principles:

To tune in layers: first measure, then normalize, suppress only deterministic benign activity, aggregate repetition, enrich context, correlate behavior, and finally prioritize incidents.

Benefits: This approach reduces analyst-visible volume without simply disabling detection coverage.

Phase 0 — Establish the control model

Before changing rules, define how alerts will be owned and governed.

0.1 Define the architecture components

Your logical pipeline should be:

```
None
Telemetry
    ↓
Wazuh decoding and detection
    ↓
Normalization
    ↓
Noise classification
    ↓
Suppression / deduplication
    ↓
Aggregation
    ↓
Enrichment
    ↓
```

```
Correlation
   ↓
Risk scoring
   ↓
Incident prioritization
   ↓
SOC response
```

Do not combine all of these functions into Wazuh XML rules. Wazuh should primarily perform:

- log collection
- decoding
- first-level detection
- basic frequency/timeframe aggregation
- event classification

A downstream layer can handle more complex:

- cross-host correlation
- enrichment
- incident grouping
- dynamic risk scoring
- case management

0.2 Define ownership

Create a tuning ownership matrix.

Component	Owner	Approval required
Wazuh detection rule	Detection engineering	Security lead
Suppression rule	Detection engineering	SOC lead
Critical suppression	Detection engineering	Security manager
Allowlist	Asset/application owner	Security
Correlation rule	Detection engineering	SOC
Risk scoring	SOC + Security engineering	Security manager

Expired exception	Automatic review	Rule owner

Every suppression should have:

```
None
suppression_id:
owner:
business_reason:
created_date:
expiration_date:
affected_rules:
affected_assets:
conditions:
risk_accepted_by:
review_frequency:
```

Important rule

No suppression without an owner and expiration date.

Phase 1 — Measure the current alert baseline

Do not start suppressing alerts immediately.

First, determine what is generating noise.

1.1 Collect at least 2–4 weeks of data

Extract these fields from Wazuh alerts:

```
None
timestamp
rule.id
```

```
rule.level
rule.description
rule.groups
agent.id
agent.name
source.ip
destination.ip
user
process.name
process.parent
event.category
```

If some fields are unavailable, normalize them later. Do not block the project waiting for perfect schemas.

1.2 Build an alert inventory

Group alerts by:

```
None
rule.id
agent
asset group
event category
source
user
hour/day
```

Create a table like:

Rule ID	Alert type	Daily volume	Unique assets	Analyst action	TP/FP
5710	SSH authentication	50,000	200	Rarely	Mostly noise
Custom-1001	PowerShell suspicious	120	40	Sometimes	Mixed

Malware rule	Malware detection	3	3	Always	High value

1.3 Calculate the noise score

A useful prioritization model is:

```
None
Noise Priority =
Alert Volume
× False Positive Rate
× Analyst Handling Cost
```

For example:

```
None
Rule A:
50,000 alerts/day
95% false positive
high analyst workload

→ tune first
```

Do **not** start with the highest-severity rule. Start with the rules creating the largest operational burden.

1.4 Deliverable

At the end of Phase 1, you should have:

```
None
Top 20 noisy rules
Top 20 high-value rules
Top 20 repetitive event patterns
Baseline alerts/day
Baseline alerts/analyst/hour
Baseline true-positive rate
```

Freeze this baseline. You will need it to prove that tuning improved the system.

Phase 2 — Normalize and classify alerts

The goal is to ensure different alerts can be compared and correlated.

2.1 Create a common event model

Define normalized fields such as:

```json
{
  "event_id": "",
  "timestamp": "",
  "source": {
    "ip": "",
    "host": ""
  },
  "destination": {
    "ip": "",
    "host": ""
  },
  "user": {
    "name": "",
    "id": ""
  },
  "host": {
    "name": "",
    "criticality": ""
  },
  "process": {
    "name": "",
    "path": "",
    "command_line": "",
    "parent": ""
```

```
  },
  "detection": {
    "rule_id": "",
    "severity": "",
    "confidence": ""
  }
}
```

You do not need every field for every event.

The objective is consistency.

2.2 Map Wazuh alerts into categories

Create a taxonomy:

```
None
AUTHENTICATION
PROCESS_EXECUTION
PRIVILEGE_ESCALATION
PERSISTENCE
DEFENSE_EVASION
MALWARE
NETWORK_ANOMALY
LATERAL_MOVEMENT
DATA_ACCESS
CONFIGURATION_CHANGE
VULNERABILITY
POLICY_VIOLATION
```

Then assign each Wazuh rule to one primary category.

Example:

```
None
Rule 5710
```

```
↓
AUTHENTICATION
↓
FAILED_LOGIN
↓
LOW_CONFIDENCE
```

2.3 Add MITRE context where useful

Example:

```
None
PowerShell execution
    ↓
Execution
    ↓
T1059.001
```

Do not use MITRE mapping purely for documentation. It becomes useful later when correlating events into attack chains.

2.4 Deliverable

A maintained mapping file:

```
None
rule_id: "100123"
category: "PROCESS_EXECUTION"
subcategory: "SCRIPT_EXECUTION"
mitre_tactic: "Execution"
mitre_technique: "T1059"
default_confidence: "MEDIUM"
```

Phase 3 — Implement suppression and known-good exceptions

This phase should reduce deterministic noise.

3.1 Classify every suppression

Use one of these categories:

```
None
KNOWN_GOOD
MAINTENANCE
EXPECTED_ADMIN_ACTIVITY
DUPLICATE
LOW_VALUE
TEMPORARY_EXCEPTION
```

Never create a suppression with the description:

```
None
"Too noisy"
```

That is not a valid security reason.

Instead:

```
None
"Approved vulnerability scanner generates repeated authenticated
login events against production Linux servers."
```

3.2 Implement precise suppression

Avoid this

```xml
<rule id="100200" level="0">
    <if_sid>5710</if_sid>
</rule>
```

This may suppress an entire detection class.

Prefer conditional suppression

Conceptually:

```
IF

rule_id = SSH_AUTH_FAILURE

AND

source.ip IN APPROVED_SCANNER_IPS

AND

destination.host IN APPROVED_SCAN_TARGETS

AND

user = approved_scan_account

THEN

level = 0
group = suppressed
```

The narrower the condition, the safer the suppression.

3.3 Create an allowlist source of truth

Do not hardcode all IP addresses into many XML files.

Maintain logical lists:

```
None
allowlists/
├── vulnerability_scanners
├── monitoring_systems
├── backup_systems
├── approved_admin_hosts
├── service_accounts
└── maintenance_windows
```

Example:

```
None
approved_admin_hosts:
  - admin-jump-01
  - admin-jump-02

vulnerability_scanners:
  - 10.10.50.20
  - 10.10.50.21
```

3.4 Add expiration controls

Every temporary suppression should be:

```
None
Created → Active → Review → Renew/Remove → Expired
```

Example:

```
None
id: SUP-2026-014
reason: Application migration
owner: Infrastructure Team
expires: 2026-09-30
```

```
rules:
  - 100500
assets:
  - app-server-01
  - app-server-02
```

3.5 Test before production

For each suppression:

1. Generate a matching benign event.
2. Confirm it is suppressed or downgraded.
3. Generate a similar malicious/unexpected event.
4. Confirm it still alerts.
5. Record the test result.

This is critical.

Phase 4 — Implement deduplication and aggregation

This phase converts repetitive alerts into summaries.

4.1 Define aggregation keys

For authentication events:

```
None
rule_id
source.ip
destination.host
user
```

For process execution:

```
None
rule_id
agent.id
process.name
process.command_line
```

For file integrity events:

```
None
rule_id
agent.id
file.path
```

Avoid using only `rule_id`, because unrelated events would be merged.

4.2 Define aggregation windows

Recommended starting points:

Event type	Window
Failed login	5–10 minutes
Network scanning	5 minutes
File modification burst	10 minutes
Process execution	5 minutes
Configuration changes	30 minutes

These are starting values. Tune based on actual event behavior.

4.3 Use Wazuh frequency/timeframe where appropriate

For repeated events, use frequency-based logic.

Conceptually:

```xml
<rule id="100301" level="8" frequency="10" timeframe="300">
    <if_matched_sid>100300</if_matched_sid>
    <description>Repeated authentication failures
detected</description>
    <group>authentication,aggregation,</group>
</rule>
```

The base event can remain low priority while repeated activity is escalated.

4.4 Implement progressive escalation

Example:

```
1–5 failures
    → P4 / aggregate only

6–20 failures
    → P3

21–100 failures
    → P2

>100 failures
    → P1

failure + successful login
    → P1 regardless of count
```

This prevents every failed login from becoming an analyst task.

4.5 Output format

Your aggregated alert should contain:

```json
{
  "incident_type": "BRUTE_FORCE",
  "first_seen": "",
  "last_seen": "",
  "event_count": 2500,
  "source_ip": "",
  "target_host": "",
  "target_users": [],
  "correlation_id": ""
}
```

The analyst should see the summary first and drill into raw events only when necessary.

Phase 5 — Implement enrichment

Raw alerts often lack enough context to prioritize correctly.

5.1 Create an asset inventory

At minimum:

```
asset_name
asset_id
environment
owner
business_unit
criticality
internet_exposed
domain_controller
production
```

Example:

```
None
host: dc-01
criticality: CROWN_JEWEL
environment: PRODUCTION
internet_exposed: false
```

5.2 Define asset criticality

Use a small scale:

```
None
CROWN_JEWEL = 25
CRITICAL    = 20
HIGH        = 15
STANDARD    = 5
LAB         = 0
```

Avoid overly complex scales.

5.3 Add identity context

For each user where possible:

```
None
username
account_type
privileged
service_account
department
normal_host
normal_activity
```

Example:

```
None
user: svc_backup
```

```
type: SERVICE_ACCOUNT
expected_hosts:
    backup-01
    backup-02
```

A service account logging into a domain controller may deserve a different priority than logging into its normal backup server.

5.4 Add vulnerability context

If an alert targets or occurs on a vulnerable asset:

```
None
Suspicious process
+
Critical vulnerability
+
Internet-exposed host
```

increase risk.

Do not automatically assume:

```
None
CVE exists = active exploitation
```

Use vulnerability data as context, not proof.

5.5 Add threat-intelligence context

Enrich:

```
None
source.ip
destination.ip
domain
```

```
file.hash
URL
```

Possible outcomes:

```
None
Known malicious
Suspicious
Unknown
Known benign
```

Threat intelligence should add confidence, not replace behavioral detection.

Phase 6 — Implement correlation rules

This is the core of the architecture.

Start with only **5–10 high-confidence correlation rules**.

Do not try to model every ATT&CK technique immediately.

Correlation 1 — Brute force followed by success

Detection logic

```
None
FAILED_AUTH
    × 10 or more
    within 10 minutes

AND

SUCCESSFUL_AUTH
```

```
    same source
    same user or target
    within 5 minutes

→ AUTH_COMPROMISE_SUSPECTED
```

Implementation steps

1. Identify failed authentication rule IDs.
2. Identify successful authentication rule IDs.
3. Normalize:
    - source IP
    - target host
    - username
4. Generate an aggregation key.
5. Store the first failure timestamp.
6. Count failures.
7. Detect successful login.
8. Search for matching activity within the correlation window.
9. Create one correlated incident.

Output:

```
None
Incident: Suspected account compromise
Priority: P1
Evidence:
- 56 failed logins
- successful login
- same source IP
- 7 minute correlation window
```

Correlation 2 — Login → privilege escalation → persistence

Logic:

```
None
SUCCESSFUL_LOGIN
        ↓
PRIVILEGE_ESCALATION
        ↓
PERSISTENCE
```

Window:

```
None
15–30 minutes
```

Correlation key:

```
None
host + user + session/process lineage
```

Output:

```
None
Incident: Privilege escalation with persistence
Priority: P1
```

Correlation 3 — Suspicious PowerShell chain

Base event:

```
None
PowerShell execution
```

Add evidence:

```
None
encoded command
```

```
download activity
suspicious parent process
credential access
network connection
persistence
```

Scoring:

```
None
PowerShell                    +20
Encoded command               +20
Download                      +15
Suspicious child process      +15
External network              +15
Persistence                   +25
```

Example:

```
None
Score = 95
→ P0
```

A normal administrative PowerShell command may remain P3 or be conditionally suppressed.

Correlation 4 — Lateral movement

Look for:

```
None
Host A
   ↓
remote authentication
   ↓
Host B
```

```
   ↓
remote service/process execution
   ↓
privileged activity
```

Correlation keys:

```
None
source_host
destination_host
username
time_window
```

This is particularly useful for:

- RDP
- SSH
- SMB
- WinRM
- remote service execution

Correlation 5 — Malware execution with follow-on behavior

Logic:

```
None
MALWARE_DETECTION

+

PROCESS_EXECUTION

+
```

```
NETWORK_CONNECTION
```

Additional evidence:

```
None
persistence
credential access
file encryption
security tool tampering
```

Priority can escalate:

```
None
Malware alert only
→ P1

Malware + execution
→ P0

Malware + execution + persistence/C2
→ P0 with immediate containment workflow
```

Phase 7 — Implement the risk-scoring engine

Keep Wazuh severity and SOC priority separate.

7.1 Recommended scoring components

```
None
Detection severity
+
```

```
Detection confidence
+
Asset criticality
+
Identity privilege
+
Threat intelligence
+
Correlation evidence
+
Behavioral anomaly
-
Known-good confidence
```

Example scale:

Factor	Range
Detection severity	0–25
Confidence	0–20
Asset criticality	0–25
Identity risk	0–10
Threat intelligence	0–10
Correlation	0–20
Known-good adjustment	0 to -30

Normalize the final result to 0–100.

7.2 Priority mapping

```
None
81–100 → P0 Critical
61–80  → P1 High
41–60  → P2 Medium
```

```
21–40  → P3 Low
0–20   → P4 Informational
```

7.3 Example calculation

```
None
Detection: suspicious PowerShell       20
Confidence: encoded command            15
Asset: domain controller               25
User: privileged                       10
Correlation: outbound connection       15
Threat intel: unknown                   0

TOTAL = 85

SOC PRIORITY = P0
```

The exact numbers should be calibrated using historical incidents.

Phase 8 — Build the SOC incident workflow

The analyst should not receive raw Wazuh events as the primary work item.

The work item should be a **correlated incident**.

8.1 Incident structure

```
JSON
{
  "incident_id": "",
  "priority": "P1",
```

```
  "title": "Suspected account compromise",
  "first_seen": "",
  "last_seen": "",
  "affected_assets": [],
  "affected_users": [],
  "attack_stage": [],
  "risk_score": 72,
  "event_count": 1532,
  "evidence": [],
  "recommended_action": ""
}
```

8.2 Analyst workflow

```
None
New incident
      ↓
Auto-enriched
      ↓
Deduplicated
      ↓
Correlated
      ↓
Risk-scored
      ↓
Assigned to analyst
      ↓
True Positive / False Positive / Benign
      ↓
Feedback into tuning
```

The final disposition should feed back into the rule engineering process.

Phase 9 — Establish the continuous tuning loop

Alert fatigue will return if the environment changes and rules are not reviewed.

Use:

```
None
Alert
   ↓
Analyst disposition
   ↓
False-positive analysis
   ↓
Root cause classification
   ↓
Rule tuning
   ↓
Test environment
   ↓
Production deployment
   ↓
Measure result
```

Weekly review

Review:

- top noisy rules
- new alert sources
- false positives
- suppressed event volume
- aggregation ratios
- newly expired exceptions

Monthly review

Review:

- correlation effectiveness
- P0/P1 true-positive rates
- asset criticality
- allowlists
- suppression rules
- analyst workload

Quarterly review

Review:

- risk-scoring weights
- detection coverage
- MITRE correlation coverage
- obsolete rules
- unused suppression exceptions

Recommended implementation sequence

I would deploy this in the following order:

Week 1–2: Baseline

```
None
Inventory alerts
→ Identify top noise sources
→ Measure analyst workload
→ Classify true/false positives
```

Week 3–4: Suppression

```
None
Implement known-good exceptions
→ Add expiration dates
→ Test suppression boundaries
```

Week 5: Deduplication

```
None
Define aggregation keys
→ Define time windows
→ Implement progressive escalation
```

Week 6–7: Enrichment

```
None
Asset criticality
→ Identity context
→ Vulnerability context
→ Threat intelligence
```

Week 8–10: Correlation

Start with:

```
None
Brute force → success
Login → privilege escalation
PowerShell → suspicious follow-on activity
Remote authentication → lateral movement
Malware → execution → network activity
```

Week 11: Risk scoring

```
None
Define scoring weights
→ Map to P0–P4
→ Validate against historical incidents
```

Week 12: SOC validation

Run the new pipeline in **parallel** with the existing alert workflow.

Compare:

```
None
Old workflow:
10,000 alerts/day

New workflow:
1,200 analyst-visible alerts
150 correlated incidents
```

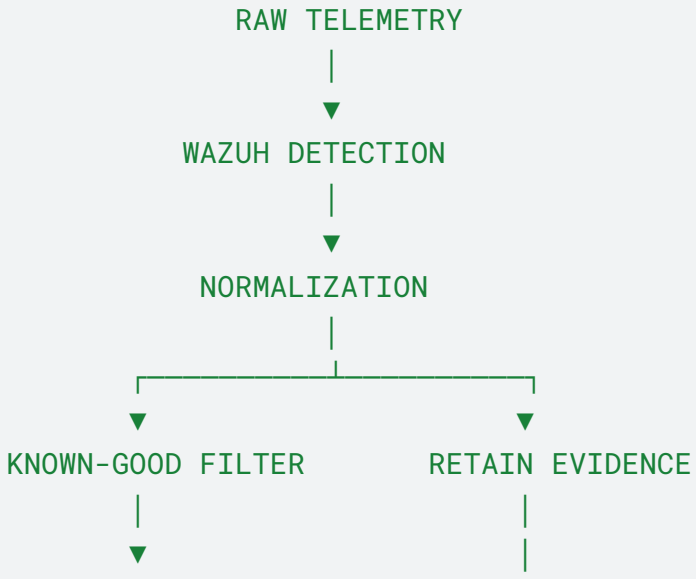
But do not judge success only by alert reduction. Validate:

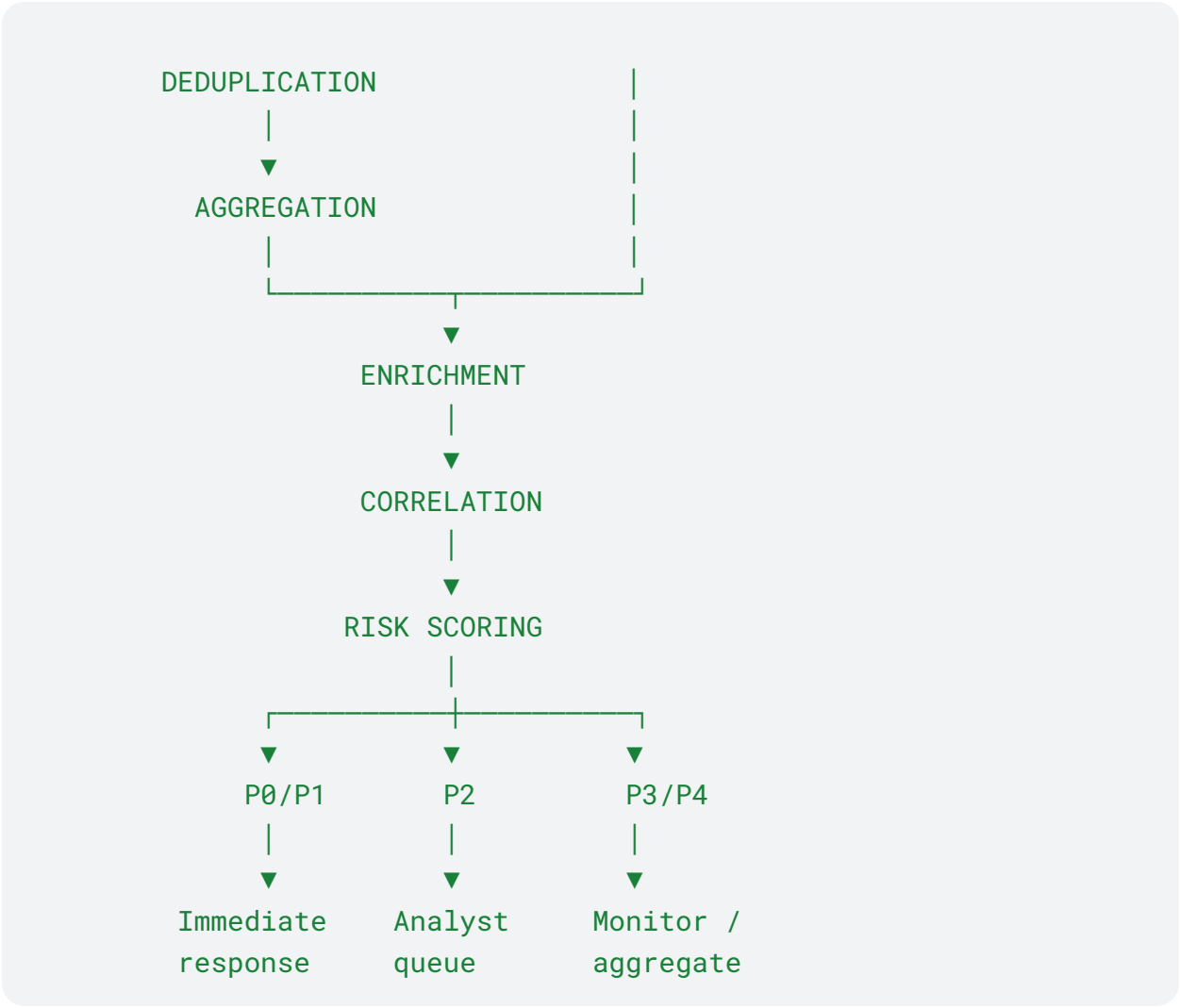
```
None
True-positive retention
False-negative increase
Mean time to triage
Analyst workload
Suppression escape rate
```

Final target state

Your mature Wazuh alert pipeline should look like this:

```
None
```




DEDUPLICATION
AGGREGATION
ENRICHMENT
CORRELATION
RISK SCORING
P0/P1
P2
P3/P4
Immediate response
Analyst queue
Monitor / aggregate

DB design

To support the end-to-end Wazuh alert-fatigue reduction, multi-stage detection pipeline, correlation, risk scoring, and governance architecture described in your document, here is a comprehensive relational database schema.-----**Database Schema Overview**

The schema is divided into 7 core functional domains:

1. **Raw Telemetry & Normalized Events** (Preserving evidence)
2. **Suppression & Allowlist Governance** (Dynamic exceptions with expiration and ownership)
3. **Aggregated & Deduplicated Alerts** (Collapsed noise objects)
4. **Correlated Incidents & Attack Chains** (Multi-event behavioral sequences)
5. **Risk Scoring & Severity Engine** (Dynamic P0–P4 calculation)
6. **SOC Incidents & Analyst Workflow** (Actionable items for analysts)
7. **System Metrics & KPIs** (Operational tuning and tracking)

-----**1. Raw Telemetry & Normalized Events**

Stores the original event telemetry and its normalized classification.

```
CREATE TABLE raw_events (
    event_id VARCHAR(64) PRIMARY KEY,
    agent_id VARCHAR(64) NOT NULL,
    agent_name VARCHAR(128),
    timestamp TIMESTAMP WITH TIME ZONE NOT NULL,
    source_ip VARCHAR(45),
    destination_ip VARCHAR(45),
    source_port INT,
    destination_port INT,
    protocol VARCHAR(16),
    username VARCHAR(128),
    raw_payload TEXT NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE normalized_events (
    normalized_event_id VARCHAR(64) PRIMARY KEY,
    raw_event_id VARCHAR(64) REFERENCES raw_events(event_id),
    rule_id INT NOT NULL,
    rule_level INT NOT NULL,
    classification_groups TEXT[], -- e.g., {base_normalized, authentication, sshd}
    mitre_attack_ids VARCHAR(16)[],
    description TEXT,
    processed_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

-----**2. Suppression & Allowlist Governance**

Implements governable exceptions with ownership and expiration (avoiding permanent bypasses).

```
CREATE TABLE suppression_rules (
    suppression_id VARCHAR(32) PRIMARY KEY,
    rule_name VARCHAR(128) NOT NULL,
    suppression_class VARCHAR(16) NOT NULL, -- S1_KNOWN_GOOD,
S2_DEDUPLICATION, S3_AGGREGATION, S4_MAINTENANCE, S5_CONDITIONAL
    owner VARCHAR(128) NOT NULL,
    reason TEXT NOT NULL,
    scope_definition JSONB NOT NULL, -- Defines source, destination, executable, account,
event_type constraints
    status VARCHAR(16) DEFAULT 'ACTIVE', -- ACTIVE, EXPIRED, REVOKED
    created_date DATE NOT NULL,
    expires_date DATE NOT NULL,
    review_cycle VARCHAR(16) DEFAULT 'MONTHLY',
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE suppression_audit_log (
    log_id SERIAL PRIMARY KEY,
    suppression_id VARCHAR(32) REFERENCES suppression_rules(suppression_id),
    action VARCHAR(32) NOT NULL, -- CREATED, REVIEWED, EXPIRED, MODIFIED
    performed_by VARCHAR(128) NOT NULL,
    notes TEXT,
    timestamp TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

-----**3. Aggregated & Deduplicated Alerts**

Collapses repetitive event bursts (e.g., 20,000 failed logins) into single actionable objects while retaining links to raw evidence.

```
CREATE TABLE aggregated_alerts (
    aggregation_id VARCHAR(64) PRIMARY KEY,
    rule_id INT NOT NULL,
    deduplication_key VARCHAR(128) NOT NULL, -- Hash of rule_id + src_ip + dst_ip +
username + event_type
    source_ip VARCHAR(45),
    destination_ip VARCHAR(45),
    username VARCHAR(128),
    event_count INT NOT NULL DEFAULT 1,
    window_start TIMESTAMP WITH TIME ZONE NOT NULL,
    window_end TIMESTAMP WITH TIME ZONE NOT NULL,
    status VARCHAR(32) DEFAULT 'OPEN', -- OPEN, SUPPRESSED, ESCALATED
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
```

```
);

-- Junction table linking aggregated alerts back to raw evidence
CREATE TABLE aggregate_evidence_mapping (
    aggregation_id VARCHAR(64) REFERENCES aggregated_alerts(aggregation_id),
    raw_event_id VARCHAR(64) REFERENCES raw_events(event_id),
    PRIMARY KEY (aggregation_id, raw_event_id)
);
```

-----**4. Correlated Incidents & Attack Chains**

Tracks multi-stage behavioral sequences (e.g., Brute Force $\rightarrow$ Successful Login $\rightarrow$ Privilege Escalation $\rightarrow$ Persistence $\rightarrow$ C2).

```
CREATE TABLE correlation_rules (
    correlation_rule_id VARCHAR(32) PRIMARY KEY,
    rule_name VARCHAR(128) NOT NULL,
    description TEXT,
    timeframe_seconds INT NOT NULL,
    target_severity VARCHAR(16) NOT NULL -- P0, P1, P2, P3
);

CREATE TABLE correlated_incidents (
    incident_id VARCHAR(64) PRIMARY KEY,
    correlation_rule_id VARCHAR(32) REFERENCES correlation_rules(correlation_rule_id),
    title VARCHAR(255) NOT NULL,
    narrative TEXT NOT NULL,
    initial_access_vector VARCHAR(128),
    compromised_account VARCHAR(128),
    target_host VARCHAR(128),
    status VARCHAR(32) DEFAULT 'DETECTED',
    detected_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Links individual normalized/aggregated alerts into the broader attack chain sequence
CREATE TABLE incident_stages (
    stage_id SERIAL PRIMARY KEY,
    incident_id VARCHAR(64) REFERENCES correlated_incidents(incident_id),
    sequence_order INT NOT NULL,
    stage_name VARCHAR(64) NOT NULL, -- e.g., INITIAL_ACCESS,
PRIVILEGE_ESCALATION, PERSISTENCE, C2
    reference_id VARCHAR(64) NOT NULL, -- Points to aggregated_alert_id or
normalized_event_id
    recorded_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

-----**5. Risk Scoring & Severity Engine**

Calculates dynamic risk scores above native Wazuh severities based on context enrichment.

```
CREATE TABLE risk_scoring_factors (
    factor_id SERIAL PRIMARY KEY,
    incident_id VARCHAR(64) REFERENCES correlated_incidents(incident_id),
    factor_type VARCHAR(64) NOT NULL, -- DETECTION_SEVERITY, ASSET_CRITICALITY,
IDENTITY_RISK, THREAT_INTELLIGENCE, BEHAVIORAL_ANOMALY
    weight_added INT NOT NULL,
    description TEXT,
    evaluated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE calculated_risk_scores (
    score_id SERIAL PRIMARY KEY,
    incident_id VARCHAR(64) REFERENCES correlated_incidents(incident_id),
    total_score INT CHECK (total_score BETWEEN 0 AND 100),
    soc_priority VARCHAR(4) NOT NULL, -- P0, P1, P2, P3, P4
    calculated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

-----**6. SOC Incidents & Analyst Workflow**

Manages operational triage, true/false positive outcomes, and analyst workload.

```
CREATE TABLE soc_triage_queue (
    triage_id VARCHAR(64) PRIMARY KEY,
    incident_id VARCHAR(64) REFERENCES correlated_incidents(incident_id),
    assigned_analyst VARCHAR(128),
    priority VARCHAR(4) NOT NULL, -- P0 to P4
    triage_status VARCHAR(32) DEFAULT 'NEW', -- NEW, INVESTIGATING, RESOLVED,
CLOSED
    resolution_outcome VARCHAR(32), -- TRUE_POSITIVE, FALSE_POSITIVE,
SUPPRESSED_NOISE
    false_positive_reason TEXT,
    mean_time_to_triage_seconds INT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    resolved_at TIMESTAMP WITH TIME ZONE ZONE
);
```

-----**7. System Metrics & KPIs**

*Tracks operational KPIs including the critical **suppression escape rate**.*

```
CREATE TABLE operational_metrics_daily (
    metric_date DATE PRIMARY KEY,
    raw_events_count BIGINT DEFAULT 0,
    wazuh_alerts_count BIGINT DEFAULT 0,
    suppressed_events_count BIGINT DEFAULT 0,
```

```
    aggregated_alerts_count BIGINT DEFAULT 0,
    correlated_incidents_count BIGINT DEFAULT 0,
    analyst_reviewed_count BIGINT DEFAULT 0,
    true_positives_count BIGINT DEFAULT 0,
    actionable_incidents_count BIGINT DEFAULT 0,
    suppression_escapes_count BIGINT DEFAULT 0, -- Suppressed events later tied to
confirmed true positives
    avg_mean_time_to_triage_sec FLOAT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

-----Would you like me to generate specific SQL indexing strategies to optimize query performance for high-volume Wazuh event ingestion?

DB query

Here is an optimized PostgreSQL indexing strategy and partitioning plan designed to handle high-volume event ingestion, rapid correlation lookups, and efficient SOC triage queries.-----**1. Table Partitioning Strategy**

High-volume telemetry tables (`raw_events` and `normalized_events`) will grow rapidly and degrade query performance unless partitioned by time.**Partitioning `raw_events` by Range (Monthly or Daily)**

```
-- Convert raw_events into a partitioned table
CREATE TABLE raw_events (
    event_id VARCHAR(64) NOT NULL,
    agent_id VARCHAR(64) NOT NULL,
    agent_name VARCHAR(128),
    timestamp TIMESTAMP WITH TIME ZONE NOT NULL,
    source_ip VARCHAR(45),
    destination_ip VARCHAR(45),
    source_port INT,
    destination_port INT,
    protocol VARCHAR(16),
    username VARCHAR(128),
    raw_payload TEXT NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (event_id, timestamp)
) PARTITION BY RANGE (timestamp);

-- Example monthly partition
CREATE TABLE raw_events_2026_08 PARTITION OF raw_events
    FOR VALUES FROM ('2026-08-01 00:00:00+00') TO ('2026-09-01 00:00:00+00');
```

Partitioning `normalized_events`

Partition correspondingly to maintain fast lookups during correlation windows:

```
CREATE TABLE normalized_events (
    normalized_event_id VARCHAR(64) NOT NULL,
    raw_event_id VARCHAR(64) NOT NULL,
    rule_id INT NOT NULL,
    rule_level INT NOT NULL,
    classification_groups TEXT[],
    mitre_attack_ids VARCHAR(16)[],
    description TEXT,
    processed_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (normalized_event_id, processed_at)
) PARTITION BY RANGE (processed_at);

CREATE TABLE normalized_events_2026_08 PARTITION OF normalized_events
    FOR VALUES FROM ('2026-08-01 00:00:00+00') TO ('2026-09-01 00:00:00+00');
```

-----**2. Indexing Strategy**

To support rapid filtering, alert aggregation, and analyst workflows, apply the following indexes:**Telemetry & Aggregation Indexes**

- **Time-range and Agent Filtering:** B-tree index on `timestamp` / `processed_at` combined with `agent_id`.
- **Deduplication Lookups:** Unique or B-tree index on `aggregated_alerts(deduplication_key, window_end)` to quickly check if an incoming alert fits an active deduplication window.

-- Fast deduplication checks
CREATE INDEX idx_aggregated_alerts_dedup ON aggregated_alerts (deduplication_key, status, window_end);

-- Evidence mapping lookups
CREATE INDEX idx_aggregate_evidence_mapping_raw ON aggregate_evidence_mapping (raw_event_id);
Array and JSONB Indexes (GIN)

Since classification groups, MITRE ATT&CK IDs, and suppression scopes use arrays and JSONB, GIN indexes prevent sequential scans during threat hunting and rule matching.
-- Fast lookup for specific MITRE techniques or classification tags
CREATE INDEX idx_normalized_events_mitre ON normalized_events USING GIN (mitre_attack_ids);
CREATE INDEX idx_normalized_events_groups ON normalized_events USING GIN (classification_groups);

-- Fast lookup for suppression rule scopes
CREATE INDEX idx_suppression_rules_scope ON suppression_rules USING GIN (scope_definition);
Analyst Workflow & Triage Indexes

Optimize the SOC queue for real-time dashboard loading and analyst assignment queries.
-- Fast queue filtering by status and priority
CREATE INDEX idx_soc_triage_queue_status_priority ON soc_triage_queue (triage_status, priority, created_at);

-- Fast lookup for analyst workloads
CREATE INDEX idx_soc_triage_queue_analyst ON soc_triage_queue (assigned_analyst) WHERE assigned_analyst IS NOT NULL;
-----**3. Maintenance & Retention Policy**

- **Automated Partition Management:** Use `pg_partman` to automatically create future telemetry partitions and drop old partitions based on your log retention policy (e.g., retain raw telemetry for 90 days, normalized events for 180 days, and incident/governance records indefinitely).
- **Vacuum Tuning:** Adjust `autovacuum_vacuum_scale_factor` and `autovacuum_analyze_scale_factor` to more aggressive thresholds on high-churn tables like `soc_triage_queue` and `aggregated_alerts` to prevent table bloat during high-volume attack bursts.

Would you like me to provide automated maintenance scripts and retention partitioning functions for this PostgreSQL setup?

DB mgt and maint

```
-- ========================================================================
-- AUTOMATED PARTITION MANAGEMENT & RETENTION FUNCTIONS (PostgreSQL)
-- ========================================================================

-- 1. Function to automatically create monthly partitions for raw_events
CREATE OR REPLACE PROCEDURE create_monthly_raw_events_partition(target_date
DATE)
AS $$
DECLARE
    start_time TIMESTAMP WITH TIME ZONE;
    end_time TIMESTAMP WITH TIME ZONE;
    partition_name TEXT;
BEGIN
    -- Calculate start and end of the target month
    start_time := DATE_TRUNC('month', target_date);
    end_time := start_time + INTERVAL '1 month';

    -- Construct partition table name, e.g., raw_events_2026_09
    partition_name := 'raw_events_' || TO_CHAR(start_time, 'YYYY_mi_MM');
    partition_name := REPLACE(partition_name, '_mi_', '_'); -- Clean format YYYY_MM

    -- Dynamically create the partition if it doesn't already exist
    EXECUTE FORMAT(
        'CREATE TABLE IF NOT EXISTS %I PARTITION OF raw_events FOR VALUES FROM
(%L) TO (%L);',
        partition_name, start_time, end_time
    );
END;
$$ LANGUAGE plpgsql;


-- 2. Function to automatically drop raw_event partitions older than retention period (e.g., 90
days)
CREATE OR REPLACE PROCEDURE drop_expired_raw_events_partitions(retention_days
INT)
AS $$
DECLARE
    partition_record RECORD;
    cutoff_date DATE;
    partition_date DATE;
    table_name TEXT;
BEGIN
    cutoff_date := CURRENT_DATE - INTERVAL '1 day' * retention_days;
```

```
    -- Iterate through child partitions of raw_events
    FOR partition_record IN
        ISelector:
        SELECT inhrelid::regclass AS child_table
        FROM pg_inherits
        JOIN pg_class parent ON pg_inherits.inhparent = parent.oid
        WHERE parent.relname = 'raw_events'
    LOOP
        table_name := partition_record.child_table::TEXT;

        -- Extract date from table name suffix (assumes format raw_events_YYYY_MM)
        BEGIN
            partition_date := TO_DATE(SUBSTRING(table_name FROM
'raw_events_([0-9]{4}_[0-9]{2})'), 'YYYY_MM');

            -- If partition month is older than cutoff, detach and drop it
            IF partition_date < DATE_TRUNC('month', cutoff_date) THEN
                EXECUTE FORMAT('ALTER TABLE raw_events DETACH PARTITION %I;',
table_name);
                EXECUTE FORMAT('DROP TABLE %I;', table_name);
            END IF;
        EXCEPTION WHEN OTHERS THEN
            -- Skip tables that don't match naming convention
            CONTINUE;
        END;
    END LOOP;
END;
$$ LANGUAGE plpgsql;


-- ================================================================
-- AUTOMATED MAINTENANCE SCHEDULING (Using pg_cron extension)
-- ================================================================

-- Schedule partition creation for the upcoming month (runs on the 25th of every month)
-- SELECT cron.schedule('create-monthly-partitions', '0 0 25 * *',
--     $$CALL create_monthly_raw_events_partition(CURRENT_DATE + INTERVAL '1
month');$$);

-- Schedule cleanup of raw telemetry older than 90 days (runs daily at midnight)
-- SELECT cron.schedule('cleanup-old-raw-events', '0 0 * * *',
--     $$CALL drop_expired_raw_events_partitions(90);$$);
```

Would you like me to provide a dashboard query template for tracking key SOC metrics like alert reduction ratios and suppression escape rates?

Dashboard query

OpenSearch / Elasticsearch query templates for tracking key SOC metrics in a Wazuh/ELK stack:1. Alert Reduction Funnel Metrics

- **Raw Ingested Events (Total Volume):**
  rule.level: [0 TO 16]
- **Wazuh Alerts (Post-Normalization & Filtering):**
  rule.level: [1 TO 16] AND NOT group: "alert_fatigue_control"
- **Actionable Incidents (P0–P2 Priority):**
  rule.level: [12 TO 16] OR group: "priority_p0" OR group: "priority_p1"

2. Suppression Volume & Escape Rates

- **Total Suppressed Events:**
  group: "network_suppression" OR group: "persistence_suppression" OR tags: "suppressed"
- **Suppression Escapes (Suppressed sources, users, or entities later correlated with a confirmed P0/P1 incident):**
  (_exists_:suppression_id) AND (rule.level: [14 TO 16] OR group: "priority_p0" OR group: "priority_p1")

3. Aggregation & Noise Control Ratios

- **Aggregated Burst Events:**
  group: "alert_aggregation" OR group: "auth_aggregate" OR group: "connection_burst"
- **Behavioral Correlation Chains:**
  group: "attack_chain" OR group: "correlated_detection" OR group: "behavior_chain"

-----Would you like me to help draft the accompanying governance structure for tracking suppression rule ownership and expiration dates?

Governance

Suppression Rule Governance Framework1. Suppression Metadata Schema

Every suppression rule or exception must include standard metadata to ensure accountability and prevent permanent, forgotten exemptions.

- **Exception ID:** Unique identifier (e.g., `EX-0042`)
- **Owner / Team:** Responsible individual or engineering group (e.g., `Security Engineering / SecOps-Tuning`)
- **Business Justification:** Documented rationale explaining why the activity is benign (e.g., `Approved vulnerability scanner performing weekly internal assessment`)
- **Scope:** Precise matching keys to avoid over-suppression (e.g., `source = 10.10.10.20 AND destination_port = 445 AND event_type = vulnerability_scan`)
- **Creation Date:** Timestamp when the exception was introduced
- **Expiration Date:** Hard expiration date (maximum 30–90 days for operational exceptions)
- **Review Cadence:** Frequency of mandatory review (e.g., `Monthly` or `Quarterly`)

-----2. Lifecycle Workflow

1. **Request & Submission:**
    - A ticket is opened detailing the noisy rule ID, the false-positive volume, and the proposed scope.
2. **Risk Assessment & Narrowing:**
    - Security Engineering reviews the rule to ensure it suppresses *events* or specific narrow keys rather than disabling detection rules entirely, adhering to the principle of "suppress events, not evidence."
3. **Deployment & Expiration Tagging:**
    - The exception is deployed with a mandatory expiration tag. If the expiration date is reached without renewal, the suppression rule automatically deactivates or triggers a high-priority review alert.
4. **Recertification:**
    - Owners receive automated reminders 14 days prior to expiration to re-verify whether the business justification is still valid.

-----3. Audit & Metric Tracking

- **Suppression Escape Rate KPI:**
  Track suppressed events that later correlate with confirmed high-severity incidents to measure tuning safety:
  $$\frac{\text{Suppressed events later associated with confirmed incidents}}{\text{Total suppressed events}}$$

- **Stale Exception Audit:**
  A monthly automated report listing all suppression rules approaching expiration or lacking a valid owner.

Would you like me to help draft a Wazuh CDB list template for managing authorized vulnerability scanners and maintenance windows?

CDB list for vul scan

```
# File: /var/ossec/etc/lists/approved-vulnerability-scanners
# Format: key:value (where key is the source IP/CIDR or hostname, and value is the asset owner/ticket reference)

10.10.10.20:SecOps-VulnerabilityManagement-INC0042
10.10.10.21:SecOps-VulnerabilityManagement-INC0042
192.168.100.50:ExternalVendor-Tenable-CRQ1092
# File: /var/ossec/etc/lists/approved-maintenance-windows
# Format: key:value (where key is the asset IP or hostname, and value is the approved maintenance ticket/window ID)

server-db-01.corp.internal:CRQ-2026-Q3-PatchWindow
server-app-02.corp.internal:CRQ-2026-Q3-PatchWindow
10.20.30.40:CRQ-2026-EmergencyHotfix
```

Configuration Registration (`ossec.conf`)

To enable Wazuh to consume these CDB lists in your custom rules, register them inside the `<ruleset>` block on the Wazuh manager (`/var/ossec/etc/ossec.conf`):

```
<ruleset>
  <!-- Existing ruleset configurations -->
  <decoder_dir>ruleset/decoders</decoder_dir>
  <rule_dir>ruleset/rules</rule_dir>
  <rule_dir>etc/rules</rule_dir>

  <!-- Custom CDB Lists for Suppression & Governance -->
  <list>etc/lists/approved-vulnerability-scanners</list>
  <list>etc/lists/approved-maintenance-windows</list>
</ruleset>
```

Example Suppression Rule Integration (`080-suppression.xml`)

```
<group name="suppression,governance,">

  <rule id="100800" level="0">
    <if_sid>100320,100330</if_sid>
    <list field="srcip"
lookup="address_match_key">etc/lists/approved-vulnerability-scanners</list>
    <description>SUPPRESSION: Expected activity from approved vulnerability
scanner</description>
    <options>no_log</options>
    <group>network_suppression,approved_scanner,</group>
  </rule>

  <rule id="100801" level="0">
    <if_group>persistence,windows_service</if_group>
```

```
    <list field="win.system.computer"
lookup="address_match_key">etc/lists/approved-maintenance-windows</list>
    <description>SUPPRESSION: Expected service change during approved maintenance
window</description>
    <options>no_log</options>
    <group>maintenance_suppression,</group>
  </rule>

</group>
```

Would you like me to help draft the automated log rotation and backup script for your custom Wazuh rules and CDB lists?

Auto log rotate and backup script

```
<!-- File: /var/ossec/etc/ossec.conf -->
<!-- Add these sections to your Wazuh manager / agent configuration for FIM and Windows event collection -->

<ossec_config>
  <!-- Linux FIM Configuration (monitoring critical persistence paths) -->
  <syscheck>
    <frequency>43200</frequency>
    <scan_on_start>yes</scan_on_start>

    <!-- Critical SSH authorized keys -->
    <directories check_all="yes">/root/.ssh/authorized_keys</directories>
    <directories check_all="yes">/home/*/.ssh/authorized_keys</directories>

    <!-- System and user cron -->
    <directories check_all="yes">/etc/crontab</directories>
    <directories check_all="yes">/etc/cron.d</directories>
    <directories check_all="yes">/var/spool/cron</directories>

    <!-- Systemd services and timers -->
    <directories check_all="yes">/etc/systemd/system</directories>

    <!-- Dynamic linker preload -->
    <directories check_all="yes">/etc/ld.so.preload</directories>

    <!-- Critical system files -->
    <directories check_all="yes">/etc/passwd</directories>
    <directories check_all="yes">/etc/shadow</directories>
    <directories check_all="yes">/etc/sudoers</directories>
    <directories check_all="yes">/etc/sudoers.d</directories>
  </syscheck>

  <!-- Windows Event Channel Collection (Sysmon & Security) -->
  <localfile>
    <location>Microsoft-Windows-Sysmon/Operational</location>
    <log_format>eventchannel</log_format>
  </localfile>

  <localfile>
    <location>Security</location>
    <log_format>eventchannel</log_format>
  </localfile>

  <localfile>
```

```
    <location>System</location>
    <log_format>eventchannel</log_format>
  </localfile>
</ossec_config>
```

Validation Test Cases for `wazuh-logtest`

Run `/var/ossec/bin/wazuh-logtest` on the Wazuh manager and paste the following sample logs to verify rule triggering:

1. **Root SSH Key Modification (FIM Rule 100301 / Level 12)**:
   {"agent": {"id": "001", "name": "linux-server-01"}, "manager": {"name": "wazuh-manager"}, "syscheck": {"event": "modified", "path": "/root/.ssh/authorized_keys", "uname_after": "root"}}
2. **Windows Service Creation (Rule 100320 / Level 9)**:
   <Event xmlns="http://schemas.microsoft.com/win/2004/08/events/event"><System><Provider Name="Microsoft-Windows-Service Control Manager" Guid="{555908d1-a6d7-4695-8e1e-269fb3d2f021}"/><EventID>7045</EventID><Version>0</Version><Level>4</Level><Task>0</Task><Opcode>0</Opcode><Keywords>0x8080000000000000</Keywords><TimeCreated SystemTime="2026-08-27T03:37:51.0000000Z"/><EventRecordID>1042</EventRecordID><Correlation/><Execution ProcessID="512" ThreadID="644"/><Channel>System</Channel><Computer>win-workstation-01</Computer><Security UserID="S-1-5-18"/></System><EventData><Data Name="ServiceName">MaliciousService</Data><Data Name="ImagePathName">C:\Users\Admin\AppData\Local\Temp\payload.exe</Data><Data Name="ServiceType">user mode service</Data><Data Name="StartType">auto start</Data><Data Name="AccountName">LocalSystem</Data></EventData></Event>
3. **Registry Run Key Modification via Sysmon (Rule 100340 / Level 9)**:
   <Event xmlns="http://schemas.microsoft.com/win/2004/08/events/event"><System><Provider Name="Microsoft-Windows-Sysmon" Guid="{5770385f-c22a-43e0-bf4c-06f569ffffbd}"/><EventID>13</EventID><Version>2</Version><Level>4</Level><Task>13</Task><Opcode>0</Opcode><Keywords>0x8000000000000000</Keywords><TimeCreated SystemTime="2026-08-27T03:37:51.0000000Z"/><EventRecordID>5021</EventRecordID><Correlation/><Execution ProcessID="1840" ThreadID="2412"/><Channel>Microsoft-Windows-Sysmon/Operational</Channel><Computer>win-workstation-01</Computer><Security UserID="S-1-5-18"/></System><EventData><Data Name="RuleName">Persistence Run Key</Data><Data Name="EventType">SetValue</Data><Data Name="UtcTime">2026-08-27 03:37:51.123</Data><Data Name="ProcessGuid">{a1b2c3d4-0000-1111-2222-333344445555}</Data><Data Name="ProcessId">4320</Data><Data

Name="Image">C:\Windows\System32\reg.exe</Data><Data
Name="TargetObject">HKLM\Software\Microsoft\Windows\CurrentVersion\Run\Backdoo
r</Data><Data
Name="Details">C:\Users\Admin\AppData\Local\Temp\evil.exe</Data></EventData></E
vent>

Would you like me to outline the next file in the custom ruleset sequence for process-to-network correlation?

Dashboard Design ALT (REACT)

```jsx
import React, { useState } from 'react';
import { AreaChart, Area, XAxis, YAxis, CartesianGrid, Tooltip,
ResponsiveContainer, PieChart, Pie, Cell, Legend } from 'recharts';
import { LayoutDashboard, Filter, ShieldAlert, Settings, CheckCircle,
Clock, XCircle, ArrowDown, Shield, Activity, FileText } from
'lucide-react';

// Mock Data based on the "Final Target State" and Phase 1-2 validation
metrics
const dailyMetrics = [
  { date: 'Aug 21', raw: 5100000, wazuh_alerts: 52000, suppressed: 36000,
aggregated: 15000, analyst_visible: 1350, correlated: 160 },
  { date: 'Aug 22', raw: 4900000, wazuh_alerts: 49000, suppressed: 34000,
aggregated: 14000, analyst_visible: 1200, correlated: 145 },
  { date: 'Aug 23', raw: 5300000, wazuh_alerts: 55000, suppressed: 38000,
aggregated: 16000, analyst_visible: 1400, correlated: 170 },
  { date: 'Aug 24', raw: 4800000, wazuh_alerts: 48000, suppressed: 33000,
aggregated: 13500, analyst_visible: 1100, correlated: 130 },
  { date: 'Aug 25', raw: 5000000, wazuh_alerts: 50000, suppressed: 35000,
aggregated: 14500, analyst_visible: 1250, correlated: 150 },
  { date: 'Aug 26', raw: 5200000, wazuh_alerts: 53000, suppressed: 37000,
aggregated: 15500, analyst_visible: 1300, correlated: 155 },
  { date: 'Aug 27', raw: 5150000, wazuh_alerts: 51000, suppressed: 36500,
aggregated: 14000, analyst_visible: 1200, correlated: 150 }
];

const priorityDistribution = [
  { name: 'P0 Critical', value: 12, color: '#ef4444' }, // Red
  { name: 'P1 High', value: 45, color: '#f97316' }, // Orange
  { name: 'P2 Medium', value: 93, color: '#eab308' }, // Yellow
  { name: 'P3 Low', value: 350, color: '#3b82f6' }, // Blue
  { name: 'P4 Info', value: 700, color: '#64748b' }, // Gray
];

// Based on Phase 2 & 3: Suppression Governance
const suppressionRules = [
  { id: 'SUP-2026-014', rule_name: 'Vuln Scanner Auth Failures', class:
'KNOWN_GOOD', owner: 'Infra Team', expires: '2026-09-30', status:
'ACTIVE', affected_sids: [5710, 5712], review_cycle: 'QUARTERLY' },
```

```
  { id: 'SUP-2026-042', rule_name: 'Backup Svc Account Access', class:
'EXPECTED_ADMIN', owner: 'Storage Ops', expires: '2026-08-28', status:
'EXPIRING_SOON', affected_sids: [554], review_cycle: 'MONTHLY' },
  { id: 'SUP-2026-051', rule_name: 'App Migration Maintenance', class:
'MAINTENANCE', owner: 'DevOps', expires: '2026-08-25', status: 'EXPIRED',
affected_sids: [100500], review_cycle: 'TEMPORARY' },
  { id: 'SUP-2026-055', rule_name: 'Approved Jumpbox SSH', class:
'KNOWN_GOOD', owner: 'SecOps', expires: '2026-12-31', status: 'ACTIVE',
affected_sids: [5715], review_cycle: 'YEARLY' },
  { id: 'SUP-2026-060', rule_name: 'Legacy App File Modifications', class:
'LOW_VALUE', owner: 'App Team B', expires: '2026-09-15', status: 'ACTIVE',
affected_sids: [550, 554], review_cycle: 'MONTHLY' }
];

// Based on Phase 4, 6 & 8: Correlated Incidents & Risk Scoring
const correlatedIncidents = [
  { id: 'INC-2026-8801', title: 'Suspected Account Compromise (Brute Force
→ Success)', priority: 'P1', risk_score: 75, target: 'dc-01
(CROWN_JEWEL)', attacker_ip: '192.168.45.10', events_aggregated: 1532,
status: 'INVESTIGATING', detected_at: '10 mins ago' },
  { id: 'INC-2026-8802', title: 'Suspicious PowerShell Chain + Network
Connect', priority: 'P0', risk_score: 95, target: 'app-srv-04 (HIGH)',
attacker_ip: '203.0.113.50', events_aggregated: 14, status: 'NEW',
detected_at: '22 mins ago' },
  { id: 'INC-2026-8795', title: 'Login → Privilege Escalation Sequence',
priority: 'P1', risk_score: 72, target: 'db-cluster-02 (CRITICAL)',
attacker_ip: '10.10.5.55', events_aggregated: 8, status: 'NEW',
detected_at: '1 hour ago' },
  { id: 'INC-2026-8790', title: 'Repetitive Auth Failures (Deduplicated
Burst)', priority: 'P3', risk_score: 35, target: 'web-front-01
(STANDARD)', attacker_ip: '45.33.22.11', events_aggregated: 4500, status:
'OPEN', detected_at: '3 hours ago' },
  { id: 'INC-2026-8782', title: 'Malware Detection (Quarantined)',
priority: 'P2', risk_score: 55, target: 'user-wkstn-402 (STANDARD)',
attacker_ip: 'N/A', events_aggregated: 1, status: 'RESOLVED', detected_at:
'5 hours ago' }
];

const StatCard = ({ title, value, subValue, trend, icon: Icon, colorClass
}) => {
```

```
  const colorMap = {
    blue: 'bg-blue-500 text-blue-400',
    emerald: 'bg-emerald-500 text-emerald-400',
    purple: 'bg-purple-500 text-purple-400',
    orange: 'bg-orange-500 text-orange-400',
  };

  const IconBgClass = colorMap[colorClass].split(' ')[0];
  const IconTextClass = colorMap[colorClass].split(' ')[1];

  return (
    <div className="bg-slate-800 p-6 rounded-xl border border-slate-700
shadow-lg relative overflow-hidden">
      <div className={`absolute top-0 right-0 w-24 h-24 ${IconBgClass}
opacity-5 rounded-bl-full`}></div>
      <div className="flex justify-between items-start relative z-10">
        <div>
          <p className="text-slate-400 text-sm font-medium
mb-1">{title}</p>
          <h3 className="text-3xl font-bold text-white mb-1">{value}</h3>
          <div className="flex items-center text-xs">
            {trend === 'down' ?
              <span className="text-emerald-400 flex
items-center"><ArrowDown className="w-3 h-3 mr-1" /> {subValue}</span> :
              <span className="text-slate-400">{subValue}</span>}
          </div>
        </div>
        <div className={`p-3 bg-slate-900 rounded-lg ${IconTextClass}
border border-slate-700`}>
          <Icon size={24} />
        </div>
      </div>
    </div>
  );
};

const StatusBadge = ({ status }) => {
  const styles = {
    ACTIVE: "bg-emerald-500/10 text-emerald-400 border-emerald-500/20",
    EXPIRING_SOON: "bg-amber-500/10 text-amber-400 border-amber-500/20",
```

```
    EXPIRED: "bg-rose-500/10 text-rose-400 border-rose-500/20",
    NEW: "bg-blue-500/10 text-blue-400 border-blue-500/20",
    INVESTIGATING: "bg-purple-500/10 text-purple-400
border-purple-500/20",
    OPEN: "bg-slate-500/10 text-slate-300 border-slate-500/20",
    RESOLVED: "bg-emerald-500/10 text-emerald-400 border-emerald-500/20",
  };

  const iconMap = {
    ACTIVE: <CheckCircle className="w-3 h-3 mr-1" />,
    EXPIRING_SOON: <Clock className="w-3 h-3 mr-1" />,
    EXPIRED: <XCircle className="w-3 h-3 mr-1" />
  };

  return (
    <span className={`px-2.5 py-1 rounded-full text-xs font-medium border
flex items-center w-max ${styles[status] || styles.OPEN}`}>
      {iconMap[status] || null}
      {status.replace('_', ' ')}
    </span>
  );
};

const PriorityBadge = ({ priority }) => {
  const colors = {
    P0: "bg-red-500 shadow-red-500/50",
    P1: "bg-orange-500 shadow-orange-500/50",
    P2: "bg-yellow-500 shadow-yellow-500/50",
    P3: "bg-blue-500 shadow-blue-500/50",
    P4: "bg-slate-400 shadow-slate-400/50"
  };
  return (
    <span className="flex items-center gap-2">
      <span className={`w-2.5 h-2.5 rounded-full
shadow-[0_0_8px_rgba(0,0,0,0.5)] ${colors[priority]}`}></span>
      <span className="text-sm font-bold text-slate-200">{priority}</span>
    </span>
  );
};
```

```
const PipelineOverview = () => (
  <div className="space-y-6 animate-in fade-in duration-500">
    {/* Metrics Funnel Row */}
    <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-4">
      <StatCard title="Wazuh Base Alerts" value="51,000" subValue="-8% vs
last week" trend="down" icon={Activity} colorClass="blue" />
      <StatCard title="Suppressed (Noise)" value="36,500" subValue="71.5%
of total volume" trend="neutral" icon={Filter} colorClass="emerald" />
      <StatCard title="Analyst Visible" value="1,200" subValue="-97.6%
overall reduction" trend="down" icon={Shield} colorClass="purple" />
      <StatCard title="Correlated Incidents" value="150"
subValue="Prioritized attack chains" trend="neutral" icon={ShieldAlert}
colorClass="orange" />
    </div>

    <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
      {/* Pipeline Funnel Chart */}
      <div className="col-span-2 bg-slate-800 p-6 rounded-xl border
border-slate-700 shadow-lg">
        <div className="flex justify-between items-center mb-6">
          <h3 className="text-lg font-semibold text-white">Alert Pipeline
Reduction (7 Days)</h3>
          <span className="text-xs text-slate-400 bg-slate-900 px-2 py-1
rounded border border-slate-700">Linear Scale</span>
        </div>
        <div className="h-72">
          <ResponsiveContainer width="100%" height="100%">
            <AreaChart data={dailyMetrics} margin={{ top: 10, right: 10,
left: -20, bottom: 0 }}>
              <defs>
                <linearGradient id="colorWazuh" x1="0" y1="0" x2="0"
y2="1">
                  <stop offset="5%" stopColor="#3b82f6"
stopOpacity={0.3}/>
                  <stop offset="95%" stopColor="#3b82f6" stopOpacity={0}/>
                </linearGradient>
                <linearGradient id="colorVisible" x1="0" y1="0" x2="0"
y2="1">
                  <stop offset="5%" stopColor="#8b5cf6"
stopOpacity={0.8}/>
```

```
                  <stop offset="95%" stopColor="#8b5cf6"
stopOpacity={0.2}/>
                </linearGradient>
              </defs>
              <CartesianGrid strokeDasharray="3 3" stroke="#334155"
vertical={false} />
              <XAxis dataKey="date" stroke="#94a3b8" fontSize={12}
tickLine={false} axisLine={false} />
              <YAxis stroke="#94a3b8" fontSize={12} tickLine={false}
axisLine={false} tickFormatter={(val) => `${val/1000}k`} />
              <Tooltip
                contentStyle={{ backgroundColor: '#1e293b', borderColor:
'#334155', color: '#f8fafc', borderRadius: '8px' }}
                itemStyle={{ color: '#e2e8f0' }}
              />
              <Area type="monotone" dataKey="wazuh_alerts" name="Base
Alerts" stroke="#3b82f6" fillOpacity={1} fill="url(#colorWazuh)"
strokeWidth={2} />
              <Area type="monotone" dataKey="analyst_visible"
name="Analyst Visible" stroke="#8b5cf6" fillOpacity={1}
fill="url(#colorVisible)" strokeWidth={2} />
            </AreaChart>
          </ResponsiveContainer>
        </div>
      </div>

      {/* Risk Scoring Distribution */}
      <div className="col-span-1 bg-slate-800 p-6 rounded-xl border
border-slate-700 shadow-lg">
        <h3 className="text-lg font-semibold text-white mb-2">Priority
Distribution</h3>
        <p className="text-sm text-slate-400 mb-4">Dynamically scored by
enrichment engine</p>
        <div className="h-64">
          <ResponsiveContainer width="100%" height="100%">
            <PieChart>
              <Pie
                data={priorityDistribution}
                innerRadius={60}
                outerRadius={80}
```

```
                paddingAngle={5}
                dataKey="value"
                stroke="none"
              >
                {priorityDistribution.map((entry, index) => (
                  <Cell key={`cell-${index}`} fill={entry.color} />
                ))}
              </Pie>
              <Tooltip
                contentStyle={{ backgroundColor: '#1e293b', borderColor:
'#334155', borderRadius: '8px', color: '#fff' }}
                itemStyle={{ color: '#fff' }}
              />
              <Legend verticalAlign="bottom" height={36} iconType="circle"
wrapperStyle={{ fontSize: '12px', color: '#94a3b8' }} />
            </PieChart>
          </ResponsiveContainer>
        </div>
      </div>
    </div>
  </div>
);

const GovernanceView = () => (
  <div className="bg-slate-800 rounded-xl border border-slate-700
shadow-lg overflow-hidden animate-in fade-in duration-500">
    <div className="p-6 border-b border-slate-700 flex flex-col
sm:flex-row justify-between items-start sm:items-center gap-4">
      <div>
        <h3 className="text-lg font-semibold text-white flex items-center
gap-2">
          <Filter className="w-5 h-5 text-emerald-400" />
          Suppression & Allowlist Governance
        </h3>
        <p className="text-sm text-slate-400 mt-1">Manage active
exceptions, ownership, and review cycles (Phase 3).</p>
      </div>
      <button className="bg-blue-600 hover:bg-blue-500 text-white px-4
py-2 rounded-lg text-sm font-medium transition-colors shadow-lg
shadow-blue-500/20 whitespace-nowrap">
```

```
        + New Exception
      </button>
    </div>
    <div className="overflow-x-auto">
      <table className="w-full text-left border-collapse min-w-[800px]">
        <thead>
          <tr className="bg-slate-900/50 text-slate-400 text-xs uppercase
tracking-wider">
            <th className="p-4 font-medium">ID / Rule Name</th>
            <th className="p-4 font-medium">Class</th>
            <th className="p-4 font-medium">Owner</th>
            <th className="p-4 font-medium">Affected Rules</th>
            <th className="p-4 font-medium">Expiration</th>
            <th className="p-4 font-medium">Status</th>
          </tr>
        </thead>
        <tbody className="divide-y divide-slate-700/50">
          {suppressionRules.map((rule) => (
            <tr key={rule.id} className="hover:bg-slate-700/30
transition-colors">
              <td className="p-4">
                <div className="text-sm font-medium
text-white">{rule.rule_name}</div>
                <div className="text-xs text-slate-500 font-mono
mt-0.5">{rule.id}</div>
              </td>
              <td className="p-4 text-sm text-slate-300">
                <span className="bg-slate-900 px-2 py-1 rounded text-xs
border border-slate-700">
                  {rule.class.replace('_', ' ')}
                </span>
              </td>
              <td className="p-4 text-sm text-slate-300 flex items-center
gap-2">
                <div className="w-6 h-6 rounded-full bg-slate-700 flex
items-center justify-center text-xs font-bold text-slate-300">
                  {rule.owner.charAt(0)}
                </div>
                {rule.owner}
              </td>
```

```
              <td className="p-4">
                <div className="flex gap-1 flex-wrap">
                  {rule.affected_sids.map(sid => (
                    <span key={sid} className="bg-slate-900 text-slate-400
px-2 py-0.5 rounded text-xs border border-slate-700">
                      SID: {sid}
                    </span>
                  ))}
                </div>
              </td>
              <td className="p-4 text-sm text-slate-300">
                <div className="flex flex-col">
                  <span>{rule.expires}</span>
                  <span className="text-xs
text-slate-500">{rule.review_cycle} review</span>
                </div>
              </td>
              <td className="p-4"><StatusBadge status={rule.status}
/></td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  </div>
);

const IncidentsView = () => (
  <div className="bg-slate-800 rounded-xl border border-slate-700
shadow-lg overflow-hidden animate-in fade-in duration-500">
    <div className="p-6 border-b border-slate-700">
      <h3 className="text-lg font-semibold text-white flex items-center
gap-2">
        <ShieldAlert className="w-5 h-5 text-orange-400" />
        Correlated SOC Incidents
      </h3>
      <p className="text-sm text-slate-400 mt-1">Prioritized alerts
resulting from deduplication, enrichment, and multi-stage correlation
(Phases 4-7).</p>
    </div>
```

```
    <div className="overflow-x-auto">
      <table className="w-full text-left border-collapse min-w-[900px]">
        <thead>
          <tr className="bg-slate-900/50 text-slate-400 text-xs uppercase
tracking-wider">
            <th className="p-4 font-medium">Incident</th>
            <th className="p-4 font-medium">Priority & Score</th>
            <th className="p-4 font-medium">Target Asset</th>
            <th className="p-4 font-medium">Source IP</th>
            <th className="p-4 font-medium">Evt Count</th>
            <th className="p-4 font-medium">Status</th>
          </tr>
        </thead>
        <tbody className="divide-y divide-slate-700/50">
          {correlatedIncidents.map((inc) => (
            <tr key={inc.id} className="hover:bg-slate-700/30
transition-colors">
              <td className="p-4">
                <div className="text-sm font-medium
text-white">{inc.title}</div>
                <div className="text-xs text-slate-500 mt-1
font-mono">{inc.id} • Detected {inc.detected_at}</div>
              </td>
              <td className="p-4">
                <PriorityBadge priority={inc.priority} />
                <div className="text-xs text-slate-400 mt-2 font-medium">
                  Risk Score: <span className={inc.risk_score > 80 ?
'text-red-400' : 'text-orange-400'}>{inc.risk_score}/100</span>
                </div>
              </td>
              <td className="p-4">
                <div className="text-sm text-slate-200 font-mono">
                  {inc.target.split(' ')[0]}
                </div>
                <div className="text-[10px] uppercase tracking-wider
text-amber-500 mt-1 font-bold">
                  {inc.target.split(' ')[1].replace(/[()]/g, '')}
                </div>
              </td>
```

```
              <td className="p-4 text-sm text-slate-300
font-mono">{inc.attacker_ip}</td>
              <td className="p-4 text-sm text-slate-300">
                <span className="bg-slate-900 px-2.5 py-1 rounded-md
text-xs border border-slate-700 font-mono">
                  {inc.events_aggregated.toLocaleString()}
                </span>
              </td>
              <td className="p-4"><StatusBadge status={inc.status} /></td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  </div>
);

export default function App() {
  const [activeTab, setActiveTab] = useState('overview');

  const navItems = [
    { id: 'overview', label: 'Pipeline Overview', icon: LayoutDashboard },
    { id: 'governance', label: 'Suppression Auth', icon: Filter },
    { id: 'incidents', label: 'Correlated Incidents', icon: ShieldAlert },
    { id: 'settings', label: 'Scoring Engine', icon: Settings },
  ];

  return (
    <div className="min-h-screen flex flex-col md:flex-row bg-slate-900
text-slate-200 font-sans selection:bg-blue-500/30">
      {/* Sidebar */}
      <aside className="w-full md:w-64 bg-slate-950 border-r
border-slate-800 flex flex-col shrink-0 relative z-20">
        <div className="p-6 border-b border-slate-800">
          <div className="flex items-center gap-2 mb-1">
            <Shield className="w-6 h-6 text-blue-500" />
            <h1 className="text-xl font-bold text-white
tracking-tight">Wazuh SOC</h1>
          </div>
```

```
          <p className="text-xs text-slate-400 font-medium tracking-wide
uppercase">Pipeline v2.0</p>
        </div>
        <nav className="flex-1 p-4 space-y-1.5">
          {navItems.map(item => (
            <button
              key={item.id}
              onClick={() => setActiveTab(item.id)}
              className={`w-full flex items-center gap-3 px-4 py-3
rounded-lg text-sm font-medium transition-all duration-200 ${
                activeTab === item.id
                ? 'bg-blue-600/10 text-blue-400 border border-blue-500/20
shadow-[inset_0_0_12px_rgba(59,130,246,0.1)]'
                : 'text-slate-400 hover:bg-slate-800/50
hover:text-slate-200 border border-transparent'
              }`}
            >
              <item.icon size={18} className={activeTab === item.id ?
'text-blue-400' : 'text-slate-500'} />
              {item.label}
            </button>
          ))}
        </nav>
        <div className="p-4 border-t border-slate-800 bg-slate-900/50">
          <div className="flex items-center gap-3 px-2 py-2">
            <div className="relative flex h-3 w-3">
              <span className="animate-ping absolute inline-flex h-full
w-full rounded-full bg-emerald-400 opacity-75"></span>
              <span className="relative inline-flex rounded-full h-3 w-3
bg-emerald-500"></span>
            </div>
            <div>
              <p className="text-xs font-medium text-slate-300">Pipeline
Active</p>
              <p className="text-[10px] text-slate-500">Processing 5.1M
events/day</p>
            </div>
          </div>
        </div>
      </aside>
```

```
      {/* Main Content */}
      <main className="flex-1 flex flex-col h-screen overflow-hidden
relative">
        {/* Header */}
        <header className="h-16 border-b border-slate-800 bg-slate-900/80
backdrop-blur-md flex items-center justify-between px-8 shrink-0 z-10">
          <h2 className="text-lg font-semibold text-white flex
items-center gap-2">
            {navItems.find(i => i.id === activeTab)?.label}
          </h2>
          <div className="flex items-center gap-4">
            <span className="text-xs font-medium text-slate-400
bg-slate-800 px-3 py-1.5 rounded-md border border-slate-700 shadow-inner">
              Last 7 Days
            </span>
            <div className="w-8 h-8 rounded-full bg-gradient-to-tr
from-blue-600 to-indigo-600 flex items-center justify-center text-sm
font-bold text-white shadow-lg shadow-blue-500/20 ring-2 ring-slate-800
cursor-pointer">
              JD
            </div>
          </div>
        </header>

        {/* Scrollable Dashboard Area */}
        <div className="flex-1 overflow-auto p-4 md:p-8 relative
custom-scrollbar">
          {/* Decorative background blurs */}
          <div className="absolute top-0 left-1/4 w-96 h-96 bg-blue-500/5
rounded-full blur-3xl pointer-events-none mix-blend-screen"></div>
          <div className="absolute bottom-1/4 right-1/4 w-96 h-96
bg-purple-500/5 rounded-full blur-3xl pointer-events-none
mix-blend-screen"></div>

          <div className="max-w-7xl mx-auto relative z-10">
            {activeTab === 'overview' && <PipelineOverview />}
            {activeTab === 'governance' && <GovernanceView />}
            {activeTab === 'incidents' && <IncidentsView />}
            {activeTab === 'settings' && (
```

```
            <div className="flex flex-col items-center justify-center
h-96 text-slate-500 animate-in fade-in zoom-in-95 duration-500">
              <div className="p-6 bg-slate-800 rounded-full mb-6 border
border-slate-700 shadow-xl">
                <Settings size={48} className="text-slate-400" />
              </div>
              <h3 className="text-xl font-semibold text-white
mb-2">Scoring Engine Config</h3>
              <p className="text-sm max-w-md text-center">
                Configuration UI for Phase 7 (Dynamic Risk Weights,
Asset Criticality multipliers, and Identity Context maps) will be deployed
in the next sprint.
              </p>
            </div>
          )}
        </div>
      </div>
    </main>

    {/* Custom Scrollbar Styles injected globally */}
    <style dangerouslySetInnerHTML={{__html: `
      .custom-scrollbar::-webkit-scrollbar { width: 8px; height: 8px; }
      .custom-scrollbar::-webkit-scrollbar-track { background:
transparent; }
      .custom-scrollbar::-webkit-scrollbar-thumb { background: #334155;
border-radius: 4px; }
      .custom-scrollbar::-webkit-scrollbar-thumb:hover { background:
#475569; }
    `}} />
  </div>
 );
}
```